

ABSTRACT

SINGH, ROHIT. Revisiting Squashed-Branch Reuse and Dynamic Predication for Reducing the Branch Misprediction Bottleneck in Superscalar Processors. (Under the direction of Dr. Eric Rotenberg).

Branch prediction accuracy has a first-order effect on the performance of superscalar processors. After decades of advancements in branch prediction, it has now become increasingly difficult to further improve prediction accuracy using conventional means. Many important workloads still suffer from high branch misprediction rates (the problem), so there is value in exploring alternate solutions to the problem. This dissertation revisits two alternate solutions: squashed-branch reuse and dynamic predication.

Squashed-branch reuse (at fetch) can eliminate the entire misprediction penalty of a branch for which a squashed outcome is available (and correct). A major difficulty in squashed-branch reuse is aligning a branch on the resolved path with its corresponding instance on the squashed path if it exists.

In this dissertation we introduce a new squashed-branch reuse scheme which uses *hierarchical loop iteration identifiers*. Identifiers of dynamic instances of a branch are *unique* and *path-invariant*, and together these properties simplify branch alignment. We realize the identifier compactly using a *signature* that is augmented as loops are entered and continued, and a small stack that saves and restores the signature before entering and after exiting loops, respectively. The signature plus branch PC together form a key which identifies the dynamic branch being fetched. Dynamic branches use their key to deposit their outcome to, and retrieve an outcome from, the *Squashed-Branch Reuse Buffer* – a cache of branch outcomes.

Dynamic predication is guided by an analytical model which compares the fetch overhead of predication against the fetch overhead of conventional speculation, but the model ignores the impact on global branch history – a crucial side-effect of predication on instruction fetch. Further, predication can be applied to certain hard-to-predict loops, but

the existing scheme for loop predication doesn't yield any performance improvement.

We propose to disallow predication on *inseparable* branches, *i.e.*, branches which have a (substantial) negative effect on the accuracy of un-predicated branches. We detect and filter inseparable branches at compile time using an *iterative analysis*. We also revamp loop predication such that both a) the decision to predicate a loop, and b) the number of iterations to predicate for (if we decide to predicate the loop), are guided by the distribution of loop iterations in the previous n visits to the loop. This way predication subsumes many more loop branch mispredictions, and sharply diminishes the chances of loop under-iterations – a key issue with the prior scheme – while predicating a loop.

© Copyright 2026 by Rohit Singh

All Rights Reserved

Revisiting Squashed-Branch Reuse and Dynamic Predication for Reducing the Branch
Misprediction Bottleneck in Superscalar Processors

by
Rohit Singh

A dissertation submitted to the Graduate Faculty of
North Carolina State University
in partial fulfillment of the
requirements for the degree of
Doctor of Philosophy

Computer Engineering

Raleigh, North Carolina
2026

APPROVED BY:

Dr. Huiyang Zhou

Dr. James Tuck

Dr. Xipeng Shen

Dr. Eric Rotenberg
Chair of Advisory Committee

DEDICATION

This work is dedicated to my father, Indra Pal Singh, who was an ingenious engineer.

BIOGRAPHY

The author received his bachelor's degree in Electronics and Electrical Communication Engineering from IIT Kharagpur in 2013. Upon graduation, he worked at Cadence Design Systems (India) as a software engineer. At Cadence, he developed software for analyzing and visualizing electro-migration, IR drop, and parasitic resistance.

The author began his academic journey at NC State in 2019 as a master's student in Computer Engineering. He converted to a PhD program in 2020, and received an en-route master's degree in 2021. During his PhD, the author interned at ARM in 2021 and at Intel in 2022. For his initial work on control independence, the author received a fellowship, jointly with another student, from Qualcomm in 2024.

After completing his PhD, the author is expected to work at ARM in the United States as a microarchitecture exploration engineer.

ACKNOWLEDGEMENTS

I would like to thank my wife Niharika for encouraging me to pursue graduate school. I want to thank her for believing in me, and staying strong during the course of my PhD program. Her emotional support during my PhD journey cannot be overstated.

I would like to thank my advisor Dr. Eric Rotenberg for taking me under his wings, and letting me grow as a person and as a researcher. I would also like to thank Dr. Zhou, Dr. Tuck, and Dr. Shen, for agreeing to be on my doctoral committee, and providing valuable feedback on my research.

I want to thank Jiayang for enhancing the simulation infrastructure, and letting me use it in my research. I thank him for competing with me for the Qualcomm Innovation Fellowship (on two separate occasions), and for his contributions on squashed-branch reuse and dynamic predication.

I would like to thank my lab mates Zach, Anirudh, Debpratim, and Nhat for engaging discussions. I also want to thank my friends Mayank, Shehla, and Christina for their support beyond academia.

This work was supported, in part, by a Qualcomm Innovation Fellowship, and an Intel gift. I acknowledge the computing resources provided by the North Carolina State University High Performance Computing Shared Core Research Facility (RRID:SCR_-022168).

CONTENTS

List of Tables	viii
List of Figures	ix
Authorship Statement	xi
Chapter 1 Introduction	1
1.1 Branch mispredictions: an enduring performance problem	1
1.2 Control independence	3
1.3 Reuse control-independent data-independent instructions	5
1.3.1 General squashed-instruction reuse at rename	5
1.3.2 Squashed-branch reuse at fetch	6
1.3.3 Why revisit squashed-branch reuse at fetch	6
1.3.4 Contributions	8
1.4 Preserve control-independent instructions	9
1.4.1 Out-of-order fetch	10
1.4.2 Multi-path fetch (dynamic predication)	10
1.4.3 Fetch elimination (exact convergence)	11
1.4.4 Why revisit dynamic predication	12
1.4.5 Contributions	12
Chapter 2 Related work	14
2.1 Techniques that fetch a single fetch block per cycle	14
2.1.1 Reuse control-independent data-independent instructions	14
2.1.2 Preserve control-independent instructions	15
2.1.3 Reducing post-misprediction pipeline-refill latency	15
2.2 Techniques that fetch multiple fetch blocks per cycle	15
2.2.1 Speculative multi-threading	15
2.2.2 Multi-path execution	16
2.2.3 Branch pre-execution using helper threads	16
Chapter 3 Squashed-branch reuse via queue	18
3.1 Introduction	18
3.2 Branch alignment for reuse via queue	22
3.2.1 Alignment logic	22
3.2.2 Examples of correct alignment on simple forward-branch mispredic- tions	24
3.2.3 Examples of incorrect alignment and missed alignment for loop- branch mispredictions	27
3.2.4 An example of incorrect alignment on a complex forward-branch misprediction	29
3.2.5 Conclusion	30
3.3 Optimizations for reuse via queue	31

3.3.1	Confidence mechanism: overriding the default predictor	31
3.3.2	Delaying the squash action	33
3.3.3	Handling divergence in the control-independent region	34
3.4	Methodology	34
3.5	Results	35
3.5.1	Confidence mechanism	35
3.5.2	Impact of divergences	38
3.5.3	Impact of delayed squash	39
Chapter 4 Squashed-branch reuse via signature stack and squashed-branch reuse buffer		40
4.1	Introduction	40
4.2	Hierarchical loop iteration identifier: concept and realization	43
4.2.1	Loops and loop hierarchy	43
4.2.2	Hierarchical loop iteration identifier	45
4.2.3	The identifier is a stack	46
4.2.4	Branch alignment using the identifier	48
4.2.5	Realizing the identifier as a Literal Stack	51
4.2.6	Reuse via Literal Stack and Squashed-Branch Reuse Buffer	53
4.3	Reuse via Signature Stack and Squashed-Branch Reuse Buffer	55
4.3.1	Signature Stack: introduction	56
4.3.2	Incrementer-based Signature Stack	58
4.3.3	LFSR-based Signature Stack	62
4.3.4	Optimized LFSR-based Signature Stack and SBRB	64
4.4	Methodology	66
4.5	Results	67
4.5.1	Overall results	67
4.5.2	Benchmark-wise results	68
4.5.3	Design space exploration	70
4.5.4	Incrementer-based SS	73
Chapter 5 Dynamic predication		74
5.1	Introduction	74
5.2	Software improvements to DMP (forward branches)	79
5.2.1	Stage-1: Initial branch selection and re-convergence analysis (an adaptation of prior work)	79
5.2.2	Stage-2: Branch filtering based on the analytical model (an adaptation of prior work)	83
5.2.3	Stage-3: Detecting and filtering inseparable branches using iterative analysis (new)	85
5.3	Software improvements to DMP (loop branches)	90
5.3.1	A review of DMP's loop predication	90
5.3.2	Investigating performance bottleneck in DMP's loop predication	93
5.3.3	Improving DMP's loop predication	95
5.4	Annotating the binary with predication hints	98

5.5	A revised DMP microarchitecture	99
5.5.1	Online profiling using profiling buffer and profiling table	99
5.5.2	Core pipeline changes for dynamic predication	104
5.5.3	Controlled predication using Dynamo	110
5.5.4	Overall storage cost	113
5.6	Methodology	113
5.7	Results	114
5.7.1	Software profiling	115
5.7.2	Predicating forward-branch aggressors	115
5.7.3	Enabling Dynamo	117
5.7.4	Including loop aggressors	118
5.8	Future work	118
	References	122

LIST OF TABLES

Table 3.1	Default parameters of the baseline superscalar processor.	36
Table 4.1	Identifier manipulations triggered on special control-flow edges.	46
Table 4.2	Literal Stack manipulations based on LIT hits.	53
Table 4.3	Literal Stack manipulations based on BTB hits.	53
Table 4.4	Signature Stack manipulation framework.	58
Table 4.5	Incrementer-based SS management.	61
Table 4.6	LFSR-based SS management.	65
Table 4.7	Cost accounting.	66
Table 4.8	Performance (% speedup over baseline) of different SBRB configurations.	72
Table 4.9	Performance (% speedup over baseline) as each SS parameter is reduced (from 64 to 2).	72
Table 4.10	Performance (% speedup over baseline) as LIT sizes are reduced.	72
Table 5.1	Fetch overhead of three different cases of a loop visit.	93
Table 5.2	Breakdown of exact-iterations, over-iterations, and under-iterations.	95
Table 5.3	Cost accounting.	113
Table 5.4	Default parameters of the baseline superscalar processor.	114

LIST OF FIGURES

Figure 1.1	Performance of 192KB TAGE-SC-L predictor and perfect branch predictor normalized to 64KB TAGE-SC-L predictor [Sez16] on a baseline superscalar.	2
Figure 1.2	Breakdown of fetch cycles on a baseline superscalar with a 64KB TAGE-SC-L predictor.	2
Figure 1.3	An example of control independence.	3
Figure 3.1	An example of misprediction elimination with squashed-branch reuse.	19
Figure 3.2	Alignment logic for the reuse-via-queue model.	23
Figure 3.3	An example code of forward branches.	25
Figure 3.4	Alignment sequence when the squashed path for br1 is {br2,br3,br4}.	25
Figure 3.5	Alignment sequence when the squashed path for br1 is {br3,br4}. . .	25
Figure 3.6	An example code of forward branches in a loop.	26
Figure 3.7	Alignment sequence when the squashed path for br2 is {br1,br2,br3} with the search radius criterion disabled.	27
Figure 3.8	Alignment sequence when the squashed path for br2 is {br1,br2,br3} with the search radius criterion enabled.	27
Figure 3.9	Alignment sequence when the squashed path for br2 is {br3} with the search termination criterion disabled.	28
Figure 3.10	Alignment sequence when the squashed path for br2 is {br3} with the search termination criterion enabled.	29
Figure 3.11	An example code of a doubly-nested loop.	29
Figure 3.12	Alignment sequence when the squashed path for br2 is {br3,br2,br1}.	30
Figure 3.13	An example code of a doubly-nested loop with a break.	30
Figure 3.14	Alignment sequence when the squashed path for br3 is {br1,br2}. . .	31
Figure 3.15	Control-flow graph of a loop in 445.gobmk benchmark.	31
Figure 3.16	Using BTB confidence counters to override the default predictor. . .	32
Figure 3.17	The updated alignment logic for handling divergences.	35
Figure 3.18	Speedup of various confidence schemes over baseline.	37
Figure 3.19	Speedup of the L-S-OP scheme with reducing counter widths.	37
Figure 3.20	Speedup of the BRC confidence predictor with varying table sizes. .	38
Figure 3.21	Performance improvement with support for divergences.	39
Figure 4.1	Control-flow graph of loops in bfs BUStep.	44
Figure 4.2	Loop hierarchy expressed as a forest.	45
Figure 4.3	An example of identifier manipulations in bfs BUStep.	47
Figure 4.4	Heuristic-driven linear branch alignment.	48
Figure 4.5	Identifier-led hierarchical branch alignment (initial setup).	49
Figure 4.6	Identifier-led hierarchical branch alignment (after squash by E and fetching D, E, G, ...).	50
Figure 4.7	A Literal Stack with a depth of three.	51
Figure 4.8	Generating signature and key from Literal Stack.	54

Figure 4.9	Squashed-branch reuse microarchitecture using Literal Stack and Squashed-Branch Reuse Buffer.	54
Figure 4.10	Signature Stack.	57
Figure 4.11	Segment and signature allocation in a linearly-evolving state space for doubly-nested loops (B nested within A).	60
Figure 4.12	Logic for updating sig on entering/continuing a loop and entering a function.	60
Figure 4.13	A 32-bit Galois LFSR.	63
Figure 4.14	A 32-bit Galois LFSR with data input, d.	63
Figure 4.15	Signature allocation in a randomly-evolving state space for doubly-nested loops (B nested within A).	64
Figure 4.16	Optimized structure sizes for SS and SBRB.	66
Figure 4.17	Speedup of different reuse configurations over baseline.	68
Figure 4.18	Benchmark-wise speedup for selected configurations (SPEC).	69
Figure 4.19	RVSS speedup and MPKI reduction (absolute) for SPEC benchmarks.	70
Figure 4.20	Benchmark-wise speedup for selected configurations (GAP).	71
Figure 4.21	RVSS speedup and MPKI reduction (absolute) for GAP benchmarks.	71
Figure 4.22	Benchmarks sensitive to limited-size LIT-H/E.	73
Figure 5.1	The result of re-convergence analysis on a forward branch in sssp (using the random input).	84
Figure 5.2	Bhr updates in conventional processor and DMP.	87
Figure 5.3	Bhr updates in improved DMP.	88
Figure 5.4	Instruction stream for a loop branch in tc which is predicted taken in the first two iterations, and predicted not-taken in the third iteration.	92
Figure 5.5	Instruction stream annotated with critical paths for the loop branch in the last iteration.	94
Figure 5.6	A high-level revised DMP microarchitecture.	100
Figure 5.7	Number of aggressors after offline stage-1, stage-2, and stage-3 processing.	115
Figure 5.8	Breakdown of aggressor types after stage-3 processing.	116
Figure 5.9	Overall speedup when forward branches are predicated.	117
Figure 5.10	Percentage speedup of SPEC benchmarks when forward branches are predicated.	118
Figure 5.11	Percentage speedup of GAP benchmarks when forward branches are predicated.	119
Figure 5.12	Overall speedup when Dynamo is enabled.	119
Figure 5.13	Percentage speedup of SPEC benchmarks when Dynamo is enabled.	120
Figure 5.14	Percentage speedup of GAP benchmarks when Dynamo is enabled.	120

AUTHORSHIP STATEMENT

Research related to squashed-branch reuse was collaborative in nature. It is covered in Chapter 1 (Section 1.3), Chapter 3, and Chapter 4, and is part of the following manuscript (status: *accepted/to-appear*):

R. Singh, J. Li and E. Rotenberg, “Augmenting the Branch Predictor with a Squashed-Branch Reuse Buffer”, *2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*, Raleigh, USA, 2026.

In squashed-branch reuse, the following sections involve contributions from other authors:

- Chapter 4 Section 4.3.3: Dr. Rotenberg initially expressed the possibility of using a Linear Feedback Shift Register (LFSR) for signature generation. Rohit Singh discovered the limitations of other signature generation mechanisms, and realized the need for LFSR-based signature generation. Jiayang Li designed the signature updates for LFSR-based Signature Stack.
- Chapter 4 Sections 4.3.4 and 4.5.3: Jiayang Li performed the design space exploration of parameters related to various structures, and arrived at a final set of values optimized for performance and storage cost.
- Chapter 4 Section 4.5.1: Jiayang Li integrated SHA-256 and CRC libraries into the simulator which were used to generate 32-bit signatures from Literal Stack.

Other sections on squashed-branch reuse were contributed solely by Rohit Singh.

Research related to dynamic predication was collaborative in nature. It is covered in Chapter 1 (Section 1.4) and Chapter 5. The following sections involve contributions from other authors:

- Chapter 5 Section 5.4: Jiayang Li designed the hint-injection scheme which annotates RISC-V instructions in-place.

- Chapter 5 Section 5.5.1: Rohit Singh and Jiayang Li co-designed profiling buffer and profiling table. Jiayang Li implemented these structures in the simulator.
- Chapter 5 Section 5.5.2: Rohit Singh and Jiayang Li co-designed predicated branch table and exit table. Jiayang Li implemented these structures in the simulator. Rohit Singh designed and implemented predicate queue and other core pipeline features related to predication.
- Chapter 5 Section 5.5.3: Jiayang Li implemented and integrated Dynamo into the simulator.

Other sections on dynamic predication were contributed solely by Rohit Singh.

No generative artificial intelligence tools were used in writing this dissertation.

Chapter 1

Introduction

1.1 Branch mispredictions: an enduring performance problem

High-performance superscalar processors need accurate branch prediction. Over time, branch predictors have become bigger and more accurate. But lately their growth has stalled. There hasn't been a major improvement in branch prediction since the TAGE-SC-L predictor [Sez16], proposed in 2016. It is still widely considered as the state-of-the-art.

Increasing the size of the predictor doesn't improve performance by much. Figure 1.1 shows increasing the size of the state-of-the-art predictor from 64KB to 192KB improves performance by only 1% on average on a wide range of SPEC (CPU2006INT and CPU2017INT) and GAPBS [BAP15] workloads on a baseline superscalar with a 512-entry ROB.

Figure 1.1 also captures the performance gap between the state-of-the-art predictor and a perfect branch predictor. It shows branch mispredictions continue to hurt performance, and there is a need for improving branch prediction accuracy and/or mitigating the performance penalty of branch mispredictions.

After a branch is fetched, and before its misprediction is detected, a processor spends

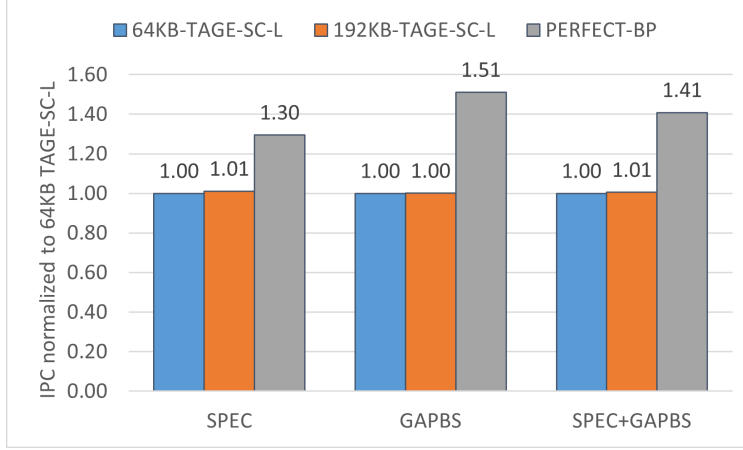


Figure 1.1: Performance of 192KB TAGE-SC-L predictor and perfect branch predictor normalized to 64KB TAGE-SC-L predictor [Sez16] on a baseline superscalar.

upwards of ten fetch cycles on the incorrect path. When the branch misprediction rates are significant, fetch cycles spent on the incorrect path take up a significant proportion of the overall fetch cycles. Our simulations show, on average, more than a third of all fetch cycles are spent on the incorrect path (see Figure 1.2), highlighting the energy penalty of branch mispredictions.

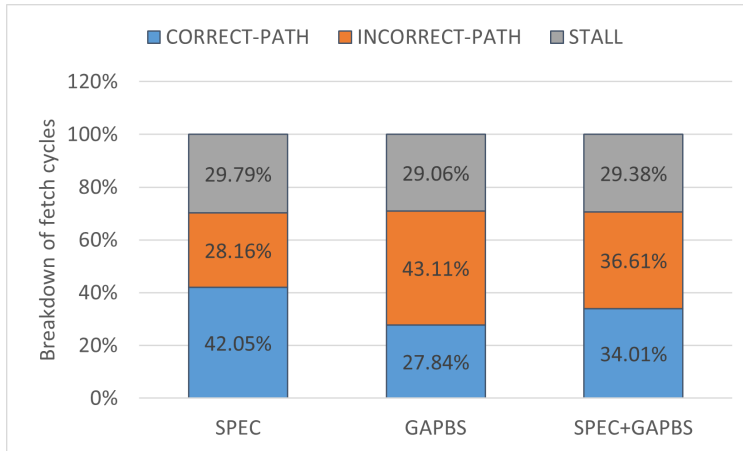


Figure 1.2: Breakdown of fetch cycles on a baseline superscalar with a 64KB TAGE-SC-L predictor.

1.2 Control independence

On a branch misprediction, a processor squashes all the instructions on the incorrect path (*i.e.*, the mispredicted path), and begins fetching on the correct path (*i.e.*, the resolved path). A full-blown squash is often wasteful as on a typical misprediction many squashed instructions are control-independent of the mispredicted branch. Control independence can be exploited to reduce the performance penalty of branch mispredictions, improving performance. Here is an example.

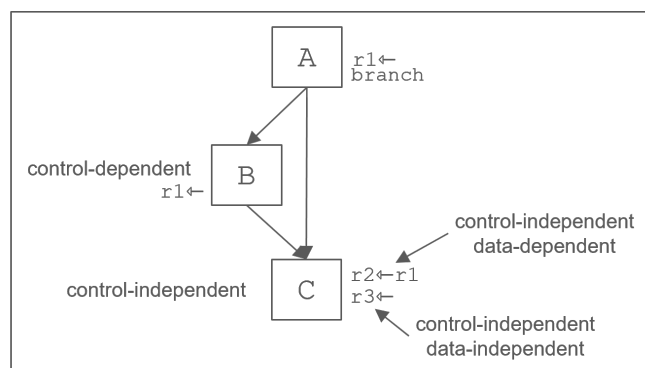
Figure 1.3 shows an example code and its corresponding control-flow graph. With respect to the branch in block A, the instructions in block B are control-dependent and the instructions in block C are control-independent. The first instruction in block C is control-independent data-dependent as its execution is data-dependent on a control-dependent instruction (through register r1). The second instruction in block C is control-independent data-independent as its execution doesn't depend on any control-dependent instruction.

```

A:  x = 0;           // r1←
    if (...)         // branch
B:   x = 1;           // r1←
C:   y = expr1(x);    // r2←r1
    z = 1;           // r3←
    ...;

```

(a) An example code



(b) Control-flow graph of the example code

Figure 1.3: An example of control independence.

A full-blown squash on a misprediction is wasteful from two angles:

- *Control-independent data-independent instructions are redundantly executed.*
 - The resolved-path control-independent data-independent instructions generate

the same values as the squashed control-independent data-independent instructions (if they were executed).

- Techniques exploiting this angle reuse squashed control-independent data-independent instructions' values on the resolved path, improving performance [SS97b; CFS99; RS00; PS12; Gal15; KC25; ASL03]. These techniques fall into a broad paradigm referred to as *reuse control-independent data-independent instructions*.
- Section 1.3 discusses the paradigm at a high level, categorizing the prior techniques into two sub-paradigms: *general squashed-instruction reuse at rename* and *squashed-branch reuse at fetch*. It then provides reasons for revisiting the squashed-branch reuse at fetch paradigm. Finally, it summarizes the major contributions of this dissertation in this area.
- Chapters 3 and 4 are dedicated to squashed-branch reuse at fetch.
- *Control-independent instructions are redundantly fetched.*
 - The resolved-path control-independent instructions fetch and decode the same instruction bits as the squashed control-independent instructions.
 - Techniques exploiting this angle preserve control-independent instructions in the window, avoiding their redundant fetch, and thereby improving performance [RJS99; RS99; CV01; Al+07; HR07; Eye+21; Kla+98; Kim+06; Cha+20; GAS04; Kot+18]. These techniques fall into a broad paradigm referred to as *preserve control-independent instructions*.
 - Section 1.4 discusses the paradigm at a high level, categorizing the techniques into three sub-paradigms: *out-of-order fetch*, *multi-path fetch (dynamic predication)*, and *fetch elimination (exact convergence)*. It then provides reasons for revisiting the multi-path fetch (dynamic predication) paradigm. Finally, it summarizes the major contributions of this dissertation in this area.

– Chapter 5 is dedicated to dynamic predication.

1.3 Reuse control-independent data-independent instructions

Under this paradigm, on a misprediction, all mispredicted-path instructions are squashed as usual, but their execution result (if available) is preserved in some buffer for future use. As resolved-path instructions are processed, they consult the buffer for a squashed result. If a result is found, it is further checked for reliability via a reuse test. If the result is found to be reliable (indicated by a successful reuse test), it can be reused to break critical program dependencies, improving performance. This paradigm is commonly known as *squash reuse*. There are two distinct sub-paradigms under squash reuse as discussed below.

1.3.1 General squashed-instruction reuse at rename

In this sub-paradigm, on a misprediction, squashed results are preserved in the physical register file [RS00; PS12; KC25], in an alternate reorder buffer [CFS99], or in a generic reuse buffer [SS97b]. As resolved-path instructions are renamed, they try to obtain a squashed result from an above-mentioned buffer. If a squashed result is found, it undergoes a reuse test. The test benefits from precise tracking of register and memory dependencies on both, the squashed and the resolved paths, succeeding only for truly control-independent data-independent results. If the test succeeds, reuse is allowed to proceed. As reuse is unambiguously correct, reused resolved-path instructions (or simply reused instructions) are removed from the pipeline, bypassing execution completely ¹.

A reused non-branch instruction breaks data-dependency by letting its consumers execute early, improving performance much like value prediction [LS96; SS97a]. A reused

¹This is true for all prior techniques except MSSR [KC25]. In MSSR, load reuse is speculative, therefore, loads are re-executed at retire.

branch breaks control-dependency by allowing itself to resolve early, improving performance if it was mispredicted.

1.3.2 Squashed-branch reuse at fetch

In this sub-paradigm, on a misprediction, squashed results (*i.e.*, branch outcomes) are preserved in an alternate branch queue [PS12; Gal15; ASL03]. As resolved-path branches are fetched, they try to obtain a squashed outcome from the queue. If an outcome is found, it undergoes a reuse test. As there exists no easy mechanism for tracking register and memory dependencies at fetch, the test is imprecise, and is fulfilled by a confidence predictor. If the test succeeds, reuse is allowed to proceed, overriding the default prediction for the branch. A reused branch stays in the pipeline, verifying its overriding prediction (*i.e.*, the reused outcome) upon execution for correctness just like any other branch predicted by the default predictor.

As reuse occurs at fetch, a reused branch avoids the entire misprediction penalty if its default prediction is incorrect (assuming the reused outcome turns out to be correct).

1.3.3 Why revisit squashed-branch reuse at fetch

- *Squashed-branch reuse is much more potent than general squashed-instruction reuse:*

The performance impact of squash reuse is highly correlated with the number of instructions eliminated from processing completely [RS00]. Instructions are eliminated from the pipeline via accelerated resolution of branches. In general squashed-instruction reuse, a branch mispredicted by the default predictor on the resolved path can resolve earlier than usual, either at rename (for a branch, *all* of whose source register values are control-independent data-independent) or at execute (for a branch with at least one source register value that is control-independent data-dependent ²).

²If all source register values are control-independent data-dependent, then at least one of the source register value's backward slice contains a control-independent data-independent value, allowing accelerated resolution.

Whereas in squashed-branch reuse, a mispredicted branch on the resolved path can *virtually* resolve at fetch ³ (for branches which are *inferred* as control-independent data-independent).

Strictly speaking, it is unclear if a squashed-branch reuse implementation would always outperform a general squashed-instruction reuse implementation because a) the former eliminates many more cycles than the latter for every early-resolved branch (on average), and b) the latter has a potentially higher coverage of early-resolved branches than the former. But if a workload has a high percentage of reusable control-independent data-independent branches and/or the fetch-to-rename pipeline is deep, a squashed-branch reuse implementation will likely outperform a general squashed-instruction reuse implementation.

Nonetheless, general squashed-instruction reuse can be combined with squashed-branch reuse if desired.

- *Squashed-branch reuse is simpler to implement:* Squashed-branch reuse operates on branches which is advantageous because branches represent the dynamic instruction stream in a compressed form. Implementation-wise, a reuse buffer in a general squashed-instruction reuse implementation would require far more entries and would need to support much more read/write traffic in comparison to a reuse buffer in a squashed-branch reuse implementation.
- *Squashed-branch reuse is much less explored than general squashed-instruction reuse:* There exist five prior works on general squashed-instruction reuse [SS97b; CFS99; RS00; PS12; KC25] whereas only three prior works have explored squashed-branch reuse [PS12; Gal15; ASL03].

³Note, in squashed-branch reuse at fetch, as reuse is speculative, the actual resolution occurs at execute. But it still breaks control dependency by allowing its likely-correct control-dependent instructions to be fetched *immediately* on reuse.

1.3.4 Contributions

This dissertation makes the following contributions in the squashed-branch reuse sub-paradigm:

- In squashed-branch reuse, resolved-path branches need to be properly aligned with their squashed-path counterparts (if they exist). In SYRANT [PS12], the authors mention a couple of heuristics for aligning branches in a reuse-via-queue scheme but they omit many important details. This dissertation describes the *alignment logic* for the reuse-via-queue model, illustrating how SYRANT’s alignment heuristics allow correct alignment and prevent incorrect alignment on simple forward-branch mispredictions⁴. It also shows how the alignment breaks on loop-branch mispredictions and complex forward-branch mispredictions.
- Building upon an initial implementation, this dissertation includes the following optimizations to the reuse-via-queue model:
 - It evaluates the various *confidence mechanisms* proposed in prior works, and arrives at an optimized design.
 - It puts forth a simple *delayed squash mechanism* to improve reuse coverage.
 - It describes a simple tweak in the alignment logic that allows *reuse past divergence in the control-independent region*.
- In the reuse-via-queue model, it becomes difficult to align branches after a loop-branch divergence or a complex forward-branch divergence. To that end, this dissertation proposes a key innovation, namely *hierarchical loop iteration identifiers*. Identifiers achieve perfect alignment despite arbitrary control-flow differences between the squashed and the resolved paths. Specifically, the dissertation makes the following contributions:

⁴Here, a branch is called a forward branch if it converges within the loop it appears in. Otherwise, it is called a loop branch.

- It describes the *conceptual identifier and its role in branch alignment*. A branch on a resolved path is said to be aligned with a branch on a squashed path if both its PC and its identifier match on the two paths.
 - It then proposes a scheme for *realizing the identifier as a Literal Stack* in the hardware. It involves conveying special control-flow edges in a program’s binary and inferring them during the program’s execution.
 - Finally, it introduces the *Squashed-Branch Reuse Buffer* – a reuse buffer implemented as a set-associative cache, indexed and tagged using a combination of a branch’s PC and its identifier. Reuse via Literal Stack and Squashed-Branch Reuse Buffer overcomes the major drawbacks of the reuse-via-queue model.
- For practical purposes, an identifier is exposed to the pipeline as a signature which is a short fixed-length representation of an identifier. A Literal Stack, though logically straightforward to design, is inefficient at supplying signatures. This dissertation proposes another key innovation, namely, the *Signature Stack*, that overcomes those inefficiencies. At a high-level, the dissertation makes the following contributions:
 - It presents two different implementations of the Signature Stack: *incrementer-based Signature Stack* and *LFSR-based Signature Stack*.
 - After a thorough design space exploration, it arrives at a final storage cost of a squashed-branch reuse implementation using an LFSR-based Signature Stack and Squashed-Branch Reuse Buffer.

1.4 Preserve control-independent instructions

Under this paradigm, on a misprediction, control-independent instructions are not re-fetched, and therefore preserved in the window. Moreover, control-independent data-independent instructions are not re-executed. The techniques differ on how they man-

age the fetch of control-dependent instructions, and correspondingly the execution of the control-independent data-dependent instructions. Based on fetch models, there are three distinct sub-paradigms, as discussed below.

1.4.1 Out-of-order fetch

In this sub-paradigm, control-dependent instructions on the predicted path of a hard-to-predict branch are either speculatively fetched and executed [RJS99; RS99; Al+07; HR07; Eye+21], or skipped altogether [CV01]. Correspondingly, control-independent data-dependent instructions are speculatively executed ⁵, or stalled in the scheduler.

If the control-dependent instructions were speculatively fetched and executed, the next set of actions depend on the branch outcome. If a branch resolves as mispredicted, the mispredicted-path control-dependent instructions are squashed, and the resolved-path control-dependent instructions are fetched, out-of-order with respect to the preserved control-independent instructions. Subsequently, the misspeculated control-independent data-dependent instructions are re-renamed and re-executed. Whereas if a branch resolves as correctly predicted, no special action is needed besides reclaiming the replay buffer resources held by correctly speculated instructions.

If the control-dependent instructions were skipped, the next set of actions are independent of the branch outcome. When a branch resolves, the resolved-path control-dependent instructions are out-of-order fetched. Subsequently, the stalled control-independent data-dependent instructions are released for execution.

1.4.2 Multi-path fetch (dynamic predication)

In this sub-paradigm, control-dependent instructions on the predicted path and the alternate path are simultaneously fetched and executed [Kla+98; Kim+06; Cha+20] ⁶. Whereas

⁵BLP [Eye+21] targets branches which have no control-independent data-dependent instructions except reduction variables.

⁶In ACB [Cha+20], control-dependent instructions are stalled in the scheduler.

control-independent data-dependent instructions are stalled in the scheduler.

When a branch resolves, the mispredicted-path control-dependent instructions are squashed, and the resolved-path control-dependent instructions are allowed to commit their results. Subsequently, the stalled control-independent data-dependent instructions are released for execution.

1.4.3 Fetch elimination (exact convergence)

There two techniques in this sub-paradigm: SBR [GAS04] and SPF [Kot+18].

In SBR, control-dependent instructions on the predicted path are speculatively fetched and executed. Correspondingly, control-independent data-dependent instructions are speculatively executed.

When a branch resolves as mispredicted, a dynamic instance of the instruction at the resolved target is located in the pipeline for exact convergence ⁷. On a hit, the effects of mispredicted-path control-dependent instructions are nullified by re-dispatching and re-issuing them as move instructions. Subsequently, the misspeculated control-independent data-dependent instructions are re-dispatched and re-issued. Note there is no out-of-order fetch as resolved-path control-dependent instructions are eliminated due to exact convergence. Otherwise, when a branch resolves as correctly predicted, the replay buffer resources held by correctly speculated instructions are reclaimed.

SPF is different from SBR in certain aspects. When a branch resolves as mispredicted, the look-up of the instruction at the resolved target is restricted to the pre-rename pipeline registers. This trick eliminates misspeculated control-independent instructions. Consequently, mispredicted-path control-dependent instructions can be simply squashed, and removed from the pipeline. There is no action needed when a branch resolves as correctly predicted.

⁷The scheme looks for the oldest dynamic instance of the instruction that is younger than the branch.

1.4.4 Why revisit dynamic predication

- *Dynamic predication is relatively simple-to-implement:* Out-of-order fetch complicates resource management [RJS99; CV01; Al-+07; HR07; Eye+21] whereas selective re-execution requires a dedicated replay buffer [RJS99; HR07; Al-+07; GAS04]⁸. Predication is relatively simple-to-implement given it neither requires out-of-order fetch nor selective re-execution. The only other technique that has the same benefit is SPF [Kot+18].
- *Dynamic predication is guided by an analytical model which compares the fetch overhead of predication against the fetch overhead of conventional speculation, but the model ignores the impact on global branch history – a crucial side-effect of predication on instruction fetch:* Like other schemes which preserve control-independent instructions in the window, predication perturbs the global branch history. When a branch is predicated, it is removed from the global branch history, reducing the accuracy of branches which were correlated with the predicated branch [Cha+20]. So even when a branch is considered a strong candidate for predication by the analytical model, it may not provide any performance benefit – or in the worst case, result in a slowdown – when predicated due to global branch history effects.

1.4.5 Contributions

This dissertation makes the following contributions to dynamic predication:

- To limit the negative effect of predication on the prediction accuracy of other branches, this dissertation proposes to disallow predication of *inseparable* branches, *i.e.*, branches which have a (substantial) negative effect on the accuracy of unpredicated branches. Inseparable branches are filtered at compile time using a novel

⁸Trace processors [RS99] are interesting in that they do not suffer from these issues. Nonetheless, they are significantly different from conventional superscalar processors.

iterative analysis. For every input branch, the analysis measures the impact of *individually* predicating the branch on un-predicated branches, and iteratively removes branches from the input set until all the remaining branches are *separable*, *i.e.*, they have little or no negative effect on the accuracy of un-predicated branches.

- A prior work showed predication can be successfully applied to certain hard-to-predict loop branches [Kim+06], but we observed no benefit from predicating loop branches using the prior scheme.

In this dissertation, we revamp loop predication such that both a) the decision to predicate a loop, and b) the number of iterations to predicate for (if we decide to predicate the loop), are guided by the loop’s iteration count for the previous n visits to the loop. This way, predication subsumes many more loop branch mispredictions, and sharply diminishes the chances of loop under-iterations – a key issue with the prior scheme – while predicating a loop.

Chapter 2

Related work

In this chapter we present a quick literature review of different approaches for reducing or eliminating the branch misprediction penalty.

2.1 Techniques that fetch a single fetch block per cycle

2.1.1 Reuse control-independent data-independent instructions

As discussed in Chapter 1, because of control independence, a full-blown misprediction squash is wasteful from the following angle: resolved-path control-independent data-independent instructions generate the same values as the squashed control-independent data-independent instructions (if they were executed).

Techniques exploiting this angle preserve (not discard) the execution results of the squashed-path instructions, and then later reuse them on the resolved path [RS00; KC25; CFS99; SS97b; PS12; Gal15; ASL03]. A reused non-branch instruction (at rename) improves performance by breaking data-dependency much like value prediction. A reused branch (at fetch or at rename) resolves early, improving performance if it was mispredicted.

2.1.2 Preserve control-independent instructions

Because of control independence, a full-blown misprediction squash is wasteful from another angle: resolved-path control-independent instructions fetch and decode the same instruction bits as the squashed control-independent instructions.

Techniques exploiting this angle preserve a branch’s control-independent instructions in the window irrespective of its outcome [RJS99; Al+07; HR07; Eye+21; CV01; Kla+98; Kim+06; Cha+20; GAS04; Kot+18]. Whenever the branch mispredicts, the performance is boosted by eliminating the redundant fetch of its control-independent instructions.

2.1.3 Reducing post-misprediction pipeline-refill latency

After a misprediction, the fetch of resolved-path instructions is almost always on the critical path as it takes several cycles to refill the pipeline with resolved-path instructions. Many techniques have looked into reducing this pipeline refill latency [BND96; Ara+02; DCP24a; Sin+24]. Performance is boosted by shortening the program’s critical path.

2.2 Techniques that fetch multiple fetch blocks per cycle

2.2.1 Speculative multi-threading

Speculative multi-threading techniques try to improve a program’s performance by breaking it into tasks, and fetching from multiple tasks simultaneously [SBV95; AD98; PFV03; Aga+07; Mal+08]. To reap the benefits of high-bandwidth fetch, the task selection algorithms try to minimize critical inter-task control and data dependencies, among various other objectives [VS98]. Depending on the technique, a task may run exclusively on a hardware thread context (for best possible performance) or share a context with other tasks (for better hardware utilization) [PFV03].

Speculative multi-threading can be beneficial for benchmarks with high misprediction rates. Tasks are typically specified in such a way that the control-flow for most (or all) of the branches within a task, converges within the same task. Therefore, a branch misprediction usually leads to a local squash, and instructions in the younger tasks are preserved. So, speculative multi-threading is similar to out-of-order fetch techniques that preserve instructions in the window, as discussed previously. But its scope is not limited to reducing the misprediction penalty – it can be used to improve the fetch rate in general.

2.2.2 Multi-path execution

In multi-path execution, to avoid the misprediction penalty for a hard-to-predict branch, instructions on both, the predicted path and the alternate path, are eagerly executed [USH95; HS96; KPG98; Ahu+98; WCT98; WTC99]. When the branch resolves, the instructions on the incorrect path are squashed, and only the instructions on the correct path are allowed to commit and update the architectural state of the machine.

Multi-path execution is one of the most general techniques to effectively reduce the branch misprediction penalty. However, it is not very scalable. To support up to n unresolved hard-to-predict branches, we need $n+1$ hardware contexts. With increasing n , the hardware utilization continues to drop as in any given interval only one context drives forward progress.

2.2.3 Branch pre-execution using helper threads

In branch pre-execution, in addition to the main program, one or more helper programs are realized and launched as separate threads [SPR00; PSR00; ZS01; Cha+99; Cha+02; GH08; KH19; SCR20; PP21; DCP24b; SR25]. Irrespective of the helper-program-creation algorithm, a helper program always includes some instructions from the backward slice of one or more hard-to-predict branches.

A helper thread which runs ahead of the main thread, is geared towards producing a

hard-to-predict branch’s result before the main thread fetches the corresponding branch instance¹. Performance is improved when the pre-computed result is available and correct, and the branch is mispredicted by the main thread.

¹This is true for all branch pre-execution techniques except TEA [DCP24b]. In TEA, the authors make a case of using a pre-executed result after the corresponding branch is fetched (but not yet executed) by the main thread.

Chapter 3

Squashed-branch reuse via queue

3.1 Introduction

Squashed-branch reuse eliminates the entire misprediction penalty of a reused control-independent data-independent branch that was mispredicted by the default predictor. Let's look at an example.

Figure 3.1 shows an example code with basic block annotations on the left. It has three forward branches: br1, br2, and br3 (sub-figure:a). With respect to br1, br2 is control-independent data-dependent (through variable x) whereas branch br3 is control-independent data-independent.

Consider this scenario. While fetching along path $\{A,B,C,E,F,G\}$, the processor has mispredicted all the three branches (the correct path is $\{A,C,D,E,G\}$), and none of them have resolved yet (sub-figure:b). After a few cycles, the pipeline resolves br3 (out-of-order with respect to br1 and br2), squashes its mispredicted-path instructions (blocks F and G), and fetches its resolved-path instructions (block G) (sub-figure:c). Shortly after, the pipeline resolves br1, squashes its mispredicted-path instructions (blocks B, C, E, and G), and begins fetching on the resolved path.

As br2 and br3 are re-fetched on the resolved-path, they are re-predicted. This time

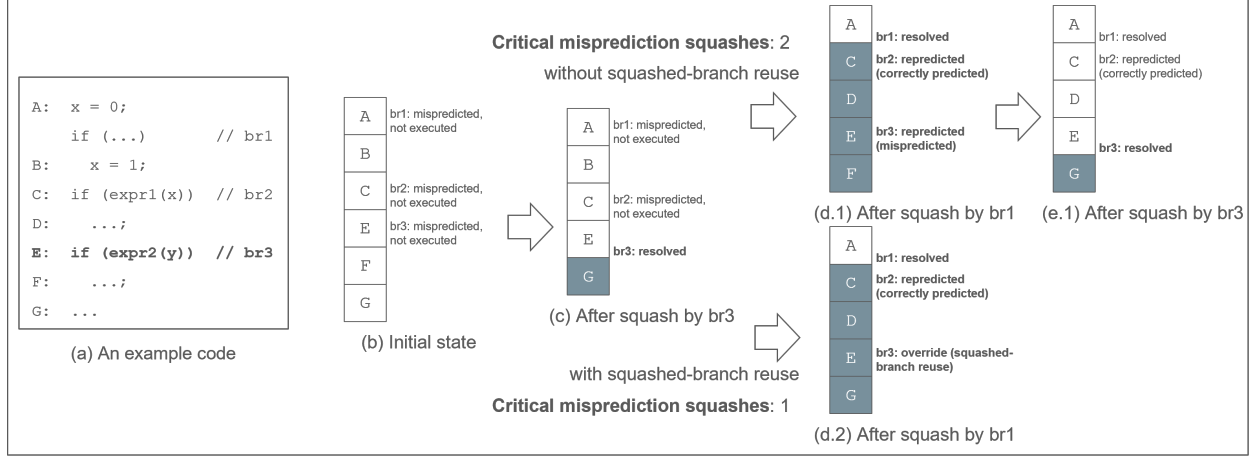


Figure 3.1: An example of misprediction elimination with squashed-branch reuse.

br2 may get correctly predicted as it is correlated with br1, but br3 may mispredict again as it has no correlation with prior branches. If br3 indeed mispredicts (sub-figure:d.1), it will cause another squash when it resolves (sub-figure:e.1).

With squashed-branch reuse, when br1’s misprediction leads to a squash, br3’s outcome is preserved (and not thrown away). This outcome is re-used to override the default branch prediction when br3 is re-fetched, avoiding a misprediction when its default prediction is incorrect (sub-figure:d.2). In this example, squash-branch reuse reduces the number of critical misprediction squashes from two to one. Note, the first misprediction caused by br3 (sub-figure:c) is not on the critical path as it occurs in the shadow of an older unresolved mispredicted branch br1.

For reuse to occur, first a resolved-path branch needs to be paired (aligned) with its corresponding squashed-path instance. Branch alignment is challenging because of loops and nested control-flow. With loops, there can be multiple outstanding instances of the same static branch on the squashed path. With nested control-flow, for a given iteration of a loop, an instance of a static branch may or may not exist on the squashed/resolved path.

Three prior works that have explored squashed-branch reuse are: SYRANT [PS12], GMD [Gal15], and BRC [ASL03]. In these works reuse occurs via a queue: first on a

misprediction the squashed-path branches (along with their outcomes when available) are saved to a queue, and then as the resolved path is fetched, the resolved-path branches are aligned with their corresponding squashed-path instances by searching the queue.

This chapter revisits branch alignment for the reuse-via-queue model and proposes various other optimizations to it. The chapter is organized as follows:

- Section 3.2 is dedicated to branch alignment for the reuse-via-queue model:
 - SYRANT and GMD propose different schemes for aligning branches on forward-branch mispredictions, in presence of loops and nested control-flows. In our assessment, SYRANT’s scheme is superior as it works (leads to a correct alignment) for many use-cases. On the contrary, GMD’s scheme fails for many use-cases, and therefore, it is not discussed in this work.

BRC’s alignment scheme is oblivious to loops and nested control-flows. It uses a brute-force approach where a resolved-path branch is aligned with the first instance of the branch on the squashed path (if it exists). As a result, BRC’s alignment accuracy is the worst among the three schemes.

In SYRANT, the authors mention a couple of heuristics for aligning branches, but they omit many important implementation details. Section 3.2.1 provides an implementation of the *alignment logic* for the reuse-via-queue model (SYRANT’s scheme). Section 3.2.2 illustrates how SYRANT’s alignment heuristics allow correct alignment and prevent incorrect alignment on simple forward-branch mispredictions. Section 3.2.3 and Section 3.2.4 show how the alignment breaks on loop-branch mispredictions and complex forward-branch mispredictions, respectively. Section 3.2.5 concludes the discussion.

- Section 3.3 covers optimizations for the reuse-via-queue model:
 - When a squashed outcome is available for a branch, a confidence predictor determines whether it overrides the branch’s default prediction. In SYRANT, the

authors mention that a single (global) counter is sufficient for this purpose, but again omit other implementation details. In GMD, the author uses per-branch (local) resetting counters that are trained based on the accuracy of squashed outcomes. In BRC, the authors use a gshare-like predictor, where three entities, namely the mispredicted branch PC, the branch history, and the fetched branch PC, are hashed together to access a tag-less table of saturating counters. These saturating counters are trained based on the accuracy of squashed outcomes in comparison to the default predictions.

A single (global) counter is inadequate when reliable outcomes (control-independent data-independent *w.r.t.* prior squashes) are interspersed with unreliable ones (control-independent data-dependent *w.r.t.* prior squashes). Per-branch (local) counters that are trained based on the accuracy of squashed outcomes are also sub-optimal as they are oblivious to the accuracy of default predictions.

Section 3.3.1 implements a *confidence mechanism* that uses per-branch (local) saturating counters which are trained based on the accuracy of squashed outcomes in comparison to the default predictions. It is effective at capturing the relative trustworthiness of squashed outcomes in comparison to default predictions, and outperforms the gshare-like predictor with a tag-less table of saturating counters.

- In a baseline superscalar, the misprediction squash may be immediate for convenience where all squashed-path instructions are removed from the pipeline as soon as the misprediction is detected. But immediate squash is too restrictive for reuse coverage, and therefore, delayed squash is much more desirable. In SYRANT, the authors highlight the benefit of delayed squash but again, do not provide any details. In BRC, the authors propose not to force-remove squashed-path instructions from the pipeline, ever. This allows every squashed-

path instruction to complete execution and exit the pipeline just like other instructions.

Keeping squashed-path instructions in the pipeline beyond a certain point can take critical pipeline resources away from the resolved-path instructions, and may result in a slowdown. In BRC, this resource pressure goes undetected and untreated.

Section 3.3.2 describes a simple *delayed squash mechanism* that boosts reuse coverage without impeding the progress of resolved-path instructions.

- When a resolved-path branch aligns with a squashed-path branch (on the first branch in the control-independent region), the two paths remain aligned unless the prediction on the squashed path differs from the prediction on the resolved path. When the predictions on the two paths differ, the paths are said to be diverged. Depending upon the workload, frequent divergences may limit reuse coverage if not handled properly. None of the prior works discuss this.

Section 3.3.3 proposes a simple tweak in the alignment logic that sets up diverged paths for re-alignment, allowing *reuse past divergence in the control-independent region*.

- Section 3.4 covers workloads and simulation methodology.
- Section 3.5 discusses the performance results of various optimizations.

3.2 Branch alignment for reuse via queue

3.2.1 Alignment logic

Figure 3.2 shows the key structures used in the reuse-via-queue model (sub-figure:a). The active queue (AQ) contains all (non-squashed) in-flight branches, and it has a fetch pointer (FP) associated with it. The shadow queue (SQ) contains squashed branches, and it has

a reuse pointer (RP) and a reuse status associated with it ¹. Initially, the reuse status is RS_INVL (invalid) (sub-figure:b).

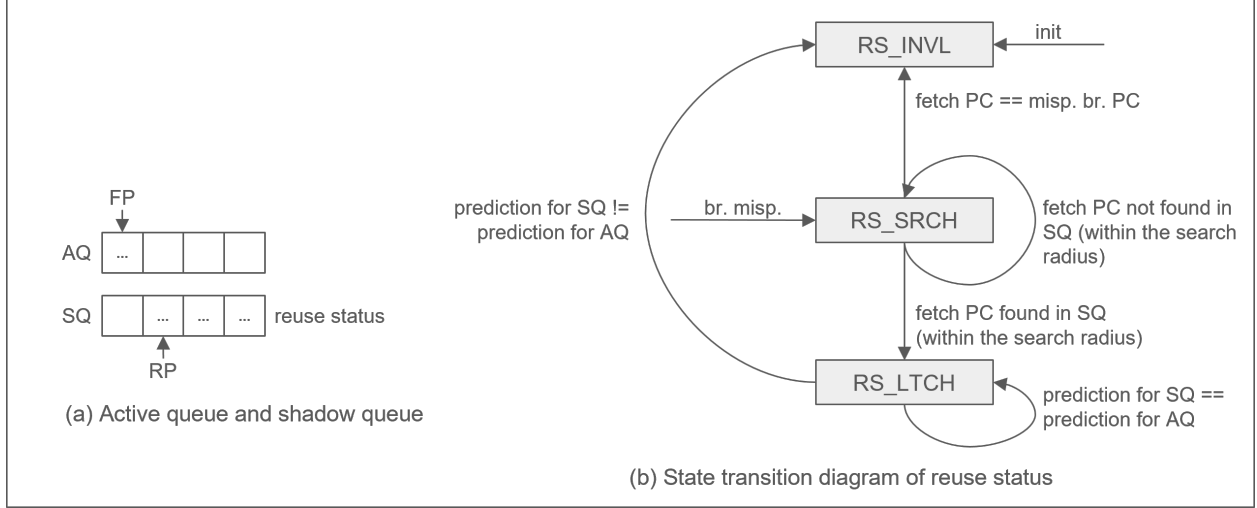


Figure 3.2: Alignment logic for the reuse-via-queue model.

On a misprediction: a) all squashed-path branches (specifically the PC, the prediction, and the outcome – if available – of each individual branch) are copied from AQ to SQ, b) the FP is rolled back to the mispredicted branch, c) the RP is set to the first branch on the squashed-path, and d) the reuse status is set to RS_SRCH (actively searching). Now when a branch is fetched on the resolved path (into AQ), it is associatively-searched in the SQ using its PC (fetch PC in sub-figure:b). To avoid latching onto a squashed-path instance from a different loop iteration (as much as possible), the search is restricted in scope: the SQ is searched from the RP (inclusive) up to the next dynamic instance of the mispredicted branch (exclusive). The search scope is referred here as the *search radius criterion* – the first heuristic mentioned in SYRANT. If the fetch PC is not found in SQ, the reuse status and the RP remain unchanged, and search is retried for the branch that is fetched next.

¹SQ has another pointer which marks the end of the squashed instruction stream (not shown in the figure).

In the RS_SRCH state, if the processor fetches another instance of the mispredicted branch, the reuse status transitions to RS_INVL, terminating the search. This is the second heuristic mentioned in SYRANT, and is referred here as the *search termination criterion*. Just like the search radius criterion, the search termination criterion is also geared towards avoiding a latch onto a different loop iteration. The intent of both the criteria will become clear with examples.

In the RS_SRCH state, if the fetched branch’s PC is found in the SQ: a) the RP is set to the matching entry, and b) the reuse status transitions to RS_LTCH (latched). The branch on the resolved path (pointed by FP) is now *aligned* with the branch on the squashed path (pointed by RP), and consequently, the squashed outcome (if available) at the SQ entry pointed by RP is a candidate for reuse. If the prediction on the resolved path matches with the prediction on the squashed path, RP is incremented by one and the reuse status remains unchanged (the next pair of branches on the two paths are also aligned). When the predictions on the two paths do not match, the reuse status transitions to RS_INVL, terminating reuse.

3.2.2 Examples of correct alignment on simple forward-branch mispredictions

Case 1: Mispredicted forward branch (not in a loop)

We first look at how branch alignment works with nested control-flow but without a loop. Figure 3.3 shows an example code with forward branches br1, br2, br3, and br4 (br2 is nested within br1). For this code, let’s consider two squash scenarios for br1.

Figure 3.4 shows the first squash scenario. The processor has fetched along path {br1,br2,br3,br4} (sub-figure:a). When br1 resolves as mispredicted, br2, br3, and br4 are copied from AQ to SQ, the RP is set to br2, and the reuse status is set to RS_SRCH (sub-figure:b). When br3 is fetched into AQ, the search in the SQ returns a hit, setting

```

if (...)    // br1
    if (...) // br2
        ...;
if (...)    // br3
    ...;
if (...)    // br4

```

Figure 3.3: An example code of forward branches.

RP to br3 and reuse status to RS_LTCH (correct alignment). SQ remains latched when br4 is fetched ² (sub-figure:d).

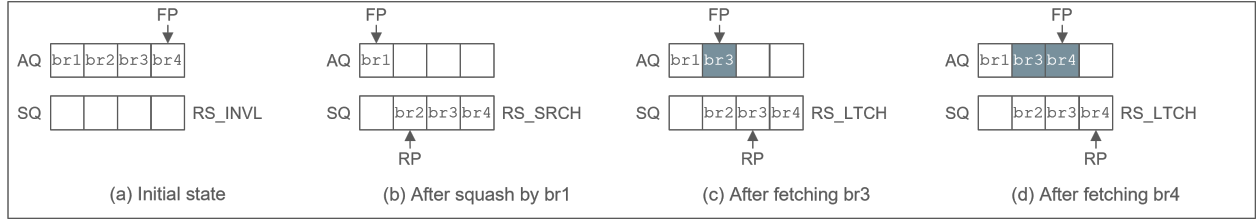


Figure 3.4: Alignment sequence when the squashed path for br1 is {br2,br3,br4}.

Figure 3.5 shows the second squash scenario where the processor has fetched along path {br1,br3,br4} (sub-figure:a). When br1 resolves as mispredicted, SQ is set up appropriately (sub-figure:b). When br2 is fetched into AQ, the search in the SQ returns a miss, keeping RP and reuse status unchanged (sub-figure:c). The search returns a hit when br3 is fetched, causing a latch (sub-figure:d; correct alignment).

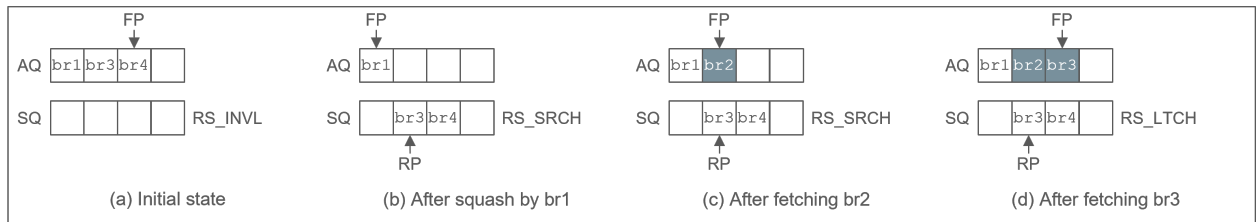


Figure 3.5: Alignment sequence when the squashed path for br1 is {br3,br4}.

²The discussion assumes that the prediction for br3 is the same on the two paths.

Case 2: Mispredicted forward branch in a loop

We now look at a case with nested control-flow inside a loop. Figure 3.6 shows a loop (A) that contains two forward branches br2 and br3 (br3 is nested within br2) and one loop branch br1. Let's consider two squash scenarios for br2.

```
Loop A:  while (...) { // br1
          if (...)      // br2
            if (...)    // br3
              ...;
          }
```

Figure 3.6: An example code of forward branches in a loop.

Figure 3.7 shows the first squash scenario but with the search radius criterion disabled. The processor has fetched along path {br1,br2,br1,br2,br3} (sub-figure:a). When br2 resolves as mispredicted (sub-figure:b), and br3 is fetched into AQ, it results in a latch (sub-figure:c). The alignment here is incorrect as the latched br3 in AQ is from the first iteration of the loop whereas the latched br3 in SQ is from the second iteration of the loop.

Figure 3.8 shows the squash scenario with the search radius criterion enabled. Now when br3 is fetched into AQ, it doesn't result in a latch, leaving the RP and the reuse status unchanged (sub-figure:c). A latch occurs when br1 is fetched (sub-figure:d). The alignment here is correct as both the latched instances belong to the second iteration of the loop.

Figure 3.9 shows the second squash scenario but with the search termination criterion disabled. The processor has fetched along path {br1,br2,br3} (sub-figure:a). After br2 resolves as mispredicted (sub-figure:b), the fetch of br1 and br2 doesn't cause a latch as they miss in the SQ (sub-figure:c,d). But the fetch of br3 causes a latch (sub-figure:e). The alignment here is incorrect as the latched br3 in AQ is from the second iteration of the

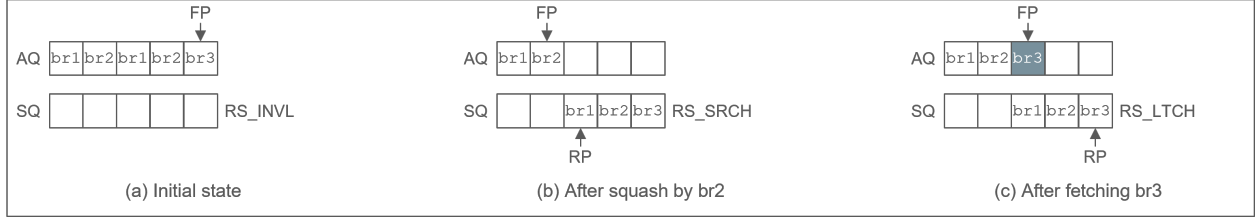


Figure 3.7: Alignment sequence when the squashed path for br2 is {br1,br2,br3} with the search radius criterion disabled.

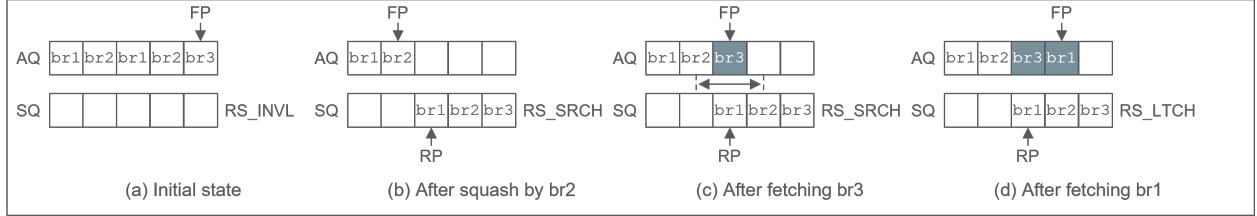


Figure 3.8: Alignment sequence when the squashed path for br2 is {br1,br2,br3} with the search radius criterion enabled.

loop whereas the latched br3 in SQ is from the first iteration of the loop.

Figure 3.10 shows the squash scenario with the search termination criterion enabled. Now when br2 is fetched into AQ, the reuse status is set to RS_INVL, terminating reuse (sub-figure:d). The fetch of br3 doesn't result in a latch as the reuse has already been terminated (sub-figure:e).

3.2.3 Examples of incorrect alignment and missed alignment for loop-branch mispredictions

Case 1: Mispredicted loop branch (missed alignment)

So far we have looked at forward-branch mispredictions. We now look at a case of a loop-branch misprediction. Figure 3.11 shows a loop (B) which contains a forward branch br3 and a loop branch br2. Loop B is nested within another loop (A). Loop A contains a loop branch br1.

Figure 3.12 shows a squash scenario for br2. The processor has fetched along path

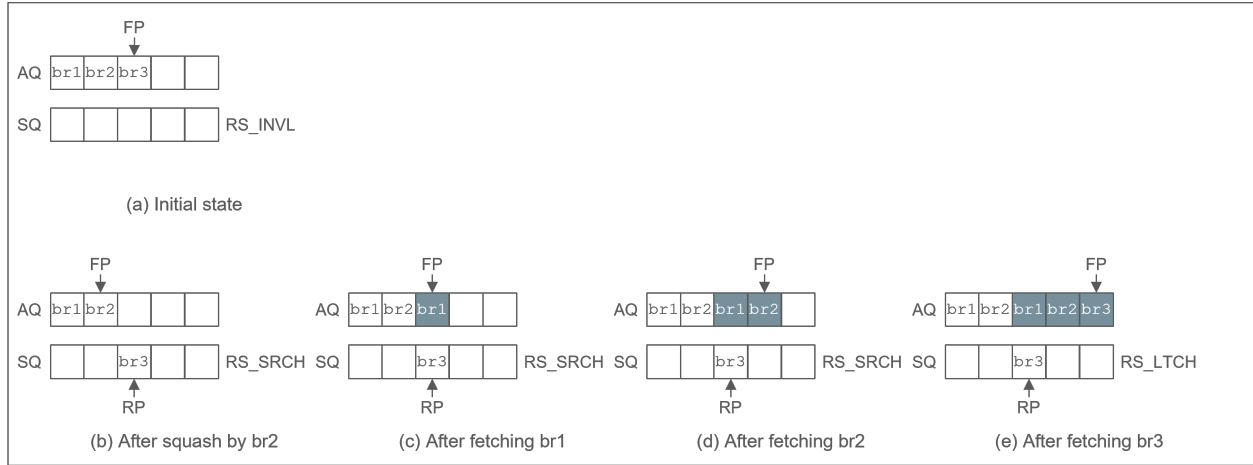


Figure 3.9: Alignment sequence when the squashed path for br2 is {br3} with the search termination criterion disabled.

br1,br2,br3,br2,br1 (sub-figure:a). Now the br2 from the first iteration of loop A resolves as mispredicted (sub-figure:b). When br1 is fetched into AQ, it doesn't latch to the br1 in SQ due to the search radius criterion (sub-figure:c). This is a missed alignment opportunity as both, the second br1 in AQ (at FP) and the br1 in SQ (at RP+2), belong to the second iteration of loop A.

Case 2: Mispredicted loop branch (incorrect alignment)

Let's look at another case of a loop branch misprediction. Figure 3.13 shows a loop (B) which contains two loop branches br2 and br3³. Loop B is nested within another loop (A). Loop A has a loop branch br1.

Figure 3.14 shows a squash scenario for branch br3. The processor has fetched along path {br1,br2,br3,br1,br2} (sub-figure:a). Now br3 resolves as mispredicted (sub-figure:b). When br2 is fetched into AQ, it leads to a latch (sub-figure:c). This is a case of an incorrect alignment as the latched br2 in AQ belongs to the first iteration of loop A whereas the latched br2 in SQ belongs to the second iteration of loop A.

³br3 is a loop branch as it encloses a break statement.

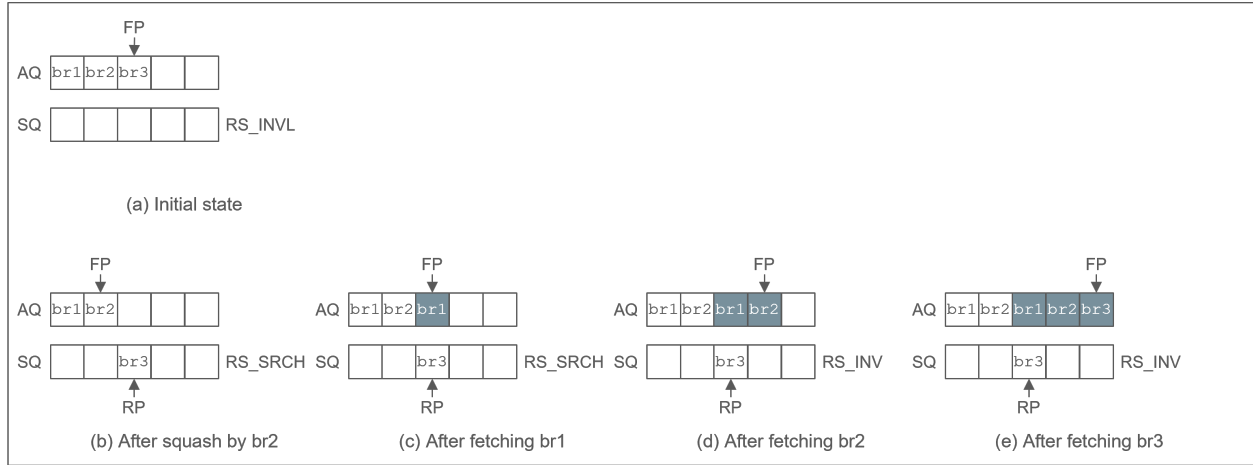


Figure 3.10: Alignment sequence when the squashed path for br2 is {br3} with the search termination criterion enabled.

```

Loop A: while (...) { // br1
Loop B:   while (...) { // br2
            if (...)    // br3
                ...;
        }
    }

```

Figure 3.11: An example code of a doubly-nested loop.

3.2.4 An example of incorrect alignment on a complex forward-branch misprediction

Figure 3.15 shows the control-flow graph of a loop in the 445.gobmk benchmark ⁴. Assume the processor has fetched along path {br1,br2,j1,br6,br1,br3,br4}. Now if br2 resolves as mispredicted, the squashed path is {j1,br6,br1,br3,br4}. When br4 is fetched into AQ, it leads to a latch on br4 in SQ. The alignment here is incorrect as the latched br4 in AQ belongs to the first iteration of the loop whereas the latched br4 in SQ belongs to the second iteration of the loop.

⁴The loop occurs at line: 3063, file: engine/owl.c. The control-flow graph shown here corresponds to the compiled version of the code.

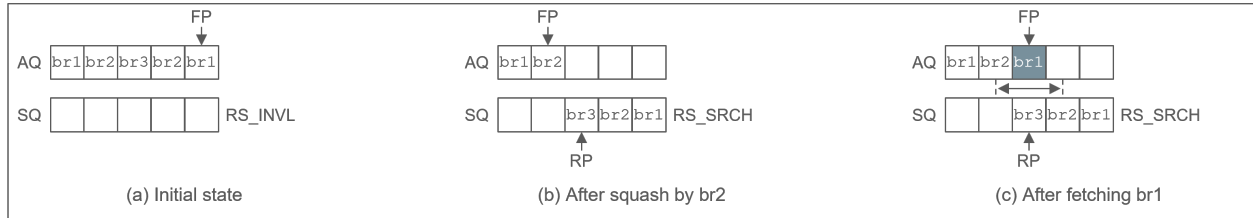


Figure 3.12: Alignment sequence when the squashed path for br2 is {br3,br2,br1}.

```

Loop A: while (...) {    // br1
Loop B:   while (...) { // br2
            if (...)    // br3
                break;
            ...;
        }
    }

```

Figure 3.13: An example code of a doubly-nested loop with a break.

3.2.5 Conclusion

We now understand how alignment works in the reuse-via-queue model. The alignment heuristics facilitate alignment on forward-branch mispredictions while avoiding a latch onto a different loop iteration as much as possible. A loop is inferred by observing another occurrence of the mispredicted branch in the SQ (via search radius criterion) or in the AQ (via the search termination criterion).

Alignment works well for what it is designed for: simple forward-branch mispredictions. Consequently, it doesn't work for loop-branch mispredictions. We also covered how the alignment might still break on a complex forward-branch misprediction. It breaks when *a mispredicted branch's control-dependent branch is not dominated by it* (br4 is not dominated by br2 in Figure 3.15).

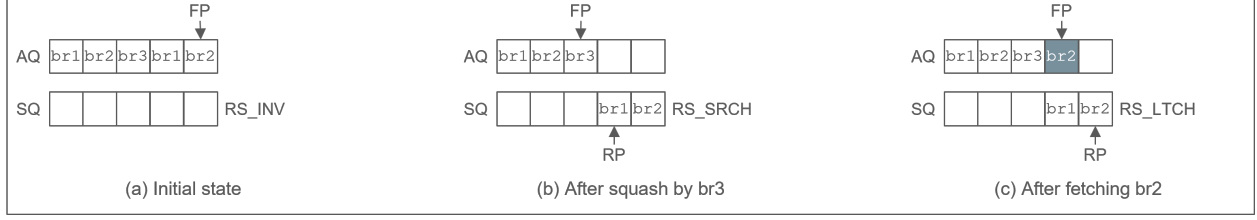


Figure 3.14: Alignment sequence when the squashed path for br3 is {br1, br2}.

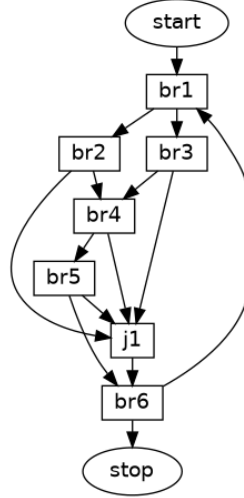


Figure 3.15: Control-flow graph of a loop in 445.gobmk benchmark.

3.3 Optimizations for reuse via queue

3.3.1 Confidence mechanism: overriding the default predictor

The SQ provides a squashed outcome only when: a) it is latched, and b) a squashed outcome is available at the latched entry. A squashed outcome, when available, should override the default prediction *if* the chances of the outcome to be correct are better than the chances of the prediction to be correct. We use per-branch (local) saturating counters to learn this behavior.

Figure 3.16 shows an implementation where the confidence counters are implemented within the BTB. Each BTB entry is extended to accommodate a n -bit confidence counter. When a branch is installed in the BTB, its confidence counter is initialized to 0. A squashed outcome may override the default prediction only if $counter > 2^{(n-1)} - 1$ (*i.e.*, deemed

confident when the counter is in the upper-half of its reachable states).

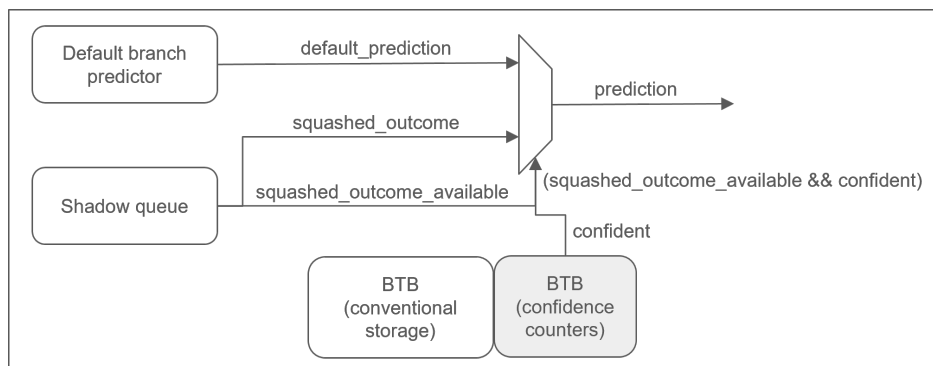


Figure 3.16: Using BTB confidence counters to override the default predictor.

Confidence counters are trained as follows. When a branch is fetched, the squashed outcome obtained from SQ and its default prediction, both are recorded in its AQ entry. The AQ entry also records the actual outcome when the branch executes. When the branch retires, its confidence counter is potentially updated based on four pieces of information: whether a squashed outcome is available, the squashed outcome (if available), the default prediction, and the actual outcome. The counter is updated only if a squashed outcome is available and it differs from the default prediction, as follows:

```

if (squashed_outcome_available && (squashed_outcome != default_prediction))
{
    if (squashed_outcome == actual_outcome) {
        if (counter < max_value) counter++;
    } else {
        if (counter > 0) counter--;
    }
}

```

BTB-based confidence counters have a trade-off. The upside is each confidence counter

needs just n bits of storage. The downside is there are as many counters as the number BTB entries which can be wasteful if the number of reliably-reusable branches is small. As an alternative, confidence counters can be implemented as a standalone PC-tagged cache with fewer entries in comparison to the BTB. A standalone confidence counter implementation is left for future work.

3.3.2 Delaying the squash action

In squashed-branch reuse, delaying the squash action (*i.e.*, deferring the removal of squashed-path instructions from the pipeline) can boost reuse coverage. A deferred squash must not delay the progress of resolved-path instructions in the pipeline, either directly (by delaying their fetch) or indirectly (by delaying their execution).

With that in mind, we propose a simple deferred squash mechanism. The squashed-path instructions before the rename stage are squashed immediately, ensuring resolved-path instructions are fetched unimpeded. The squashed-path instructions after the rename stage continue dispatching, issuing, and executing like usual, until the first resolved-path bundle reaches the rename stage. Once the first resolved-path bundle reaches the rename stage, the remaining squashed-path instructions are squashed, releasing the resources held by them. Our results assume this deferred squash mechanism unless stated otherwise.

In the period of deferment initiated by a branch brB, a misprediction may get posted by a branch younger than brB (let's call it brC) or a branch older than brB (let's call it brA). The misprediction posted by brC is simply ignored. The misprediction posted by brA ends the period of deferment for brB, squashing the remaining squashed-path instructions with respect to brB.

In absence of delayed squash, a baseline superscalar may still provide a meaningful reuse coverage due to out-of-order execution. Therefore, in principal, delayed squash is an optimization for out-of-order-scheduled processors. On the other hand, delayed squash is a necessity when squashed-branch reuse is implemented in in-order-scheduled processors.

3.3.3 Handling divergence in the control-independent region

A latched SQ remains latched until the prediction on the squashed path and the prediction on the resolved path differ. When the predictions differ, the paths are said to be diverged, terminating reuse (refer to the transition from RS_LTCH to RS_INVL in Figure 3.2). Such divergences may limit reuse coverage if not handled properly.

Divergences can be handled by modifying the alignment logic as follows. On a divergence:

- Increment RP and transition to RS_SRCH.
- Enforce the search radius criterion *w.r.t.* the *diverged* branch: when in RS_SRCH state, limit the search scope to the next dynamic instance of the diverged branch.
- Enforce the search termination criterion *w.r.t.* the *diverged* branch: when in RS_SRCH state, terminate reuse upon fetching the next dynamic instance of the diverged branch.

Figure 3.17 shows the updated alignment logic (the updates are marked in bold font). Note in this logic, barring the initial copy from AQ to SQ, a branch misprediction (which sets the reuse status to RS_SRCH for the first time) is handled identically to a divergence (which sets the reuse status to RS_SRCH subsequently).

3.4 Methodology

We evaluate the reuse-via-queue model using an in-house, RISC-V, execution-driven, execute-at-execute superscalar processor simulator. Default parameters of the baseline superscalar processor are shown in Table 3.1.

We compiled the SPEC 2006 and SPEC 2017 integer benchmarks and GAPBS benchmarks [BAP15] using LLVM (repository: [LLVc], branch: release/16.x, commit: 464bda7,

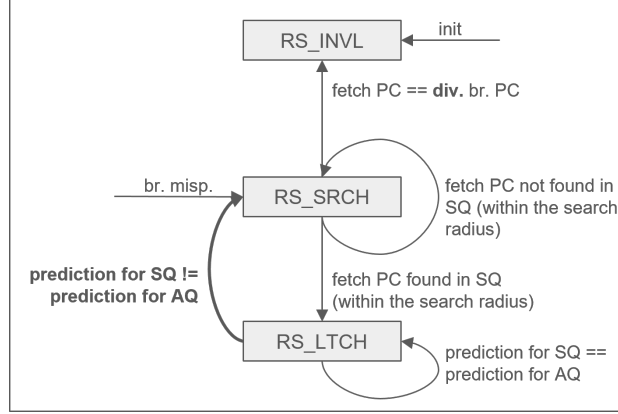


Figure 3.17: The updated alignment logic for handling divergences.

optimization flag: -O3). The SPEC 2017 benchmark, `exchange_2`, is not available as it has Fortran source, which our LLVM RISC-V compiler cannot compile.

For each SPEC benchmark, we use its ref input that has the highest weighted-average MPKI over all its SimPoints. For GAPBS benchmarks, we use three real-world input graphs: Road, Twitter, and Web [BAP15]. Up to ten 100-million-instruction SimPoints [She+02] were generated for each benchmark.

3.5 Results

3.5.1 Confidence mechanism

We varied a few key design parameters to evaluate different confidence schemes. Each scheme is encoded as $\langle A \rangle - \langle B \rangle - \langle C \rangle$ where:

- $\langle A \rangle$ can be ‘L’ (per-branch *local* confidence counters) or ‘G’ (a single *global* confidence counter).
- $\langle B \rangle$ can be ‘S’ (*Saturating* counters: increment/decrement, confident above mid-point threshold) or ‘R’ (*Resetting* counters: increment/reset-to-0, confident when at maximum).

Table 3.1: Default parameters of the baseline superscalar processor.

Parameter	Value
fetch-to-execute depth	12 stages
fetch/dispatch/issue/retire widths	8/8/8/8
execution lanes	2 ld/st, 4 simple ALU, 2 complex/fp ALU
ROB/PRF/LQ/SQ/IQ	512/576/256/256/128
branch checkpoints	64
squash model	delayed (Sec. 3.3.2)
branch predictor	64KB TAGE-SC-L [Sez16]
BTB	8K entries, 4-way
RAS	64 entries
L1 I\$	64KB, 4-way, 64B block
L1 D\$	64KB, 4-way, 64B block, 4-cyc. load-to-use
L2 \$	1MB, 8-way, 64B block, 10 cyc.
L3 \$	8MB, 16-way, 128B block, 30 cyc.
main memory latency	100 cyc.

- $\langle C \rangle$ can be ‘OP’ (*outcome*-correct and *prediction*-incorrect: gain conf.; *outcome*-incorrect and *prediction*-correct: lose conf.) or ‘O’ (*outcome*-correct: gain conf., *outcome*-incorrect: lose conf.).

Figure 3.18 shows the geomean speedup of the four local and the four global confidence schemes over baseline. All the schemes use 4-bit counters.

Across the local and global schemes, saturating counters perform well when trained based on the accuracy of squashed outcomes in comparison to default prediction (L/G-S-OP better than L/G-S-O). Whereas resetting counters perform better when trained based on the accuracy of squashed outcomes (L/G-R-O better than L/G-R-OP).

L-S-OP performs the best overall (speedup: 1.77%). L-R-O performs the second best overall (speedup: 1.65%). G-S-OP performs the third best overall (speedup: 1.61%).

Note G-S-OP/G-R-O work very well for GAP workloads but not so much for SPEC workloads. This is because in GAP workloads, most squashed-branch outcomes are reliable. Whereas in SPEC workloads, reliable squashed-branch outcomes are co-mingled with

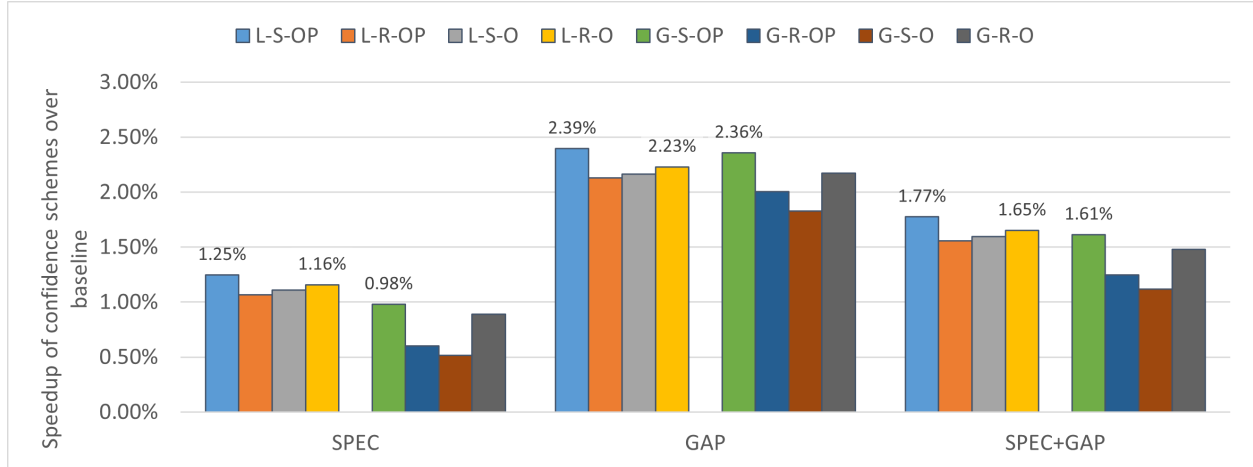


Figure 3.18: Speedup of various confidence schemes over baseline.

unreliable ones rendering a global counter ineffective.

Figure 3.19 shows the speedup of the L-S-OP scheme with reducing counter widths. Overall, 3-bit counters perform as well as 4-bit counters.

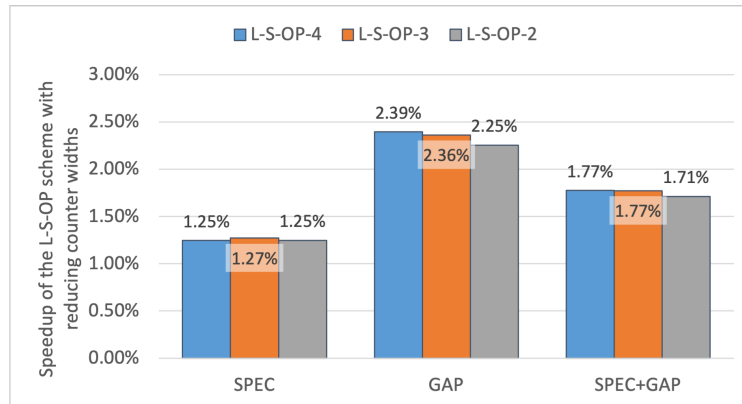


Figure 3.19: Speedup of the L-S-OP scheme with reducing counter widths.

We also implemented BRC’s gshare-like confidence predictor [ASL03]. In this predictor, the mispredicted branch PC (left-shifted by $m/2$ bits), the global branch history (m bits), and the fetched branch PC (m bits) are hashed together to form an m -bit index. This index is used to access a 2^m -entry tag-less table of 2-bit saturating counters.

Figure 3.20 shows the speedup of BRC’s confidence predictor when m is increased from 8 to 14 in steps of two (corresponding table sizes are: 256, 1024, 4096, and 16384). Similar to a gshare predictor, increasing the table size first improves performance due to reduced aliasing, but then hurts performance due to increased training time. Overall, table size 4096 (BRC-12) performs the best, achieving an overall speedup of 1.74%.

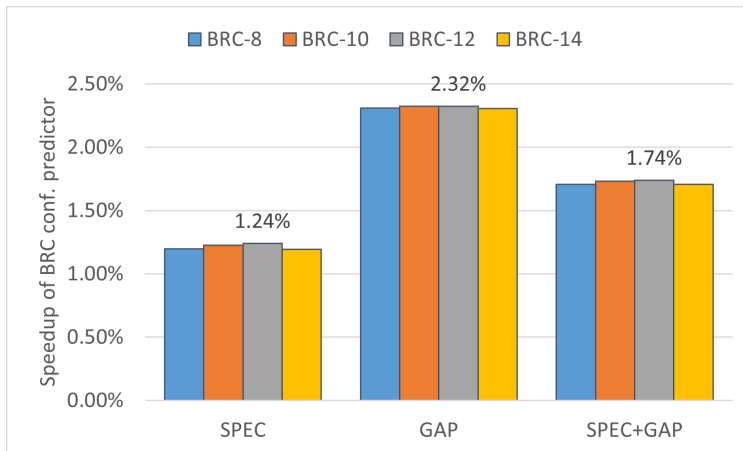


Figure 3.20: Speedup of the BRC confidence predictor with varying table sizes.

3.5.2 Impact of divergences

Figure 3.21 shows the reuse speedup, both without and with the support for divergences, using the L-S-OP-3 confidence scheme. The overall speedup goes from 1.77% without support for divergences (RVQ: reuse via queue) to 2.83% with support for divergences (RVQ-DIV). Note the speedup improvement is much more pronounced in GAP workloads (goes from 2.36% to 4.29%), suggesting a high percentage of reusable branches beyond the first divergence.

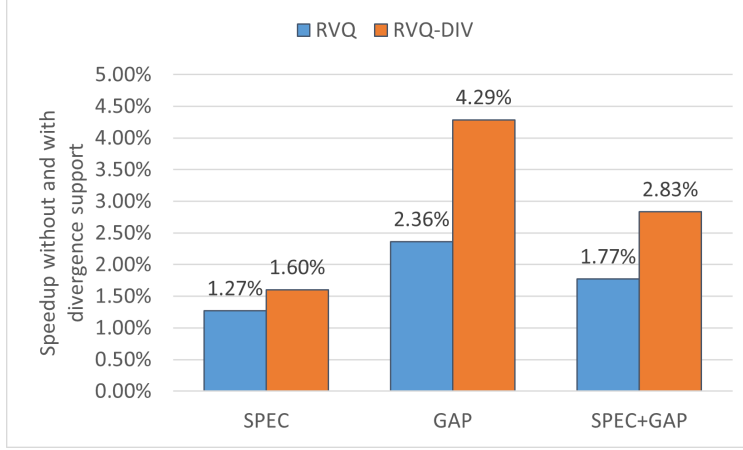


Figure 3.21: Performance improvement with support for divergences.

3.5.3 Impact of delayed squash

The results discussed so far assume the delayed squash mode. With delayed squash, reuse with support for divergences (RVQ-DIV) provides an overall speedup of 2.83% over its baseline. Whereas with immediate squash, the reuse speedup drops to 1.12% over its baseline⁵. It is clear that delayed squash is crucial for boosting reuse performance. Its benefit increases with increasing fetch-to-rename pipeline depth (the fetch-to-rename depth in our simulations is 8 cycles).

⁵Note the baseline performance is different for the two squash modes. The baseline with delayed squash mode performs 1.55% better than the baseline with immediate squash mode due to executing more CI loads (and initiating more cache misses) before the squash action.

Chapter 4

Squashed-branch reuse via signature stack and squashed-branch reuse buffer

4.1 Introduction

The reuse-via-queue model has a fundamental drawback:

- *Missed/incorrect alignment in case of loop-branch divergences and complex forward-branch divergences:* Both missed alignment and incorrect alignment are undesirable as they may reduce coverage and/or pollute confidence for otherwise reliably-reusable branches.

Rather than inferring loops, as an alternative, branches can be aligned using the merge point (the first control-independent instruction) of non-biased/divergent branches. Merge points can be learned in hardware [CTW04] or can be learned in the software and then supplied to the hardware [Kim+06]. But this alternative also comes with a trade-off: though it may be possible to learn the merge points (which can be multiple for some branches) for all divergent branches (including loop branches),

buffering and accessing these merge points for all divergent branches in the working set can be expensive ¹.

The reuse-via-queue model has other secondary drawbacks:

- *Unused squashed outcomes are dropped when a new misprediction is posted:* When a misprediction is posted in the window, SQ might already be in use for an earlier misprediction squash. In this case, SQ is flushed and set-up for the new misprediction squash, terminating reuse for the earlier misprediction squash.

It is possible to eliminate the above-mentioned structural hazard by using multiple SQs (similar to [KC25]). Apart from the added hardware cost, a multi-SQ design is hard-to-optimize as the optimal number of SQs is highly dependent on the workload and its execution phase.

- *SQ is implemented as a priority-encoded CAM (content-addressable memory):* When a branch PC is searched in the SQ, there can be zero or multiple hits. A latch should occur only when a matching entry lies within the search radius. This functionality requires the use of a priority-encoded CAM, which has higher power and timing overheads compared to a RAM (random-access memory).

In this chapter, we look at a new method for aligning branches. In this method, each dynamic instance of a branch is *uniquely* identified using a *hierarchical loop iteration identifier*. As the name suggests, the identifier of a branch carries the iteration count of not only the loop it appears in (directly), but also the parent loops it appears in (indirectly). For example, the identifier of a branch within a double-nested loop (B nested within A) is $\{A.iterA, B.iterB\}$. Note, *uniqueness* is obeyed for every branch within a function.

The identifier is maintained in the fetch unit where it is: a) assigned to branches when they are fetched, b) speculatively updated on loop-related edges, and c) precisely rolled-back on a misprediction. The identifier of a dynamic branch instance is also *path-invariant*:

¹Ideally we need join nodes instead of merge points for the purpose of alignment, though the two can be the same for many branches. See Section 4.2.4 for details.

the identifier of a dynamic branch instance on the resolved path matches with its counterpart on the squashed path (if it exists), despite arbitrary differences in control-flow observed on the two paths. Note, *path-invariance* is obeyed for every dynamic branch instance of a function. *Uniqueness* and *path-invariance*, together, play a central role in simplifying branch alignment.

Section 4.2 provides a formal description of the identifier, describes a scheme to realize the identifier in hardware with the compiler’s assistance, and shows how the use of identifiers simplifies squashed-branch reuse:

- Section 4.2.1 discusses loops from a compiler’s perspective, and looks at how they occur in a program. Using this knowledge, Section 4.2.2 formally defines the *hierarchical loop iteration identifier*. Section 4.2.3 shows how the identifier is manipulated using basic stack operations triggered by special control-flow edges. Section 4.2.4 shows how identifiers simplify branch alignment using an example.
- Section 4.2.5 describes an implementation which realizes the identifier as a *Literal Stack* in the hardware. It involves conveying the special control-flow edges in a program’s binary and inferring them during the program’s execution.
- The identifier is exposed to the pipeline as a *signature*. Every dynamic branch instance can be uniquely identified using a *key*, which is a combination of its signature and its PC. Section 4.2.6 introduces the *Squashed-Branch Reuse Buffer*: a reuse buffer implemented as a set-associative cache, indexed and tagged using keys. Reuse via Literal Stack and Squashed-Branch Reuse Buffer overcomes the major drawbacks of the reuse-via-queue model.

Though the Literal Stack is straightforward to implement, it is inefficient at supplying signatures. Section 4.3 proposes a new design for signature generation, called *Signature Stack*. We present two Signature Stack implementations, and discuss how they couple with the Squashed-Branch Reuse Buffer (SBRB) in realizing efficient squashed-branch reuse.

- In Signature Stack, an ongoing signature is maintained in a fetch unit register, that by itself represents (in compressed form) the current identifier. Section 4.3.1 discusses the Signature Stack design, introduces the different components involved in the design, and lays out a framework for manipulating a Signature Stack that is independent of its implementation.
- Section 4.3.2 discusses *incrementer-based Signature Stack*: our first Signature Stack implementation. In this implementation, whenever a loop is entered, it is allocated a unique segment of signatures for its iterations. This implementation works well with a fully-associative SBRB, but not necessarily with other SBRB organizations.
- Section 4.3.3 discusses *LFSR-based Signature Stack*, an implementation which effectively decouples Signature Stack from SBRB. Section 4.3.4 summarizes the final storage cost of the squashed-branch reuse implementation using LFSR-based Signature Stack and SBRB, arrived upon after a thorough design space exploration.
- Section 4.4 covers the methodology. Section 4.5 discusses the results.

4.2 Hierarchical loop iteration identifier: concept and realization

4.2.1 Loops and loop hierarchy

In this work, we take a compiler’s view of a loop [LLVb]. In this view, every loop in a program can be identified with a unique header block. The header block represents a single point of entry into the loop, and therefore dominates the rest of the blocks in the loop.

Figure 4.1 shows the control-flow graph (CFG) of the loops in the BUStep function of

bfs, a program from the GAPBS benchmark suite [BAP15]². There are two loops. The inner loop contains blocks D and E, and is identified with header block D. The outer loop contains blocks B, C, F, and G directly, and blocks D and E transitively. It is identified with header block B.

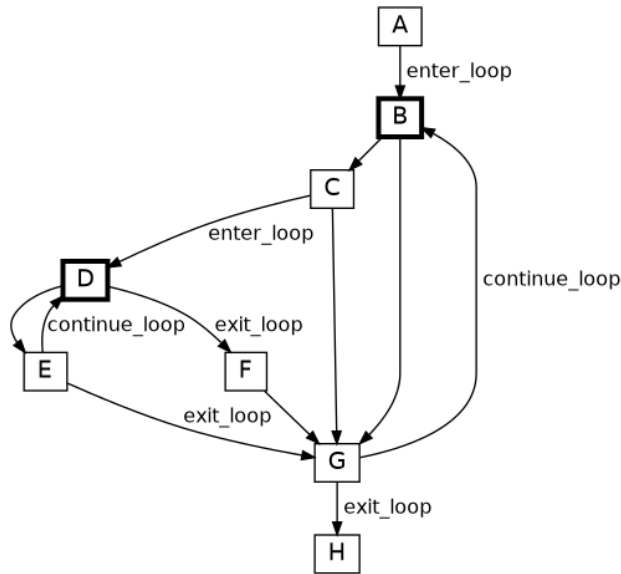


Figure 4.1: Control-flow graph of loops in bfs BUSStep.

For a loop, iteration count is defined as the number of executions of the header block before exiting the loop [LLVb]. The presence of a unique header block differentiates loops from other non-loop cyclic control-flow in a program. For non-loop cycles, there isn't a uniquely identifiable header block. Therefore, the iteration count is not well-defined as it depends on the choice of the header block. Non-loop cycles can occur due to manually-optimized assembly programming, unconventional programming constructs like *goto*, certain compiler transformations, *etc.* We expect their presence to be limited compared to loops and do not consider them further.

Nested loops form a hierarchy which can be represented as a tree [LLVb]. The outermost

²The loop occurs at line: 51, file: bfs.cc. The control-flow graph shown here corresponds to the compiled version of the code.

loop appears at the root of the tree. Figure 4.2 sub-figure:a shows the tree of loops of the bfs BUStep function. Note, here the loops are identified with their respective header blocks.

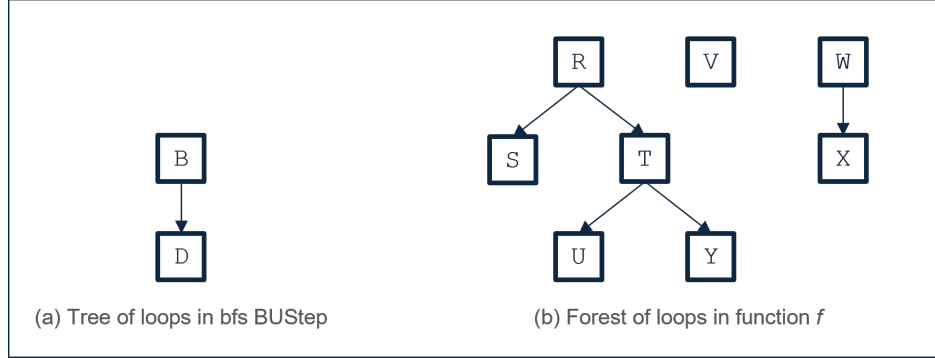


Figure 4.2: Loop hierarchy expressed as a forest.

Since a function can have multiple outermost (top-level) loops, the loops in a function form a forest (*i.e.*, a collection of trees) [LLVb]. Figure 4.2 sub-figure:b shows a forest of loops in some function f . Every loop has a loop depth associated with it. The outermost loop has a loop depth of 1, and the depth increases downwards in a tree. The loop depth of a basic block (which has at most one branch) is the depth of the (innermost) loop it appears in. A top-level branch has a loop depth of 0.

4.2.2 Hierarchical loop iteration identifier

Definition 4.1 (Hierarchical loop iteration identifier). The hierarchical loop iteration identifier of a dynamic instance of a branch directly inside loop L constitutes: (a) the loop identity (*i.e.*, the header block) and (b) the iteration count, of all the loops which are ancestor to L , including L .

For example, the identifier for a dynamic branch occurring in the loop U of function f is $\{R.iterR, T.iterT, U.iterU\}$. We can now define branch alignment using identifiers.

Definition 4.2 (Alignment (via identifiers)). A branch R on the resolved path is

said to be aligned with a branch S on the squashed path if: $(PC_R, identifier_R) == (PC_S, identifier_S)$.

4.2.3 The identifier is a stack

The identifier is a stack of header block and iteration count pairs. The identifier is manipulated on three special control-flow edges. For a loop L , these edges are:

- *enter_loop*(L): an edge from a block outside L to its header block.
- *continue_loop*: an edge from a block inside L to its header block.
- *exit_loop*(pop_amount): an edge from a block inside L to a block outside L . The pop_amount is the difference between the loop depth of L and that of the edge's destination block. The destination block usually belongs to the parent loop of L or is a top-level block if L itself is a top-level loop. In both cases, pop_amount is one. In other cases, the pop_amount is more than one.³

Once the required edges are determined for all the loops in a function, manipulating the identifier is straightforward as captured in Table 4.1.

Table 4.1: Identifier manipulations triggered on special control-flow edges.

Edge	Stack operation
<i>enter_loop</i> (L)	<code>push({L, 1});</code>
<i>continue_loop</i>	<code>top().iteration_count++;</code>
<i>exit_loop</i> (pop_amount)	<code>while (pop_amount--) pop();</code>

³The discussion assumes that the destination block either belongs to a loop that is an ancestor of L or is a top-level block. Otherwise, the pop_amount is one more than the difference between the loop depth of L and that of the edge's destination block.

An example of identifier manipulations

Figure 4.3 shows stack manipulations on a dynamic sequence of edges for the bfs BUStep function (refer to Figure 4.1). The processor has fetched along path {A,B,C,G,B,C,D,E,D,F,G,H}. The outer loop (B) is iterated twice. The inner loop (D) is not visited in the first iteration of B, and is iterated twice in the second iteration of B. The exact sequence of edges is captured in the first row. The identifier is manipulated on the special edges. The identifier transitions from {} to {B.1} (*enter_loop(B)*), {B.1} to {B.2} (*continue_loop*), {B.2} to {B.2, D.1} (*enter_loop(D)*), {B.2, D.1} to {B.2, D.2} (*continue_loop*), {B.2, D.2} to {B.2} (*exit_loop(1)*), and {B.2} to {} (*exit_loop(1)*).

Edge sequence	Init	A->B	B->C	C->G	G->B	B->C	C->D	D->E	E->D	D->F	F->G	G->H
Edge specialty	-	enter_loop(B)	-	-	continue_loop	-	enter_loop(D)	-	continue_loop	exit_loop(1)	-	exit_loop(1)
Stack op.	-	push({B,1})	-	-	top().iter++	-	push({D,1})	-	top().iter++	pop(1)	-	pop(1)
Identifier (after op.)	{}	{B.1}	{B.1}	{B.1}	{B.2}	{B.2}	{B.2,D.1}	{B.2,D.1}	{B.2,D.2}	{B.2}	{B.2}	{}

Figure 4.3: An example of identifier manipulations in bfs BUStep.

Dealing with overlapping special edges

In the bfs BUStep function, there is no case where the same edge represents a special edge for one loop and a *different* special edge for another loop. But in general, there may be such overlapping special edges. When there are overlapping special edges, there can be at most one *enter_loop* or *continue_loop* among them because these special edge types are unique: they are unique for a loop due to distinct source blocks (outside *vs.* inside the loop, respectively), and they are unique across loops due to distinct destination blocks (*i.e.*, the header blocks). Thus, valid overlap scenarios are when the same edge represents one or more *exit_loop* edges of various pop_amounts, and optionally a single *enter_loop* or *continue_loop* edge.

If there is an overlap, the identifier may need to be manipulated more than once. If the overlap involves just *exit_loop* edges, then the identifier is manipulated once, based on the highest *pop_amount*. If the overlap involves an *enter_loop* or a *continue_loop* edge, the identifier is manipulated twice: first for the *exit_loop* edge(s) and then for the *enter_loop/continue_loop* edge.⁴

4.2.4 Branch alignment using the identifier

We now look at an example of a loop branch misprediction. The example showcases the strength of identifier-led hierarchical alignment over heuristic-driven linear alignment (as covered in Chapter 3).

Consider the bfs BUStep function (refer to Figure 4.1). The processor has fetched along path {A,B,C,D,E,G,B,C,D,F,G,H}. The outer loop (B) is iterated twice. The inner loop (D) is iterated once for both the iterations of B. Figure 4.4 sub-figure:a shows the setup for linear alignment ⁵.

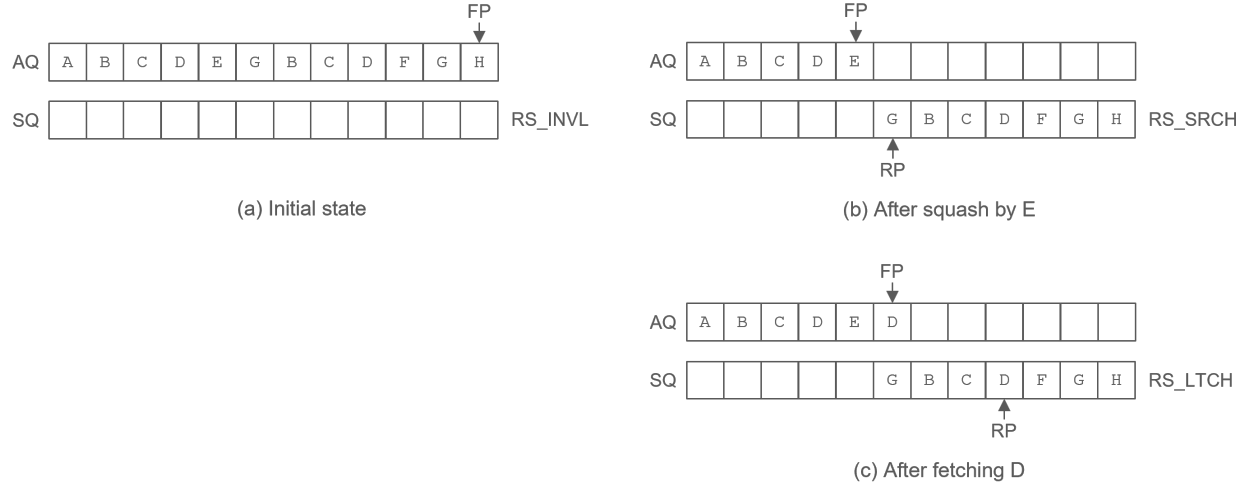


Figure 4.4: Heuristic-driven linear branch alignment.

⁴Special edge overlaps can be reduced or eliminated using certain special compiler passes. The techniques explained here are general and do not require any such compiler pass.

⁵In this example, we are using the basic block symbols instead of the branches for convenience. All basic blocks except A, F, and H, indeed end in a conditional branch.

Now assume that loop D is under-iterated in the first iteration of B: it should have iterated twice, but it only iterated once. Given that, when branch E resolves as mispredicted (sub-figure:b), and branch D is fetched on the resolved path, it causes a latch (sub-figure:c). The alignment here is incorrect as the latched instance on the resolved path belongs to the first iteration of B whereas the latched instance on the squashed path belongs to the second iteration of B.

Let's look at how alignment is achieved using identifiers. Figure 4.5 shows the initial fetch sequence. The blocks are height-adjusted according to their loop depth, and grouped based on their identifier. The special control-flow edges mark the transition from source identifier group to the destination identifier group.

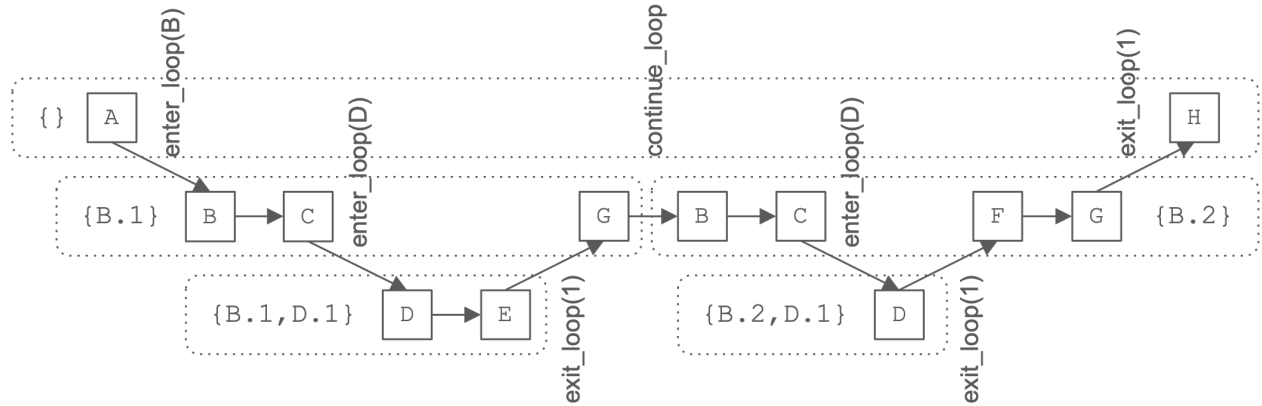


Figure 4.5: Identifier-led hierarchical branch alignment (initial setup).

When branch E resolves as mispredicted, and a new iteration needs to be added, a new identifier ($\{B.1, D.2\}$) is created (Figure 4.6). When loop D is eventually exited (via edge E to G), the identifier is restored to $\{B.1\}$. The identifier of G fetched on the resolved path matches with its correct instance of G on the squashed path. The identifiers on the two paths remain aligned until further divergences (there are no other divergences in this example).

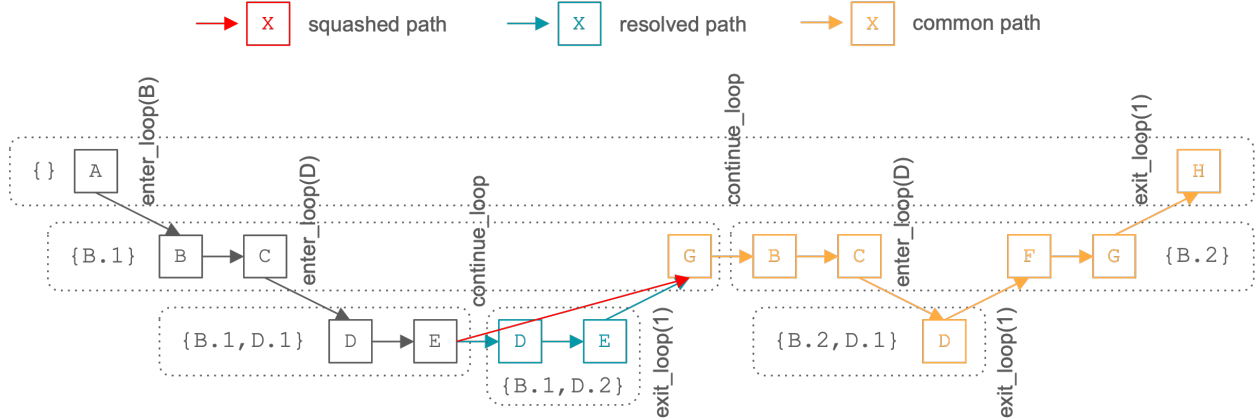


Figure 4.6: Identifier-led hierarchical branch alignment (after squash by E and fetching D, E, G, ...).

We would like to highlight a couple of important facts about branch alignment using identifiers:

- *Upon a divergence, convergence occurs (implicitly) at the earliest program point:* Convergence, which is marked by the first instance of alignment after a divergence, occurs implicitly when the $(PC, identifier)$ pair on the resolved path matches with an instance on the squashed path. This program point does not always correspond to the merge point described in techniques that exploit control independence, although the two can be the same for many dynamic branches. Instead, this program point corresponds to a thread join node in SIMD execution [LLVa; RMH21].
- *The stack-property of the identifier is fundamental for isolating loop-related divergences:* When the squashed path and the resolved path diverge, the divergence may or may not be related to loops. Loop-related divergences are said to occur when some squashed-path/resolved-path branches before the merge point, have a different identifier than the source diverged branch.

A loop-related divergence can occur when a loop is initially under-iterated or over-iterated (for example, when a dynamic branch E diverges in the bfs BUSTep function). It can also occur when a loop is initially visited or not visited, when guarded by a

forward branch (for example, when a dynamic branch C diverges in the bfs BUStep function).

Due to its stack-property, the identifier observed after exiting a loop always matches with the identifier observed before entering the loop, despite: a) an arbitrary number of iterations of the loop, and b) whether or not other loops were visited within the loop. Therefore, the identifier not only provides unique names for new loop iterations and new loop visits, but at the same time, it provides a *consistent* name upon exiting a loop. This consistency, stemming from the stack-property of the identifier, is fundamental for isolating loop-related divergences, and ensuring convergence after any such divergence when the corresponding loop exits.

4.2.5 Realizing the identifier as a Literal Stack

The identifier can be implemented in hardware as a stack of header PC (*i.e.*, the first PC of a header block) and iteration count pairs. We call it a Literal Stack. Figure 4.7 shows an example stack with a depth of three. The compiler needs to convey the special control-flow edges in the program’s binary, so that hardware can manage the stack at runtime.⁶

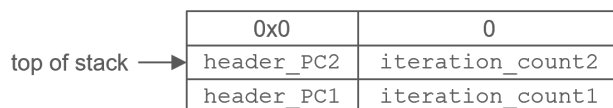


Figure 4.7: A Literal Stack with a depth of three.

Conveying the special control-flow edges to the hardware

The source of an *exit_loop* edge is a block that ends in a conditional branch. So an *exit_loop* edge is conveyed by its corresponding branch’s PC, the exit direction, and the

⁶It may be possible to infer loops, hence, special control-flow edges, using a hardware-only scheme. We leave this for future work.

pop_amount.

However, the source of an *enter_loop* edge or a *continue_loop* edge may not be a branch instruction (in the case where the edge is a fall-through edge). Still, the destination of all such edges is the header PC. Moreover, an *enter_loop* edge can be differentiated from a *continue_loop* edge reaching the same destination header PC by comparing the destination header PC with the header PC on the top of the stack: a mismatch signifies *enter_loop* and a match signifies *continue_loop*. So, all *enter_loop* and *continue_loop* edges for a loop are succinctly conveyed by the loop’s header PC.

In this work, the header PCs and the exit branches (composed of branch PC, exit direction, and pop_amount) are conveyed in a dedicated program segment, called the Loop Information Segment (LIS). When a process is launched or resumed after a context-switch, the LIS is copied into the Loop Information Table (LIT). The LIT is sub-divided into two tables, one for header PCs (LIT-H) and the other for exit branches (LIT-E). Both LIT-H and LIT-E are searched by PC. As an alternative, the header PCs and exit branches can also be conveyed using in-lined hint instructions. We leave this for future work.

Manipulating the Literal Stack during a program’s execution

At runtime, a hit in LIT-H is relayed as *hit_header*(PC). A hit in LIT-E is relayed as *hit_exit*(pop_amount) only if the prediction for that exit branch matches with the exit direction. Table 4.2 shows the Literal Stack manipulations based on the hit information from the LIT. Please note the equivalence with Table 4.1.

In addition to LIT hits, the Literal Stack is also manipulated on call and return instructions (the existing BTB signals calls and returns). This is necessary to ensure unique identifiers for branches across different potential invocations of the same function, from the same loop and iteration. A call causes a stack-push, initializing the “header PC” to 0 (to avoid collisions with valid loop header PCs) and the “iteration count” to the call instruction’s PC (PC of the call site). A return causes a pop. Table 4.3 captures the manipulations

Table 4.2: Literal Stack manipulations based on LIT hits.

LIT hits	Stack operations
<i>hit_header</i> (PC)	<pre> if (PC != top().header_PC) // enter loop push({PC, 1}); else // cont. loop top().iteration_count++; </pre>
<i>hit_exit</i> (pop_amount)	<pre> // exit loop while (pop_amount--) pop(); </pre>

based on BTB hits.

Table 4.3: Literal Stack manipulations based on BTB hits.

BTB hits	Stack operations
<i>hit_call</i> (PC)	<code>push({0, PC});</code>
<i>hit_return</i>	<code>pop();</code>

4.2.6 Reuse via Literal Stack and Squashed-Branch Reuse Buffer

The Literal Stack embodies the identifier. For practical purposes, the identifier is exposed to the pipeline as a *signature* which is a short fixed-length representation of the identifier. Signatures are different for the same branch PC in different loop iterations, but they are the same for different branch PCs in a given loop iteration. Signatures are combined with PCs to generate *keys* which are unique for every dynamic branch instance. Signature and key generation is captured in Figure 4.8.

Now branch alignment can be expressed in terms of keys as follows.

Definition 4.3 (Alignment (via keys)). A branch R on the resolved path is said to be aligned with a branch S on the squashed path if: $(key_R) == (key_S)$.

The use of keys allows the reuse buffer to be implemented as random access memory (RAM), and we call it *Squashed-Branch Reuse Buffer* (SBRB). SBRB is a set-associative

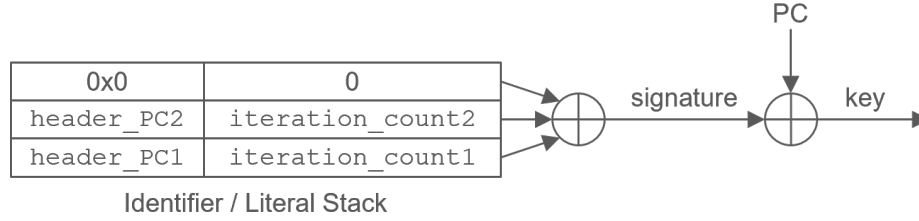


Figure 4.8: Generating signature and key from Literal Stack.

cache, indexed and tagged using keys. Figure 4.9 shows the squashed-branch reuse microarchitecture using Literal Stack and SBRB.

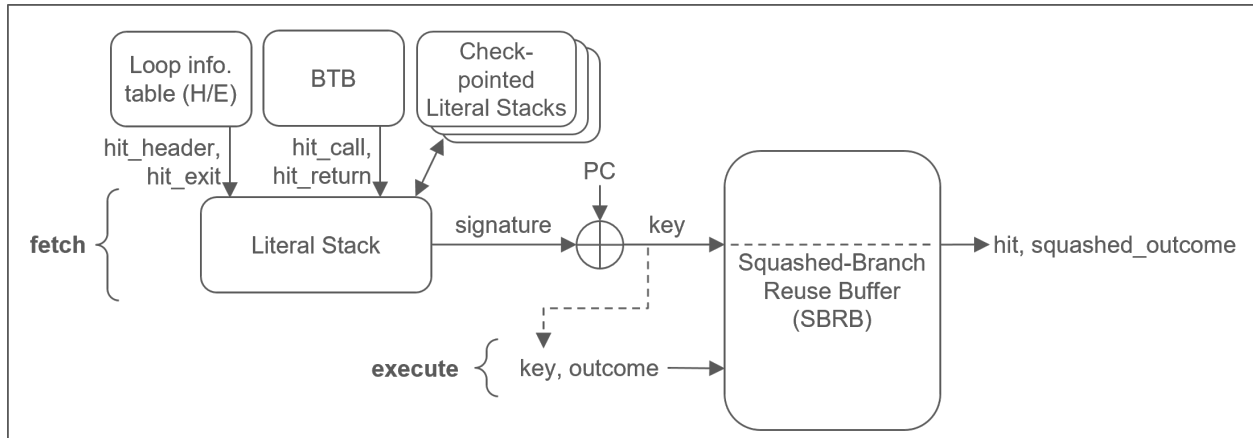


Figure 4.9: Squashed-branch reuse microarchitecture using Literal Stack and Squashed-Branch Reuse Buffer.

The fetch unit maintains the Literal Stack using inputs from the Loop Information Table (LIT) and the BTB, and assigns a unique key to every fetched branch. When a branch executes, its key is used to deposit its outcome to the SBRB (execute data-path). On a misprediction, the Literal Stack is precisely rolled-back using a checkpointed Literal Stack. When a branch is fetched, the SBRB is searched for a squashed outcome using its key (fetch data-path). On a hit, the squashed outcome may override the default prediction depending on the input from the confidence predictor.

Reuse via Literal Stack and SBRB has the following benefits over reuse via queue:

- *Perfect alignment using path-invariant identifiers:* Branch alignment is near-perfect ⁷. Compared to the reuse-via-queue model, it is oblivious to forward-branch divergences and loop-branch divergences.

- *Squashed outcomes are retained across squashes (SBRB is oblivious to squashes), and they are evicted gracefully:* When an outcome is deposited in the SBRB using a key, it stays in the SBRB until it is evicted by another outcome with a different key. It should be noted that a dynamic branch instance may deposit an outcome multiple times (using the same key) in case it is squashed and re-fetched more than once.

Just like other superscalar structures, the SBRB is optimized empirically by performing a design space exploration of its constituent parameters. The goal here is to minimize the occurrences where an outcome is evicted before its corresponding branch is fetched for the *last time*.

In SBRB, squashed outcomes from several prior squashes may co-exist within the SBRB transparently. This allows better utilization of the reuse buffer compared to the reuse-via-queue model where the utilization may drop significantly for each additional SQ.

- *SBRB is implemented as a RAM:* A RAM-based reuse buffer is easier to scale-up compared to a CAM-based buffer.

4.3 Reuse via Signature Stack and Squashed-Branch Reuse Buffer

To compute a signature using the Literal Stack, *all* the bits in the stack have to be suitably hashed together. This is not a pragmatic design because the hash needs to be performed whenever the stack is modified (essentially every cycle in the worst case). Also, while

⁷Branch alignment may still break in presence of non-loop cyclic control-flows.

hashing of PCs with XOR preserves adequate information, it is less clear how to effectively hash iteration counts.

4.3.1 Signature Stack: introduction

Just like the hierarchical loop iteration identifier, the signatures generated from the Literal Stack obey the following properties:

- *Uniqueness*: Every dynamic instance of a branch is assigned a unique signature. This holds for every branch within a program.
- *Path-invariance*: The signature of a dynamic branch instance on the resolved path matches with its counterpart on the squashed-path (if it exists), despite arbitrary differences in control-flow observed in the two paths. This holds for every dynamic branch instance of a program.

Therefore, instead of maintaining a Literal Stack (and generating a new signature from it after every modification), for branch alignment, it is sufficient to maintain an ongoing signature, that by itself represents (in compressed form) the current identifier, assuming it is manipulated appropriately to satisfy the above two properties.

Further, in Section 4.2.4, we observed that the stack-nature of the identifier is fundamental for isolating loop-related divergences. Therefore, we do need a stack in the new design, and this stack should be used to save the signature before entering a loop and restore the signature after exiting a loop.

Lastly, we still need the header PC, both for the current loop and the saved loops, to infer special control-flow edges.

So in the new design, we have:

- A register (*hpc*) that stores the header PC of the current loop.
- A register (*sig*) that stores the current signature.

- A stack to save/restore *hpc* and *sig*.

We call this design Signature Stack (SS). The SS components are shown in Figure 4.10⁸.

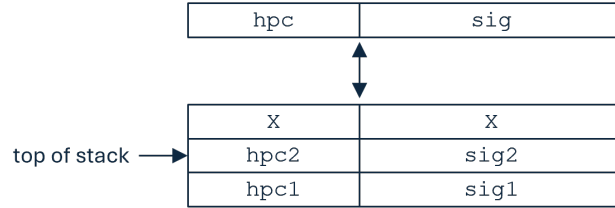


Figure 4.10: Signature Stack.

Before discussing any particular SS implementation, we describe a general SS manipulation framework which covers the save/restore stack manipulations as well as *hpc* manipulations (as these are meant to be consistent across implementations), and leaves out *sig* manipulations (as these vary with implementations):

- Entering a loop: A loop is entered when a loop header PC is observed that differs from the *hpc*. First, the values of $\{hpc, sig\}$ are saved by pushing them to the save/restore stack. This allows updating *sig* for the newly-entered loop, while still being able to revert back to the *sig* that existed before this loop, when it is eventually exited. Second, *hpc* is set to the entered-loop's header PC. Lastly, *sig* is updated (by some means; ignored for now).
- Continuing a loop: A loop is continued when a loop header PC is observed that matches with the *hpc*. In this case, *sig* is updated (by some means; ignored for now).
- Exiting a loop: The save/restore stack is popped the requisite number of times (pop amount) and $\{hpc, sig\}$ are restored from the final popped stack entry.

⁸We also have SS checkpoints, to precisely-rollback SS on a misprediction (not shown in the figure). A SS checkpoint contains the entire SS, *i.e.*, the two registers and the save/restore stack.

- Entering a function: First, the values of $\{hpc, sig\}$ are saved by pushing them to the save/restore stack. Second, hpc is set to 0. Lastly, sig is updated (by some means; ignored for now).
- Exiting a function: The save/restore stack is popped, restoring $\{hpc, sig\}$ from the popped stack entry.

Table 4.4 summarizes the Signature Stack manipulation framework. Now we just need to implement sig updates related to entering a loop, continuing a loop, and entering a function (which are marked by ?? in the table).

Table 4.4: Signature Stack manipulation framework.

LIT and BTB hits	SS operations
<i>hit_header</i> (PC)	<pre> if (PC != hpc) { // enter loop push({hpc, sig}); // save hpc = PC; sig = ??; } else { // cont. loop sig = ??; } </pre>
<i>hit_exit</i> (pop_amount)	<pre> // exit loop while (pop_amount--) pop({hpc, sig}); // restore </pre>
<i>hit_call</i> (PC)	<pre> push({hpc, sig}); // save hpc = 0; sig = ??; </pre>
<i>hit_return</i>	<pre> pop({hpc, sig}); // restore </pre>

4.3.2 Incrementer-based Signature Stack

We begin with the sig update on continuing a loop. A straightforward way to update sig is to simply increment it. For a n -bit sig , we do not need a n -bit-wide incrementer. Instead, the width of the incrementer depends on M , where M is the number of entries in the SBRB.

In the worst case, when there is only one branch in a long running loop, we would like to assign unique signatures to M consecutive dynamic instances of the branch, so that these instances can utilize the entire SBRB. Therefore, we only need a m -bit-wide incrementer, where $m = \lceil \log_2(M) \rceil$.

We also need to update *sig* upon entering a loop. If we leave the *sig* unmodified, there is a problem for multiply-nested loops: the signatures obtained for iterations of a loop B for a given iteration of its outer loop A almost entirely overlap with the signatures obtained for iterations of B in the neighboring iterations of A. For instance, the signature for $\{A.1, B.2\}$ collides with $\{A.2, B.1\}$, $\{A.1, B.3\}$ collides with $\{A.2, B.2\}$, and so on. When signatures collide for two different iterations, the signature of every branch from the first iteration collides with the signature of the corresponding branch (with same PC) from the second iteration.

To avoid this problem, we divide the space of 2^n states of the n -bit *sig* into a series of segments where each segment encompasses 2^m states. Whenever a loop is visited, we allocate a new segment to it, while ensuring this segment does not get assigned to other visits to the loop. We accomplish this using a left-shifter: upon entering a loop, a new segment is allocated by left-shifting *sig* by m bits. We get different segments on different visits to the loop because the location of a given segment depends on the value of *sig* before entering the loop.

We illustrate segment and signature allocation for doubly-nested loops (B nested within A) (see Figure 4.11). The figure shows a 2^n state space logically divided into segments. When A is entered, segment x is allocated to it: after being left-shifted, *sig* points to the first slot in segment x . When B is entered in the first iteration of A, it is allocated segment y (left-shifted *sig* points to the first slot in y). With each continuation of B, *sig* is incremented by one. When B exits, *sig* is restored to the value it had before entering B (via a stack pop). Like before, *sig* is incremented when A is continued. When B is entered in the second iteration of A, it is allocated segment $y + 1$. Note, this segment is adjacent

to B's segment in the first iteration of A because the signatures of $\{A.1\}$ and $\{A.2\}$ differ by one.

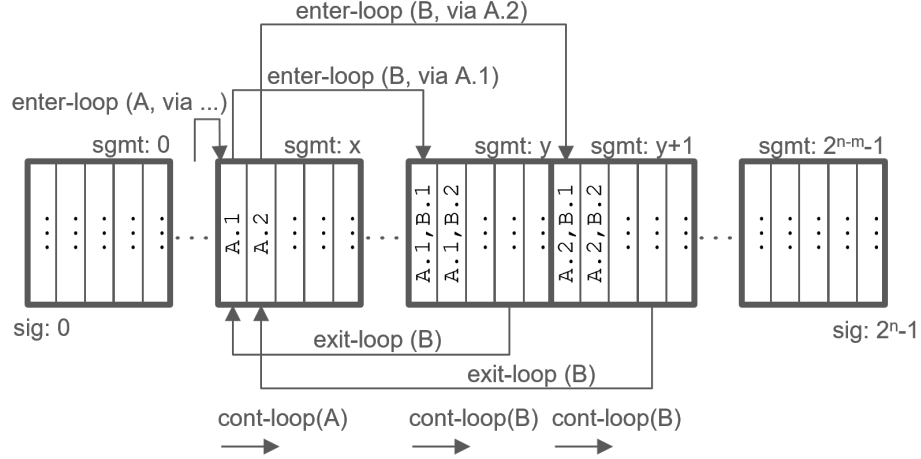


Figure 4.11: Segment and signature allocation in a linearly-evolving state space for doubly-nested loops (B nested within A).

On function enter, we simply hash the existing *sig* with the caller PC to obtain a new *sig*.

Figure 4.12 shows the logic for updating *sig* on entering/continuing a loop, and entering a function.

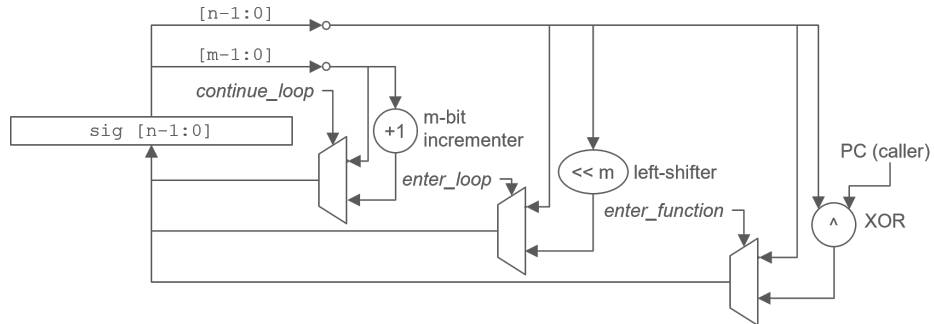


Figure 4.12: Logic for updating *sig* on entering/continuing a loop and entering a function.

A full description of incrementer-based SS is given in Table 4.5 (*sig* updates on entering/continuing a loop, and entering a function, are annotated with comments).

Table 4.5: Incrementer-based SS management.

LIT and BTB hits	SS operations
<i>hit_header</i> (PC)	<pre> if (PC != hpc) { // enter loop push({hpc, sig}); // save hpc = PC; // <<left-shift by m>> sig = sig << m; } else { // cont. loop // <<m-bit increment>> sig[m-1:0] = sig[m-1:0] + 1; } </pre>
<i>hit_exit</i> (pop_amount)	<pre> // exit loop while (pop_amount--) pop({hpc, sig}); // restore </pre>
<i>hit_call</i> (PC)	<pre> push({hpc, sig}); // save hpc = 0; // <<XOR>> sig = sig ^ PC; </pre>
<i>hit_return</i>	<pre> pop({hpc, sig}); // restore </pre>

Until now, we focused on minimizing signature collisions on different instances of the same branch. But ultimately, we are interested in minimizing key collisions. With that in mind, we should not simply hash the branch PC with the *sig* to generate a key. Otherwise, the key assigned to a branch with PC X in iteration i of a loop will collide with the branch with PC $X - d$ in iteration $i + d$ of the loop.

To avoid such collisions, we propose to first reverse a branch's PC (obtained by swapping the bit at distance x from the LSB with the bit at distance x from the MSB, for every x in the lower half of the PC) and then hash it with its *sig* to generate a key: $\text{key} = \text{sig} \wedge \text{rev}(\text{PC})$.

This concludes the discussion of the design of incrementer-based SS. Incrementer-based

SS is easy to couple with a fully-associative SBRB, but it is unclear how to efficiently couple it with a set-associative SBRB. The issue is discussed below.

At one extreme, there can be a long-running loop with a single branch in it, in which case, different instances of the same branch PC should equally spread across all the sets in the SBRB. This requires using the loop-iteration-related bits of the key (*i.e.*, the lower k -bits of the key) as index bits to access the SBRB. At the other extreme, there can be a long loop with many branches in it, in which case, instances of different branch PCs should equally spread across all the sets in the SBRB. This requires using the branch-PC-related bits of the key (*i.e.*, the upper k -bits of the key) as index bits to access the SBRB. Given these two opposing requirements, it is unclear how to pick the index and tag bits from the key to access a set-associative SBRB ⁹. Another undesirable aspect of incrementer-based SS is that the design of SS is coupled with the design of SBRB: the incrementer and left-shifter logic depend on the size of the SBRB.

4.3.3 LFSR-based Signature Stack

To use a set-associative SBRB out-of-the-box, we need randomly-varying signatures (and keys). To serve this requirement, we propose to implement *sig* as a Linear Feedback Shift Register (LFSR). In this work, we use a Galois LFSR [LFS; Sta73] (Figure 4.13 shows a 32-bit Galois LFSR). With a non-zero seed value, a n -bit LFSR is capable of generating a non-repeating pseudo-random sequence of N numbers (where N is close to 2^n). So practically, the state space of an LFSR-based SS is as big as that of an incrementer-based SS. However, an LFSR-based SS decouples SS implementation from the SBRB's implementation.

The LFSR is augmented with the ability to bitwise-XOR *sig* with an arbitrary data input (of the same bit-width) as it is shifted, as shown in Figure 4.14. An incremental update of *sig* can be written as: `sig = LFSR_step(sig ^ data)`.

⁹Note, we do have a solution to this problem. To access the SBRB, a branch can obtain the index bits by hashing the lower k bits with the upper k bits of the key, and use all n bits of the key as tag. But this solution increases the lookup time and the number of tag bits.

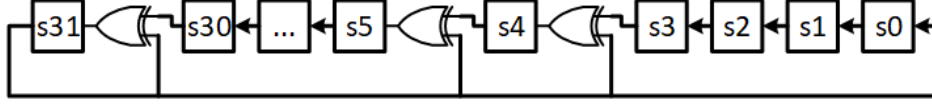


Figure 4.13: A 32-bit Galois LFSR.

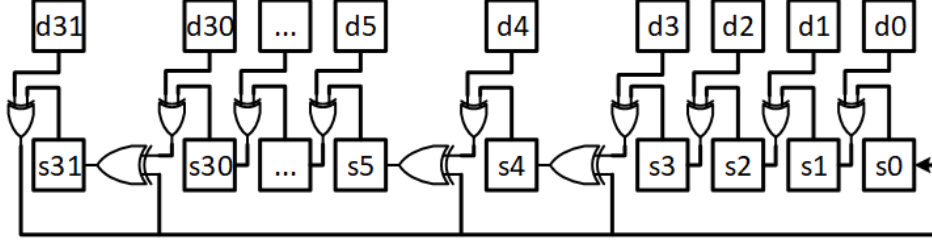


Figure 4.14: A 32-bit Galois LFSR with data input, d .

On continuing a loop, sig is updated as follows: $sig = \text{LFSR_step}(sig \wedge 0)$. With 0 as data input, sig is updated to the next state in the LFSR's randomly-evolving state space. On entering a loop, sig is updated as follows: $sig = \text{LFSR_step}(sig \wedge PC)$ (where PC is the entered-loop's header PC). This operation migrates the existing sig to a new (seed) state in the LFSR's state space.

We demonstrate signature allocation in the LFSR-based SS for doubly-nested loops (B is nested within A, just like we did for the incrementer-based SS). Figure 4.15 shows the LFSR's randomly-evolving state space spanning from state $S(0)$ to state $S(N-1)$. When A is entered, sig is migrated to a new (seed) state labeled as $\{A.1\}$ (via a `LFSR_step` with data input as A's header PC). When B is entered in the first iteration of A, sig is again migrated to a new (seed) state labeled as $\{A.1, B.1\}$ (via `LFSR_step` with data input as B's header PC). With each continuation of B, sig is updated to the next state in the sequence (via a `LFSR_step` with data input as 0). When B exits, sig is restored to the value it had before entering B (via a stack pop). Like before, sig is updated to the next state in the sequence when A is continued. When B is entered in the second iteration of A, sig is migrated to an entirely new (seed) state labeled as $\{A.2, B.1\}$. Note, unlike the incrementer-based SS, the signatures allocated to iterations from different visits to B may collide. But we expect collisions should be relatively rare for a sufficiently large state space.

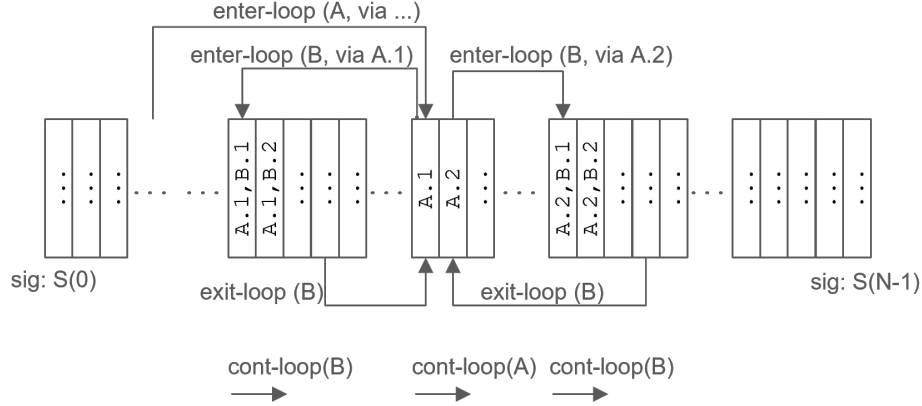


Figure 4.15: Signature allocation in a randomly-evolving state space for doubly-nested loops (B nested within A).

On entering a function, *sig* is updated as follows: $\text{sig} = \text{LFSR_step}(\text{sig} \hat{=} \text{PC})$ (where PC is the caller PC).

A full description of the LFSR-based SS is given in Table 4.6 (*sig* updates on entering/continuing a loop, and entering a function, are annotated with comments).

A key is obtained by simply hashing a branch’s signature with its PC: $\text{key} = \text{sig} \hat{=} \text{PC}$. A branch accesses the SBRB using the lower k bits of the key as index and upper $n - k$ bits of the key as tag.

4.3.4 Optimized LFSR-based Signature Stack and SBRB

We now explore and determine reasonable values of parameters associated with the three primary design structures: SBRB, SS, and LITs. Section 4.5.3 walks through the design space exploration of these parameters, discusses the intermediate results, and arrives at a reasonable set of values optimized for performance and storage cost. In this section, we summarize the optimized design, showing an itemized storage cost associated with the end implementation.

Figure 4.16 summarizes the optimized design. Design space exploration of SS parameters reveals that an 8-bit *hpc* register (set using a folded 64-bit header PC), a 32-bit *sig*

Table 4.6: LFSR-based SS management.

LIT/BTB hits	SS operations
<i>hit_header</i> (PC)	<pre> if (PC != hpc) { // enter loop push({hpc, sig}); // save hpc = PC; // <<migrate to new LFSR state>> sig = LFSR_step(sig ^ PC); } else { // cont. loop // <<move to next LFSR state>> sig = LFSR_step(step ^ 0); } </pre>
<i>hit_exit</i> (pop_amount)	<pre> // exit loop while (pop_amount--) pop({hpc, sig}); // restore </pre>
<i>hit_call</i> (PC)	<pre> push({hpc, sig}); // save hpc = 0; // <<migrate to new LFSR state>> sig = LFSR_step(sig ^ PC); </pre>
<i>hit_return</i>	<pre> pop({hpc, sig}); // restore </pre>

register, and an 8-deep save/restore stack are sufficient to capture most squashed-branch reuse opportunity (sub-figure:a).

A 32-bit signature obtained at fetch is hashed with a folded 64-bit PC to obtain a 32-bit key. The key accesses a 256-entry SBRB organized as a 64-set 4-way set-associative cache. The SBRB is indexed with the lower 6 bits of the key and tagged with the upper 26 bits of the key. Besides a tag, a SBRB entry contains a valid bit, recency information for replacement (2 bits), and an outcome (1 bit), consuming 30 bits in total (sub-figure-b).

Lastly, a 150-entry LIT-H and 300-entry LIT-E perform reasonably well, and close to the unbounded versions of the same. Table 4.7 provides an accounting for the storage cost associated with different structures involved in squashed-branch reuse. The total storage cost is 11 KB.

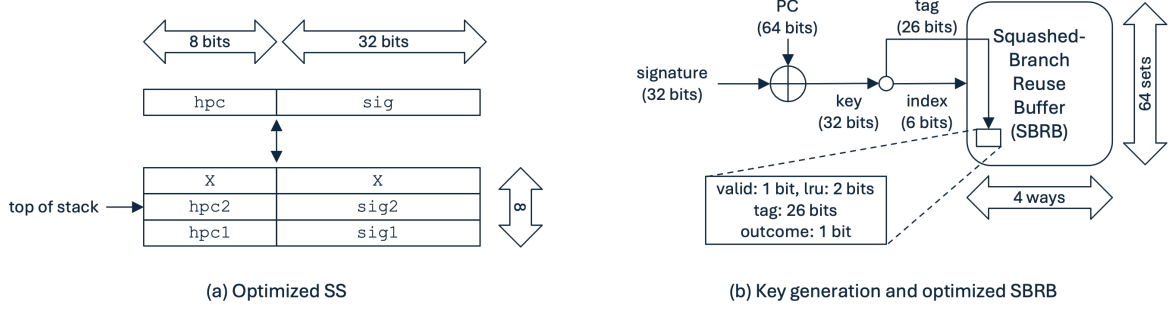


Figure 4.16: Optimized structure sizes for SS and SBRB.

Table 4.7: Cost accounting.

Component	Breakdown	Cost
LIT-H	150 entries, 62 bits/entry	1,162.5 B
LIT-E	300 entries, 66 bits/entry (PC:62, dir:1, popcnt:3)	2,475 B
SS	<i>hpc</i> :8, <i>sig</i> :32 stack: 8 entries, 40 bits/entry stack pointer:3	45.375 B
SS chkpts	(64 chkpts + 1 ret. SS) x SS cost	2,949.375 B
SBRB	256 entries, 4-way assoc., 30 bits/entry (valid:1, lru:2, tag:26, outcome:1)	960 B
BTB conf.	8K entries, 3 bits/entry	3,072 B
BQ extra bits	592 entries, +8 bits/entry (+6 for ret. SS: hdr/exit/dir/popcnt:1/1/1/3) (+2 bits for conf. training)	592 B
Total		11 KB

4.4 Methodology

The methodology adopted in this Chapter is the same as presented in Chapter 3 Section 3.4. Besides it, we added an LLVM compiler pass to generate loop information.

To keep the sizes of LIT-H and LIT-E reasonable, profiling was performed using different benchmark inputs (different from those used for evaluation) to measure frequencies of loops. Loops were ranked accordingly and filled into the limited LIS. All train inputs were used for profiling SPEC. Two synthetic graphs (Kronecker and random, both with 2^{19} vertices) and one alternate real-world graph (road-PA [LK14]) were used for profiling GAPBS.

4.5 Results

In our experiments, we include measurements for the following configurations.

- *192KB-TAGE-SC-L*: baseline with a 192KB TAGE-SC-L predictor.
- *RVQ*: squashed-branch reuse implementation using queues, but without support for divergences (see Chapter 3 Section 3.5.2).
- *RVQ-DIV*: same as RVQ but with support for divergences.
- *RVSS*: squashed-branch reuse implementation using LFSR-based SS and SBRB. SBRB, SS, and LIT parameters use optimized values.
- *RVLS*: squashed-branch reuse implementation using a 256-entry Literal Stack and SBRB. The Literal Stack’s components are run through the SHA-256 secure hash algorithm to get a 256-bit fingerprint. A CRC pass reduces the fingerprint to a 32-bit signature. SBRB and LIT parameters use optimized values.

Note:

- The baseline uses a 64KB TAGE-SC-L predictor.
- All reuse configurations use the L-S-OP-3 confidence scheme for overriding the default predictor (see Chapter 3 Section 3.5.1).
- RVSS results assume a LFSR-based SS. Results for an increment-based SS are discussed in Section 4.5.4.

4.5.1 Overall results

Figure 4.17 shows the speedup of different reuse configurations as well as of 192KB-TAGE-SC-L over baseline. Overall, 192KB-TAGE-SC-L provides a speedup of 0.73%. Its speedup on GAP is very low (just 0.10%). GAP benchmarks have only a few hard-to-predict

branches, and in most cases, the direction of these branches depend on recently-loaded memory values (and not so much on branch history). Overall, RVQ and RVQ-DIV perform substantially better than 192KB-TAGE-SC-L.

RVSS provides a speedup of 2.08% on SPEC, 7.25% on GAP, and 4.43% overall, considerably outperforming the better reuse-via-queue scheme, RVQ-DIV. This increase in speedup comes from better branch alignment and outcome retention from several prior squashes.

Overall, RVSS performs within 0.02% of RVLS, which uses a collision-resistant cryptographic hash of the full identifier. This shows signatures coming from an appropriately-configured LFSR-based SS serve as effective proxies for full identifiers.

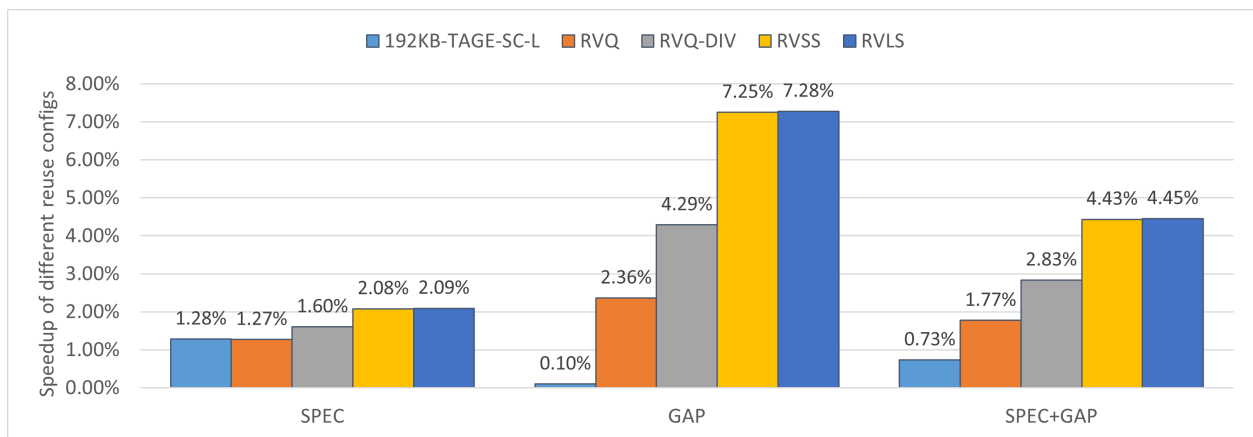


Figure 4.17: Speedup of different reuse configurations over baseline.

4.5.2 Benchmark-wise results

SPEC

We discuss benchmark-wise results for three configurations: 192KB-TAGE-SC-L, RVQ-DIV, and RVSS. Figure 4.18 shows the speedups for SPEC benchmarks sorted by decreasing RVSS speedup. The up-sized predictor noticeably outperforms the other two reuse

implementations on 445.gobmk, 403.gcc, 602.gcc, and 600.perlbench. These benchmarks benefit from a bigger predictor as it helps in capturing more static branches and their dynamic branch patterns.

RVQ-DIV outperforms RVSS on 401.bzip2 and 400.perlbench. These two benchmarks have non-loop cycles, resulting in a fixed signature despite iterative behavior.

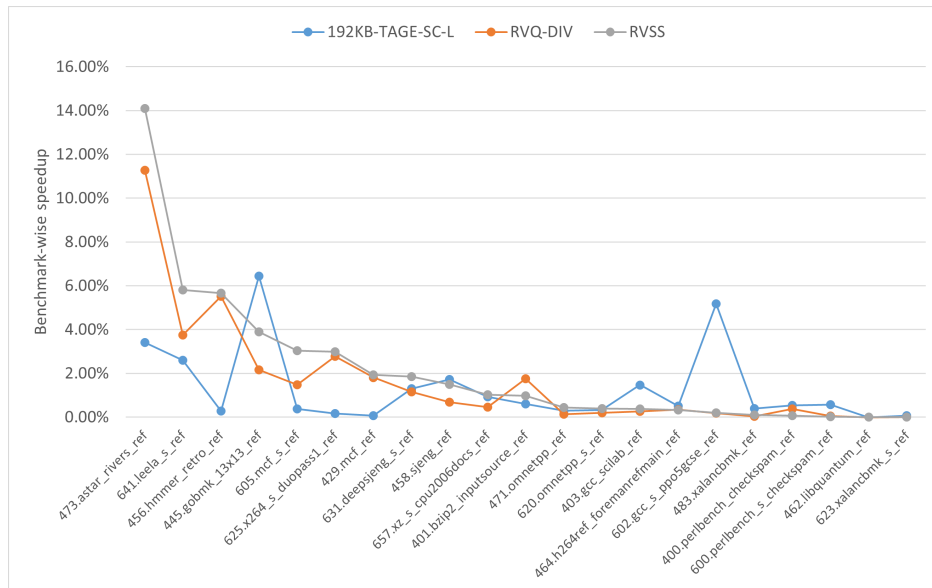


Figure 4.18: Benchmark-wise speedup for selected configurations (SPEC).

Figure 4.19 shows the speedup and absolute reduction in MPKI (mispredictions per kilo-instructions) achieved by RVSS. The speedup values (in gray) correspond to the left vertical axis and the MPKI reduction values (in yellow) correspond to the right vertical axis.

Squashed-branch reuse eliminates branch mispredictions, and this has a direct impact on performance: the cause and effect is evident in the figure. 473.astar benefits the most, with squashed-branch reuse eliminating 3.66 mispredictions per-kilo-instructions, and improving performance by 14.09%.

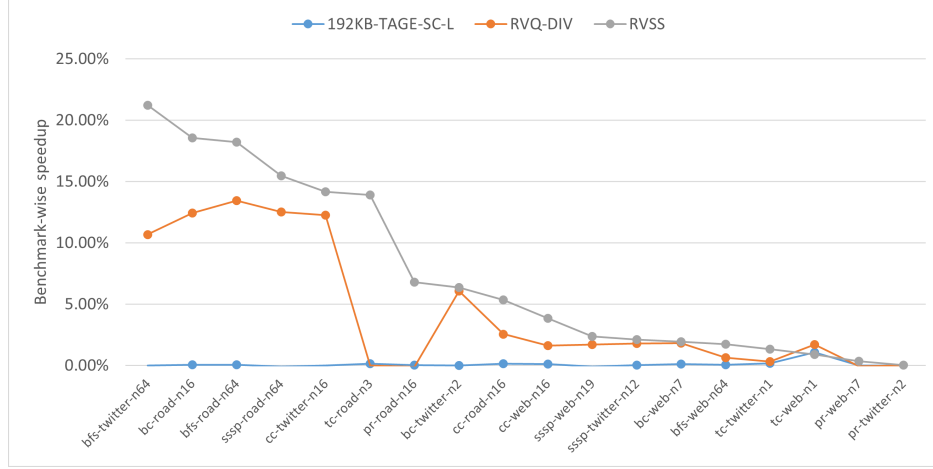


Figure 4.20: Benchmark-wise speedup for selected configurations (GAP).

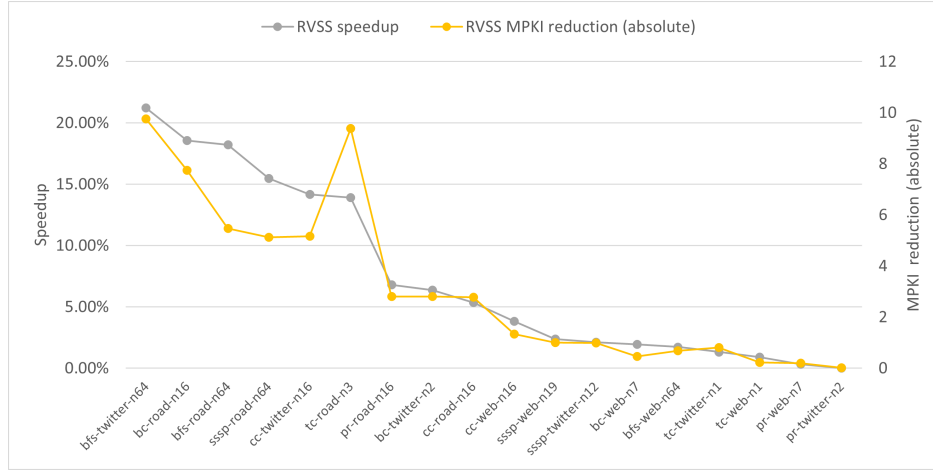


Figure 4.21: RVSS speedup and MPKI reduction (absolute) for GAP benchmarks.

SBRB

Table 4.8 shows performance as SBRB size (number of entries) is successively doubled, for both a 4-way set-associative SBRB and a fully-associative SBRB. Based on the results, we use a 256-entry 4-way set-associative SBRB in our final/optimized design.

Signature Management Parameters

Table 4.9 and Table 4.10 show the performance impact of individually reducing SS parameters and LITs, respectively, while keeping the other parameters at their maximum settings

Table 4.8: Performance (% speedup over baseline) of different SBRB configurations.

	16	32	64	128	256	512	1024	2048
4-way set-assoc.	3.06%	3.61%	4.11%	4.40%	4.49%	4.50%	4.51%	4.51%
fully assoc.	3.19%	3.71%	4.26%	4.49%	4.51%	4.51%	4.51%	4.51%

(64-entry stack, 64-bit *sig*, 64-bit *hpc*, unbounded LIT-H/LIT-E). LIT-E size is 2x LIT-H size.

Based on the sweeps, our final/optimized design is: 8-entry stack, 32-bit *sig*, 8-bit *hpc*, 150-entry LIT-H, and 300-entry LIT-E.

Table 4.9: Performance (% speedup over baseline) as each SS parameter is reduced (from 64 to 2).

	64	32	16	8	4	2
stack depth	4.49%	4.49%	4.49%	4.49%	4.46%	4.40%
<i>sig</i> width	4.49%	4.48%	4.35%	0.61%	-	-
<i>hpc</i> width	4.49%	4.49%	4.49%	4.49%	4.48%	4.08%

Table 4.10: Performance (% speedup over baseline) as LIT sizes are reduced.

	All	250/500	200/400	150/300	100/200	50/100
LIT-H/LIT-E	4.49%	4.45%	4.45%	4.44%	4.42%	4.38%

Most benchmarks are not impacted by the limited-size LIT-H/E, but some are. Figure 4.22 shows benchmarks for which the difference [% speedup with unbounded LIT-H/E] - [% speedup with 150/300 LIT-H/E] is at least 0.1% (other parameters at maximum settings).

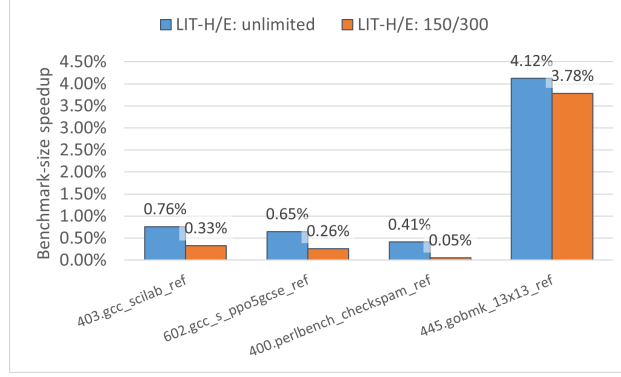


Figure 4.22: Benchmarks sensitive to limited-size LIT-H/E.

4.5.4 Incrementer-based SS

An incrementer-based SS, combined with a 256-entry fully-associative SBRB, provides an overall speedup of 4.49%. This is slightly better than the 4.46% speedup of the LFSR-based SS with 256-entry fully-associative SBRB. (Other parameters are from the final/optimized design: 8-entry stack, 32-bit *sig*, 8-bit *hpc*, 150-entry LIT-H, 300-entry LIT-E.)

Chapter 5

Dynamic predication

5.1 Introduction

Dynamic predication is a form of eager execution, where upon fetching a hard-to-predict branch, instructions from its not-taken and taken paths are eagerly fetched and executed [Kla+98; Kim+06; Cha+20]. Control-independent data-dependent instructions synchronize on data live-outs, but control-independent data-independent instructions execute normally. When the branch resolves, control-dependent instructions on the incorrect path are squashed and silenced in the pipeline whereas instructions on the correct path are allowed to proceed and commit their results. As opposed to static predication, dynamic predication is applied to a branch when it is hard-to-predict at runtime.

Although predication subsumes branch mispredictions, it has some obvious overheads. Front-end overheads are much more direct, and much more easy to control, compared to back-end overheads (related to ROB and issue queue). Understandably, all prior works account for fetch overheads in some way [Kla+98; Kim+06; Cha+20]. Nevertheless, all overheads must be accounted for and/or properly dealt with, ensuring predication not only improves performance but also limits slowdowns.

All prior schemes restrict predication to two paths, *i.e.*, they disallow multi-path pred-

ication. That said, conditional branches may appear on a predicated path, and if they do occur, they are predicted (not predicated).

In DHP [Kla+98], predication is applied only to simple hammock branches, *i.e.*, if- and if-else-type branches. Further, predication is restricted to forward hammocks¹ to avoid fetch discontinuities. A profiling compiler is used to select and mark candidate branches in the binary, and a confidence estimator [JRS96] is used to know if a branch is hard-to-predict at runtime. After fetching a branch’s control-dependent instructions, a conditional move instruction (cmove) is injected for every register live-out. Cmove instructions, which depend on the branch predicate, guard the release of register live-outs to control-independent data-dependent instructions until the branch resolves.

In DMP [Kim+06], predication is extended to nested hammocks, frequently hammocks, and simple loops. Though DMP improves coverage over DHP substantially, it is similar to DHP in other aspects: a) it selects and marks candidate branches using a profiling compiler [Kim+07], b) it uses a confidence estimator to determine if a branch is hard-to-predict at runtime, and c) it uses select micro-ops (seluops) which have the same functionality as cmoves, to synchronize register live-outs.

In ACB [Cha+20], predication is restricted to simple hammocks, yet many more hammock-layouts are allowed than DHP’s forward hammocks. Unlike DHP and DMP, ACB obtains candidate branches using online re-convergence analysis and profiling. A single rename map table is used to rename *all* control-dependent instructions of a predicated branch – instructions are renamed serially, first, on the not-taken path, and then, on the taken path. Each register-producing control-dependent instruction is a) forced to include its (logical) destination register as an additional source register, and b) forced to wait in the issue queue until the predicated branch resolves. Register synchronization is accomplished by releasing the predicated-true instructions as normal, and releasing the predicated-false instructions as move/relay instructions (similar to SBR [GAS04]). Further, ACB sharply

¹These are branches whose not-taken and taken control-dependent instructions appear sequentially next to the branch.

reduces slowdowns by throttling predication using Dynamo – an online mechanism to measure performance, and gradually enable/disable predication on candidate branches based on observed performance.

In this chapter, we revisit DMP, and propose various software and hardware improvements to it.

We begin with software improvements to DMP. We use a three-stage profiling-compiler pipeline to arrive at a filtered set of candidate branches that are profitable-to-predicate from the standpoint of fetch. Section 5.2 discusses the pipeline focusing on forward branches:

- Stage-1 performs a re-convergence analysis on each hard-to-predict branch to obtain a) a merge point, b) a set of exit points (if any), and c) instruction/branch usage on the not-taken and taken paths of the branch. Branches with a valid merge point and without any unsuitable control-dependent instruction are sent for stage-2 processing.

The stage-1 described here is similar to what appears in DMP but with one important difference. In DMP, frequently hammocks are characterized by multiple merge points, but in this work, such hammocks are characterized by one merge point and one or more exit points. Predication perturbs global branch history, and the design choice of having a single merge point ensures that, *if* predication has a negative effect on the accuracy of other branches (via global branch history), the effect is much more controlled and consistent.

Stage-1 of the software pipeline is discussed in Section 5.2.1.

- In stage-2, we apply the analytical model which, for each hard-to-predict branch, compares the overhead of redundant fetch against the overhead of conventional speculation. Branches whose predication overhead is lower than speculation overhead are sent for stage-3 processing.

The stage-2 described here is similar to what appears in DMP (and other works). It

is summarized in Section 5.2.2.

- Predicated branches, along with their control-dependent branches, if any, are removed from the global branch history. A partial branch history reduces the prediction accuracy of branches which were correlated with the predicated branches. Therefore, although predication subsumes the mispredictions of predicated branches, it increases the mispredictions of correlated branches which are not predicated.

Stage-3 looks at the negative effects of predication on branch prediction accuracy. We propose to disallow predication on *inseparable* branches, *i.e.*, branches which have a (substantial) negative effect on the accuracy of un-predicated branches. We determine and filter inseparable branches in this stage using a novel *iterative analysis*. For every input branch, the analysis measures the impact of *individually* predicating the branch on un-predicated branches, and iteratively removes branches from the input set until all the remaining branches are *separable*, *i.e.*, they have little or no negative effect on the accuracy of un-predicated branches. Only separable branches are communicated to the hardware.

The iterative analysis is discussed in Section 5.2.3. From this point on, candidate hard-to-predict branches are referred to as aggressor branches.

In DMP, the authors presented a scheme for predicating hard-to-predict loop branches, and they also reported performance improvements with loop predication. But in our experiments, we didn't observe any benefit from predicating loop branches using their scheme:

- DMP's below-par loop-predication performance is due to high occurrence of under-iterations which is a direct consequence of relying on branch prediction to guide predication. In this work, we revamp loop predication such that, instead of relying on branch prediction, both a) the decision to predicate a loop, and b) the number of iterations to predicate for (if we decide to predicate the loop), are guided by the distribution of loop iterations in the previous n visits to the loop. Using this

approach, predication subsumes many more loop branch mispredictions, and sharply diminishes the chances of loop under-iterations – a key issue with the prior scheme – while predicating a loop.

Section 5.3 discusses a new stage-2 analytical model for loop branches that limits loop under-iterations and looks at profitability from the standpoint of fetch.

The final set of aggressor branches, along with their merge and exit points, are annotated in the RISC-V binary using a special encoding scheme. The scheme is interesting in that it doesn’t increase the number of instructions in the final annotated binary. The encoding scheme is covered in Section 5.4.

Section 5.5 proposes a revised DMP microarchitecture:

- A software-supplied aggressor branch may be easy-to-predict or unprofitable-to-predicate at runtime. To avoid unwanted predication, the processor performs online profiling using a *profiling buffer* and a *profiling table*. The profiled information feeds into an online version of stage-1 delinquency-based check and stage-2 analytical-model-based check, and predication is applied to an aggressor branch only if it passes these checks.

Among prior works, ACB’s online profiling is the most mature (in DHP and DMP, profiling is limited to qualifying branches as hard-to-predict using a confidence estimator). The profiling discussed here is different from ACB’s profiling in that it doesn’t require re-convergence analysis (merge points are supplied by the software) and it includes loop profiling (that feeds into loop predication). Online profiling is discussed in Section 5.5.1.

- Branches which pass the stage-1/2 checks are added to a *predicated branch table*, which is then used by the fetch unit to apply predication to the selected aggressor branches. Predicates are allocated/released in-order using a *predicate queue*. The fetch unit avoids fetching beyond an exit point using an *exit table*.

Section 5.5.2 discusses the core pipeline changes for an adapted DMP-style dynamic predication.

- Despite being beneficial from the standpoint of fetch, predication can still cause slowdown due to increased pressure on the issue queue and ROB. To limit the negative effects of such pressures, the authors of ACB proposed Dynamo, an online mechanism that provides safety against potential slowdowns. We found Dynamo to be highly useful.

In Section 5.5.3, we show how Dynamo is integrated into the microarchitecture.

- Section 5.5.4 summarizes the cost of new structures added to the pipeline.

Sections 5.6, 5.7, and 5.8 discuss methodology, results, and future work, respectively.

5.2 Software improvements to DMP (forward branches)

5.2.1 Stage-1: Initial branch selection and re-convergence analysis (an adaptation of prior work)

For the first stage, we augment spike [spi], a RISC-V ISA simulator, with a 64KB TAGE-SC-L branch predictor [Sez16]. Using the predictor-augmented simulator, we perform branch prediction simulations, and generate a misprediction profile, one for each train input for every benchmark. Branches which meet a *misprediction rate threshold* (default: 5%) **and** a *misprediction per-kilo-instruction threshold* (default: 0.01) are labeled as hard-to-predict. This initial set of hard-to-predict branches, along with the not-taken/taken biases (*i.e.*, edge profiles) of all observed branches, are forwarded to the re-convergence analysis. For branches with a low visit count – determined by a *dynamic count per-kilo-*

instruction threshold (default: 0.01) – we do not use their dynamic not-taken/taken biases. Instead we rely on their static biases.

We first describe a re-convergence analysis which ignores cycles and assumes the input branch has a valid post-dominator. We later show how the analysis can be generalized. The re-convergence analysis of an input branch is split into two passes: a *forward pass* that finds a merge point and a *backward pass* that finds all exits.

Forward pass

To set up the forward pass, the not-taken and taken successors of the input branch a) are marked as *reachable*, and b) initialize their *reaching probability* using the edge profile.

The forward pass visits the blocks marked as *reachable* in a loop. The oldest block in the topological order that is unvisited **and** marked as *reachable*, is picked for a visit. For the visited block, the following steps are taken (in order):

1. If the visited block comes after the input branch’s post-dominator in the topological order, the forward pass is terminated ².
2. If the visited block is a *join block* [LLVa] (*i.e.*, there exist two disjoint paths from the input branch to the visited block) **and** its *reaching probability* is high enough – determined by a *re-convergence threshold* (default: 1 minus 10% of input branch’s misprediction rate) – it is selected as a merge point, and the forward pass is exited.
3. If the visited block’s *reaching probability* is very low – determined by an *exit threshold* (default: 2% of input branch’s misprediction rate) – it is marked as *likely-exit*. If marked as *likely-exit*, the block skips further processing.
4. The visited block’s successors are marked as *reachable* and their *reaching probability* is updated.

²Such a scenario may occur when many blocks are marked as *likely-exit*.

Backward pass

The backward pass runs only when the forward pass generates a valid merge point. To set up the backward pass, the merge block's predecessors, excluding the ones which were not marked as *reachable*, are marked as *control-dependent*.

The backward pass visits the blocks marked as *control-dependent* in a loop. The youngest block in the topological order that is unvisited **and** marked as *control-dependent*, is picked for a visit. For the visited block, the following steps are taken (in order):

1. For each successor of the visited block, if the successor is not marked as *control-dependent* **and** it is not the merge block, it is marked as *exit*.
2. For each predecessor of the visited block, if the predecessor is not marked as *control-dependent* **and** it was marked as *reachable* **and** it was not marked as *likely-exit*, it is marked as *control-dependent*.

Generalized forward/backward passes

For forward branches, the re-convergence analysis must disallow cycles from appearing within an input branch's control-dependent region. To account for cycles, the analysis is adjusted as follows:

- Instead of the conventional topological order, the analysis uses a modified topological order where each block occurs after all its predecessors, except if that predecessor is a cycle header ³.
- Step:3 of the forward pass is augmented such that if the visited block is a cycle header, it is marked as *likely-exit* irrespective of its *reaching probability* (and step:4 is skipped for it).

³For more details, please refer [RMH21].

It is possible for an input branch to have no valid post-dominator. This scenario can occur when the not-taken/taken paths end in a return instruction. This is a valid re-convergence scenario referred to as *merge-via-return*, and we allow predication on such input branches, similar to DMP. If the input branch doesn't have a valid post dominator, the analysis is adjusted as follows:

- A special *return-sink* block is added to the CFG.
- All the blocks that end in a return instruction are forced to add the *return-sink* block as their sole successor. This trick allows the *return-sink* block to serve as the input branch's post-dominator.

Finalizing stage-1 output

In addition to the merge point, the forward pass also determines the maximum number of instructions/branches consumed by the not-taken and taken paths, separately, up to the merge point. These resource usages are used in stage-2 processing.

The resource usages can also be used to statically filter branches which consume too many resources. We filter branches which exceed a *maximum instruction threshold* (default: 64) on the not-taken or the taken path.

We also filter an input branch if any of its control-dependent blocks contains a call or an indirect branch. Embedded calls are forbidden as we do not have callee functions' resource usages. Embedded indirect branches are forbidden as our hardware does not allow multi-path predication. For the same reason, we also filter input branches whose control-dependent blocks contain other hard-to-predict input branches.

To summarize, a hard-to-predict input branch is selected for stage-2 processing only if it has a valid merge point **and** its not-taken/taken paths do not consume too many resources **and** its control-dependent blocks do not contain a call, an indirect, or a hard-to-predict input branch. For each selected input branch, the following is communicated to stage-2:

- its PC, misprediction rate, and not-taken/taken biases,
- whether it's a merge-via-return type branch,
- its merge PC (if applicable) and set of exit PCs (if any), and
- the number of instructions/branches on its not-taken/taken paths.

An example

Figure 5.1 shows the result of re-convergence analysis for a hard-to-predict branch (bb.6) in sssp's RelaxEdges function. The analysis proactively marks two exits, bb.8 and bb.15, removing many infrequently-visited blocks from the control-dependent region. This ultimately allows branch bb.6 to be predicated. Branch bb.6 re-converges at bb.5.

5.2.2 Stage-2: Branch filtering based on the analytical model (an adaptation of prior work)

Fetch overhead of predication

Predication involves redundant fetch. When a branch is predicated, its not-taken path is redundant if it resolves as taken, and vice-versa. The overhead (cost) of a redundant not-taken-path fetch (`cost_nt`) is estimated as follows:

$$(1.1) \text{ cost_inst_nt} = \text{ceil}(\text{insts_nt} / \text{fw})$$

$$(1.2) \text{ cost_br_nt} = \text{brs_nt}$$

$$(1.3) \text{ cost_nt} = \max(\text{cost_inst_nt}, \text{cost_br_nt})$$

where, `insts_nt`/`brs_nt` are the number of instructions/branches consumed on the not-taken path, `cost_inst_nt`/`cost_br_nt` are the fetch cycles consumed on the not-taken path considering instructions/branches, and `fw` is the *fetch width* of the processor (default: 8).

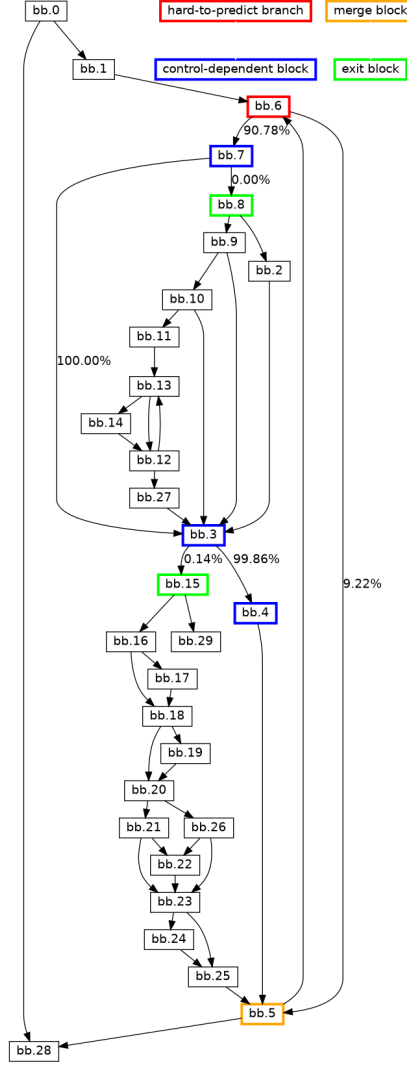


Figure 5.1: The result of re-convergence analysis on a forward branch in sssp (using the random input).

The overhead of a redundant taken-path fetch (`cost_t`) can be estimated similarly (substituting not-taken-path resource usages with taken-path). Using these two estimates, a per-instance predication overhead (`predication_overhead`) is obtained as follows:

$$(2.1) \text{ predication_overhead} = (\text{bias_nt} * \text{cost_t}) + (\text{bias_t} * \text{cost_nt})$$

where, `cost_t` is the fetch cycles consumed on the taken path considering both instructions and branches and `bias_nt/bias_t` are the biases of the not-taken/taken paths (note: the sum of biases is 1).

Fetch overhead of conventional speculation

For an input branch, a per-instance speculation overhead (`speculation_overhead`) can be estimated as follows (this is the average number of cycles spent fetching instructions on the wrong path):

$$(3.1) \text{ speculation_overhead} = \text{m_rate} * (\text{misp_penalty} + \text{added_sch_delay})$$

where, `m_rate` is the misprediction rate of the branch, `misp_penalty` is the *minimum branch misprediction penalty* (default: 12) and `added_sch_delay` is the number additional cycles spent by a branch in the scheduler on average (default: 0).

Finalizing stage-2 output

Input branches whose estimated predication overhead is less than estimated speculation overhead are sent for stage-3 processing.

5.2.3 Stage-3: Detecting and filtering inseparable branches using iterative analysis (new)

Branch prediction alongside predication: background

Branch predictors use global branch history to make predictions. The branch history is maintained in a branch history register (bhr) at fetch. Figure 5.2 sub-figure:a shows how bhr is updated in a conventional superscalar for a sample control-flow graph. The initial value of bhr is 4'b0100. First, b1 is predicted as taken, so bhr is updated to 4'b1001 (first left-shifted, and then or-ed with 1 which represents taken). Then b3 is predicted as not-taken, so bhr is updated to 4'b0010. Lastly, b4 is predicted as not-taken.

Predication alters the order in which instructions are fetched, perturbing the global branch history. This ultimately affects the overall branch prediction accuracy. Let's see how prior works supported branch prediction alongside predication.

In DMP, upon fetching a predication-eligible branch, the processor switches to dynamic predication mode (or dpred-mode). During the switch, the existing bhr is forked into two, setting up the resultant bhrs to predict branches on the predicted/alternate paths of the predicated branch. After the merge point is observed on both the paths, the processor exits dpred-mode. The authors didn't mention which of the two bhrs survives after the processor exits dpred-mode. We assume it's the predicted-path bhr.

Figure 5.2 sub-figure:b shows how bhr is updated in DMP. Initially only bhr.p (predicted-path bhr) is active, and its value is 4'b0100. Bhr.p is used to predict b1 (which is moot as b1 is predicated). When b1 causes a fork, bhr.a (alternate-path bhr) is initialized to 4'b1000 and bhr.p is initialized to 4'b1001. Bhr.a is used to predict branches on the alternate path up to the merge point (which includes b2). On the other hand, bhr.p is used to predict branches on the predicted path up to the merge point (which includes b3) and beyond (which includes b4).

In DMP, if the predicated branch resolves as correctly predicted, control-independent branches are predicted with a *precise* history – for each control-independent branch, the bhr used for predicting it corresponds to the bhr used by the conventional superscalar. But if the branch resolves as mispredicted, control-independent branches are predicted with an *imprecise* history. On the whole, in DMP, prediction and training use an *inconsistent* history.

In DHP and ACB, a predicated branch does not update the bhr, *i.e.*, it is omitted from the branch history. Therefore, control-independent branches are *always* predicted with a *partial* history. The authors of both DHP and ACB report that, in general, prediction using a *partial* history performs better than prediction using an *inconsistent* history. But since these paradigms disallow predication on branches which encapsulate other conditional branches, it is unclear how to predict encapsulated branches in paradigms which do allow them (like DMP) while still maintaining a history that is *partial* but nevertheless *consistent*.

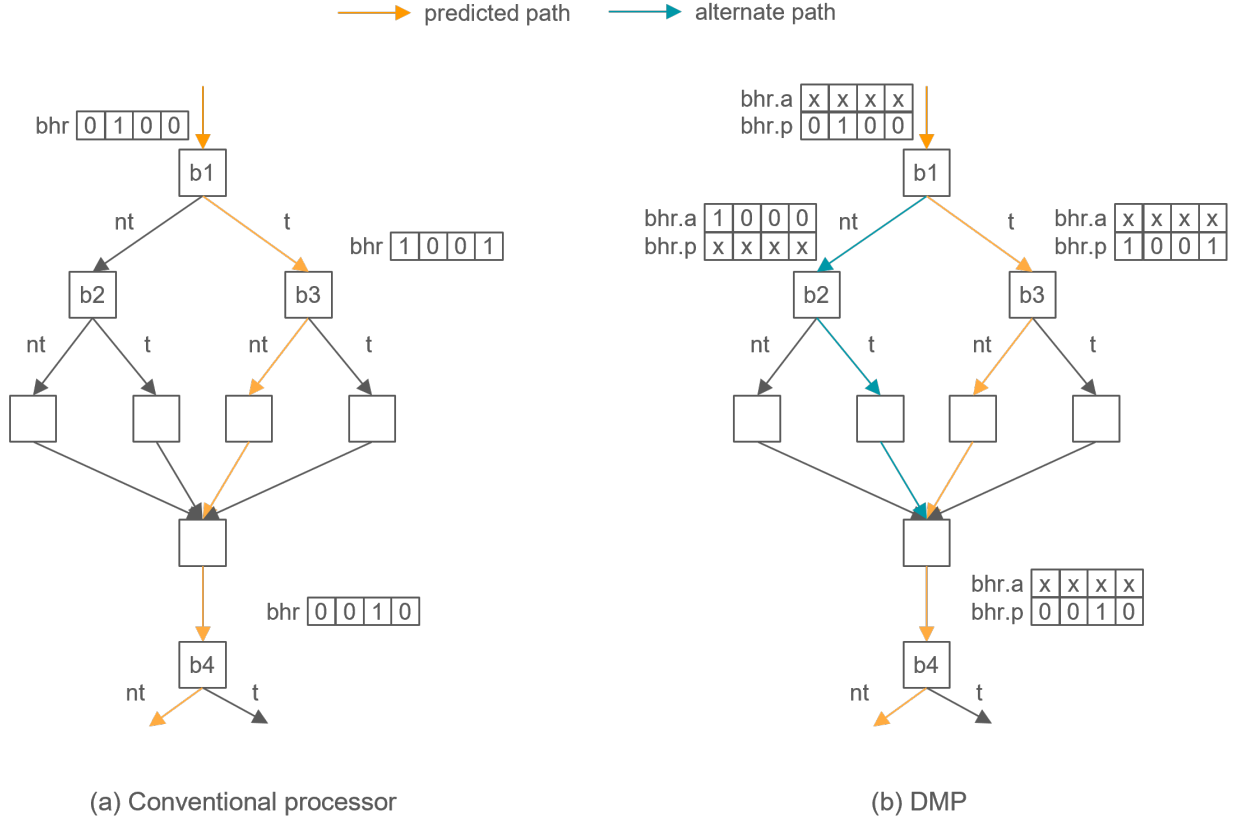


Figure 5.2: Bhr updates in conventional processor and DMP.

Branch prediction alongside predication: this work

We improve upon DMP’s scheme, and propose branch prediction using three bhrs: bhr.m (main bhr), bhr.nt (not-taken-path bhr), and bhr.t (taken-path bhr). When the processor is not in dpred-mode, it uses bhr.m to predict branches. During the switch to dpred-mode, the bhr.m is forked into two new bhrs, setting up the new bhrs (bhr.nt/bhr.t) to predict branches on the not-taken/taken paths of the predicated branch. When both the paths re-converge, the processor resumes prediction using bhr.m. Figure 5.3 captures this scheme using the same example as before. Note b4 is predicted using bhr.m which has a *partial* history: the predicated branch (b1) and all its inner conditional branches (b2, b3) are omitted from the branch history.

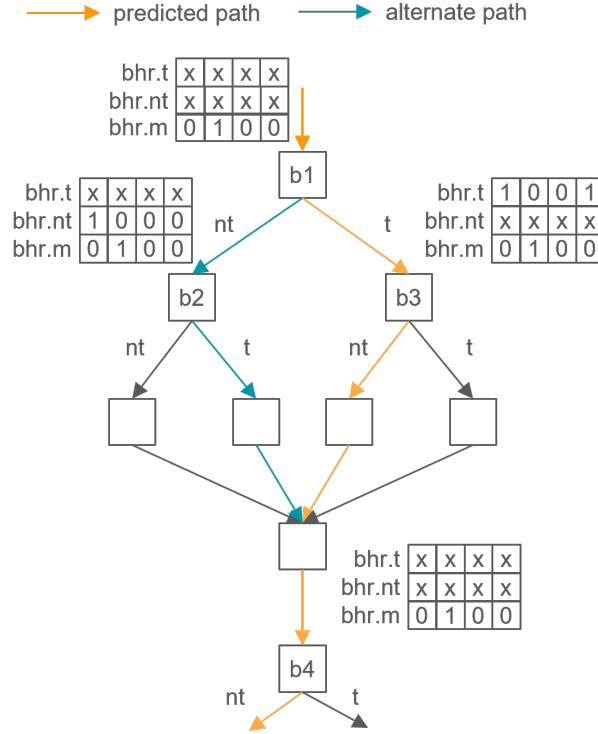


Figure 5.3: Bhr updates in improved DMP.

Introduction to stage-3

Although a partial history is desirable, it may still hurt performance. Consider, for a hard-to-predict branch br1, one of its control-independent branches br2 is highly correlated with br1. As a result, even though br1 is hard-to-predict, br2 is not. When predication is applied to br1, with a partial history, br2 starts to mispredict, at a rate similar to br1 in the worst case. Such a scenario can easily degrade performance, especially when predication cannot be applied to br2.

We call branches like br1, *inseparable* branches, because they are too important to be removed/separated from global branch history. Predication should be avoided on inseparable branches as the downside of predicating an inseparable branch (extra mispredictions induced by predication on un-predicated branches) can easily outweigh the upside (mispredictions subsumed by predication).

In stage-3, we refer to predication-eligible branches as *aggressor* branches, and to the rest of the branches as *victim* branches. The goal of stage-3 is to migrate all inseparable branches from the aggressor set to the victim set so that all remaining branches in the aggressor set are separable.

Stage-3 core: iterative aggressor/victim analysis

To determine whether an aggressor branch is inseparable/separable, we compute its *inseparability factor*. For an aggressor, the factor is computed using the following procedure:

- Run two branch prediction simulations to generate two misprediction profiles, one that uses a complete history (base-prof) and the other that uses a partial history where the aggressor branch is omitted from the history (agg-prof).
- Then using the two profiles, compute the increase in the number of mispredictions for victim branches (`extra_victim_misp`) by subtracting the number of mispredictions for victim branches in base-prof (`victim_misp_base-prof`) from the number of mispredictions for victim branches in agg-prof (`victim_misp_agg-prof`):

$$(4.1) \text{ extra_victim_misp} = \text{victim_misp_agg-prof} - \text{victim_misp_base-prof}$$

- Generate the inseparability factor by dividing the above number by the number of mispredictions for the aggressor in base-prof (`agg_misp_base-prof`):

$$(4.2) \text{ inseparability_factor} = \text{extra_victim_misp} / \text{agg_misp_base-prof}$$

An aggressor branch is deemed inseparable if its inseparability factor is more than a *minimum inseparability threshold* (default: 0.10). We now describe a procedure that iteratively analyzes aggressor branches and migrates inseparable branches from the aggressor set to the victim set:

1. For each input aggressor branch, run a branch prediction simulation to generate its corresponding agg-prof. Run another branch prediction simulation to generate a base-prof. Note, only $n + 1$ simulations are required for n input aggressors.
2. For each aggressor, compute its inseparability factor using its agg-prof and base-prof.
3. If the highest observed inseparability factor is lower than the *minimum inseparability threshold*, exit the procedure.
4. Migrate the aggressor with the highest observed inseparability factor from the aggressor set to the victim set.
5. If the aggressor set is empty, exit the procedure. Otherwise go to step:2.

Finalizing stage-3 output

After the iterative analysis, we get an optimized aggressor set, one for each training input for all benchmarks. For each benchmark, all its train aggressor sets are merged into a final aggressor set. Branches in the final aggressor set are communicated in the binary via hints.

5.3 Software improvements to DMP (loop branches)

5.3.1 A review of DMP’s loop predication

In DMP, the authors proposed a scheme to predicate hard-to-predict loop branches. For simplicity, they restricted predication to innermost loops with only one loop branch (*i.e.*, loops with only one exiting block [LLVb]). Let’s understand their scheme using an example.

Figure 5.4 sub-figure:a shows a hard-to-predict loop from benchmark tc [BAP15]. The conditional branch in the code is a loop branch whose taken target is label:A and not-taken target is label:B. Consider a visit to the loop where the loop branch is predicted taken in the first two iterations, and predicted not-taken in the third iteration. Sub-figure:b shows the

corresponding dynamic instruction stream with conventional speculation (without loop predication). The instruction stream, where logical registers are replaced with physical registers, is easy to follow.

Sub-figure:c shows the corresponding dynamic instruction stream when the loop is predicated. Predication is initiated when the loop branch is predicted taken (see the first loop branch; predicate generated: p1). Once a loop branch is predicated, all subsequently-fetched predicted-taken loop branches are also predicated (see the second loop branch; predicate generated: p2). Predication stops when the loop branch is predicted not-taken (see the third loop branch; no predicate generated). Select micro-ops, one for each register live-out, are injected after every predicated loop iteration. These micro-ops ensure post-loop instructions receive correct live-out values if predication is *rollback-free*.

Loop predication is rollback-free when the loop is not *under-iterated* (it's either *over-iterated* or *exact-iterated*) during predication. In our example, the loop is not under-iterated if the correct execution requires one, two, or three iterations:

- Correct execution requires one iteration: p1 resolves as not-taken, p2 also resolves as not-taken ⁴.
- Correct execution requires two iterations: p1 resolves as taken, p2 resolves as not-taken.
- Correct execution requires three iterations: p1 resolves as taken, p2 resolves as taken, and the predicted-not-taken loop branch resolves as not-taken (confirming its prediction).

On the other hand, loop predication causes a rollback when the loop is *under-iterated*. In our example, the loop is under-iterated if the correct execution requires four or more iterations.

⁴When a loop predicate resolves as not-taken, all subsequent predicates for that loop unconditionally resolve as not-taken.

```

r6 = addi r6, -4
A: r28 = lw r6, 4
r6 = addi r6, 4
blt r28, r7, A
B: r7 = xor r7, r28

```

(a) A loop in tc.

```

pr2 = addi pr1, -4
A: pr10 = lw pr2, 4
pr11 = addi pr2, 4
blt pr10, pr3, A // prediction: t
A: pr20 = lw pr11, 4
pr21 = addi pr11, 4
blt pr20, pr3, A // prediction: t
A: pr30 = lw pr21, 4
pr31 = addi pr21, 4
blt pr30, pr3, A // prediction: nt
B: pr40 = xor pr3, pr30

```

(b) Instruction stream with conventional speculation.

```

() pr2 = addi pr1, -4
A: () pr10 = lw pr2, 4
() pr11 = addi pr2, 4
() blt pr10, pr3, A // prediction: t // p1<-
A: (p1.t) pr20 = lw pr11, 4
(p1.t) pr21 = addi pr11, 4
(p1.t) blt pr20, pr3, A // prediction: t // p2<-
() sel-uop pr22 = p1 ? pr20 : pr10 // r28
() sel-uop pr23 = p1 ? pr21 : pr11 // r6
A: (p2.t) pr30 = lw pr23, 4
(p2.t) pr31 = addi pr23, 4
(p2.t) blt pr30, pr3, A // prediction: nt
() sel-uop pr32 = p2 ? pr30 : pr22 // r28
() sel-uop pr33 = p2 ? pr31 : pr23 // r6
B: () pr40 = xor pr3, pr32

```

(c) Instruction stream with loop predication.

Figure 5.4: Instruction stream for a loop branch in tc which is predicted taken in the first two iterations, and predicted not-taken in the third iteration.

Similar to forward branches, the authors of DMP proposed an analytical model to assess the overhead of predication against the overhead of conventional speculation for loop branches [Kim+07]. The model first assesses the predication/speculation overhead for four different cases – *exact-iteration*, *under-iteration*, *over-iteration*, and a special case of *over-iteration* – then aggregates them based on the probability of each case to arrive at a conclusion. But due to the impracticality of the model, the authors resorted to three simple heuristics. As per the heuristics, a hard-to-predict loop branch is filtered if the number of instructions in the loop body exceeds a threshold, **or** the average number of instructions per loop visit exceeds a threshold, **or** the average number of loop iterations exceeds a threshold.

We implemented DMP’s loop predication in our simulator, and evaluated its performance. To our surprise, the scheme didn’t yield any speedup on the set of benchmarks we considered.

5.3.2 Investigating performance bottleneck in DMP's loop predication

For loop predication, there are three different cases to consider: exact-iteration, over-iteration, and under-iteration. Assuming identical branch prediction accuracy with and without predication, the overhead of the three cases are summarized in Table 5.1.

Table 5.1: Fetch overhead of three different cases of a loop visit.

	Without predication (conventional speculation)	With predication
<i>exact-iteration</i>	-	-
<i>over-iteration</i>	minimum branch misprediction penalty	cycles spent on fetching additional iterations
<i>under-iteration</i>	minimum branch misprediction penalty	minimum branch misprediction penalty + additional delay in branch resolution

In the exact-iteration case, both predication and conventional speculation have no fetch overhead. Predication has an edge over its counterpart only in the over-iteration case (assuming the branch misprediction penalty overshadows cycles spent on fetching additional instructions). For the under-iteration case, predication not only causes a rollback (an antithesis of predication), but its performance penalty is a significantly higher than its counterpart. Let's take a look.

When the loop is under-iterated, the loop branch in the last iteration is mispredicted (the correct execution requires at least one more iteration), and its resolution is on the critical path. This branch resolves *significantly slower* when predication is enabled compared to conventional speculation.

To put the delay into perspective, in Figure 5.5, we show the critical path of tc's loop branch in the final iteration. With conventional speculation (sub-figure:a), assuming every instruction takes one cycle to execute, the final branch can execute in the fifth cycle.

Whereas with predication (sub-figure:b), the branch execution is delayed by one cycle (20% worse than conventional speculation).

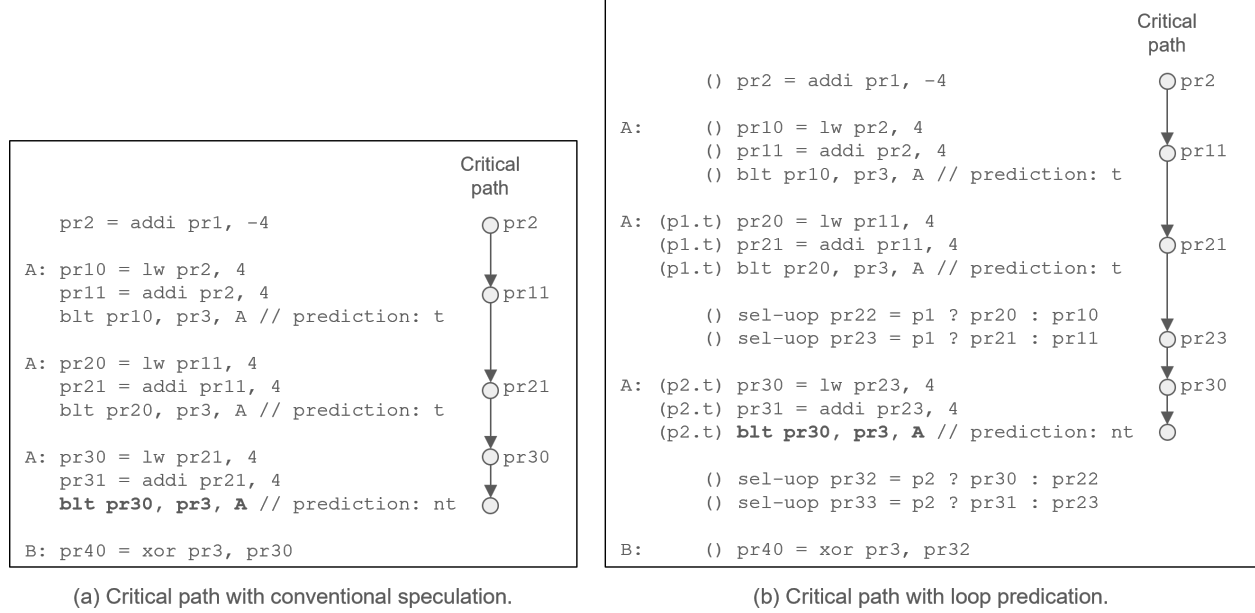


Figure 5.5: Instruction stream annotated with critical paths for the loop branch in the last iteration.

To check the relative occurrence of the three cases, we profiled our benchmarks using the 64KB TAGE-SC-L predictor. Table 5.2 shows the breakdown of the three cases for all benchmarks which had one or more hard-to-predict loops ⁵.

The data reveals that loop under-iterations are almost as frequent as loop over-iterations (7.09% *vs.* 8.80% on average). The high occurrence of loop under-iterations coupled with their high fetch penalty, are argued to be the major performance bottleneck in DMP’s loop predication.

⁵The data uses loops after the stage-1 selection process (see Section 5.3.3). For each benchmark we averaged the measurements for all hard-to-predict loops observed in its train input runs.

Table 5.2: Breakdown of exact-iterations, over-iterations, and under-iterations.

	Exact-iterated (correctly predicted)	Under-iterated (mispredicted)	Over-iterated (mispredicted)
401.bzip2	85.97%	1.94%	12.09%
403.gcc	84.11%	6.97%	8.92%
429.mcf	85.35%	5.84%	8.81%
445.gobmk	88.81%	5.71%	5.48%
458.sjeng	92.20%	3.50%	4.30%
631.deepsjeng_s	93.51%	3.94%	2.55%
641.leela_s	77.34%	9.38%	13.28%
657.xz_s	84.09%	5.85%	10.07%
bc	82.16%	9.57%	8.27%
bfs	81.92%	9.53%	8.55%
pr	82.72%	8.68%	8.59%
tc	71.14%	14.12%	14.75%
Average	84.11%	7.09%	8.80%

5.3.3 Improving DMP’s loop predication

Stage-1: Initial branch selection (an adaptation of prior work)

Similar to DMP, we only consider innermost loops with only one loop branch. Loop branches which meet the *misprediction rate threshold* **and** the *misprediction per-kilo-instruction threshold* are labeled as hard-to-predict (similar to forward branches; see Section 5.2.1).

A loop branch doesn’t require a re-convergence analysis. A loop branch has two successors, one is inside the loop and the other is outside the loop. The successor outside the loop is the merge block. In our scheme, the merge point is communicated as the direction of the loop branch which exits the loop.

A hard-to-predict loop branch is selected for stage-2 processing only if the number of instructions in the loop body is below a *maximum loop-body instruction threshold* (default: 32) **and** the loop body does not contain a call, an indirect, or a hard-to-predict forward branch. For each selected input loop branch, the following is communicated to stage-2:

- its PC and misprediction rate,
- the direction of loop-exit, and
- the number of instructions/branches in its loop body.

Stage-2: Branch filtering based on a (new) analytical model

We propose a loop predication scheme where a loop is predicated for a predetermined number of iterations, moving away from branch-predictor-driven predication.

For a loop, we first profile its trip counts to obtain a trip count distribution (a histogram) ⁶. Using this histogram, we compute the number of iterations required to *sufficiently* minimize the chances of loop under-iterations. Finally, we compare the predication overhead using the computed number of iterations against the speculation overhead for the same. If the predication overhead is smaller, the loop is selected for stage-3 processing. Otherwise it is dropped.

For a loop, the following procedure is used to arrive at the number of predicated loop iterations:

1. Compute the expected number of mispredictions per loop visit (`exp_misps_per_visit`) as follows:

$$(5.1) \text{ exp_misps_per_visit} = \text{total_misp_count} / \text{total_visits}$$

where `total_misp_count` is the total number of times the loop branch is mispredicted, and `total_visits` is the total number of times the loop is visited.

2. Profile the loop to obtain its trip count histogram. Compute the mean (`mean`) and standard-deviation (`sigma`) of the trip count histogram.
3. Assume that a loop can be predicated only for (`mean + f * sigma`) iterations, where `f` is the sigma multiplier. For three different values of sigma multiplier, one, two,

⁶In this work, trip count corresponds to the number of times the loop branch's in-loop successor is exercised before exiting the loop.

and three, estimate the probability of the event where a randomly chosen visit to the loop is under-iterated (`prob_under_iter(f)`). Assuming a normal distribution, these probabilities are:

$$(5.2) \text{ prob_under_iter}(1) = 0.3173 / 2$$

$$(5.3) \text{ prob_under_iter}(2) = 0.0455 / 2$$

$$(5.4) \text{ prob_under_iter}(3) = 0.0027 / 2$$

where 0.3173, 0.0455, and 0.0027 are the fractions of trip count bins outside $(\text{mean} + 1 * \text{sigma})$, $(\text{mean} + 2 * \text{sigma})$, and $(\text{mean} + 3 * \text{sigma})$, respectively [std].

4. Choose the smallest sigma multiplier for which the probability of under-iteration for a randomly chosen visit is sufficiently lower than the expected number of mispredictions for the visit. Or choose the smallest `f` such that:

$$(5.5) \text{ prob_under_iter}(f) / \text{exp_misps_per_visit} < \text{under_iter_threshold}$$

where `under_iter_threshold` is the *under-iteration threshold* (default: 0.10).

5. If a valid sigma multiplier is obtained in the previous step, proceed with the analytical model. Otherwise, do not consider this loop branch for predication.

Using the selected sigma multiplier (`fs`), the loop's per-visit predication overhead (`predication_overhead`) can be obtained as follows:

$$(6.1) \text{ pred_iters} = \text{ceil}(\text{mean} + \text{fs} * \text{sigma})$$

$$(6.2) \text{ added_iters_per_visit} = \text{pred_iters} - \text{mean}$$

$$(6.3) \text{ cost_inst} = \text{ceil}((\text{added_iters_per_visit} * \text{insts_per_iter}) / \text{fw})$$

$$(6.4) \text{ cost_br} = \text{added_iters_per_visit} * \text{brs_per_iter}$$

$$(6.5) \text{ predication_overhead} = \max(\text{cost_inst}, \text{cost_br})$$

where `pred_iters` is the number of predication loop iterations, `added_iters_per_visit` is the expected number of extra iterations observed dur-

ing a visit, `insts_per_iter/brs_per_iter` are the number of instructions/branches in the loop body, and `cost_inst/cost_br` is the extra fetch cycles consumed per visit considering instructions/branches.

The per-visit speculation overhead (`speculation_overhead`) is:

$$(7.1) \text{ speculation_overhead} = \text{exp_misps_per_visit} * (\text{misp_penalty} + \text{added_sch_delay})$$

where, `misp_penalty` and `added_sch_delay` are the same as discussed in Section 5.2.2.

The loop branches whose estimated predication overhead is less than estimated speculation overhead are sent for stage-3 processing.

Stage-3: Detecting and filtering inseparable loop branches

Stage-3 is run together (globally) on all input forward/loop branches. The final forward/loop branch aggressors are communicated in the binary via hints as discussed next.

5.4 Annotating the binary with predication hints

For each benchmark, the final set of aggressors is disaggregated into three lists – a list of aggressor PCs, a list of merge PCs, and a list of exit PCs.

In RISC-V, for a normal (uncompressed) instruction, `inst[1:0]` (*i.e.*, the lower two bits of a 32-bit instruction) is always 2'b11. The use of the other three `inst[1:0]` values (2'b00, 2'b01, 2'b10) is reserved for compressed instructions. For systems which do not intend to use compressed instructions, this is a missed opportunity – 75% of opcode space is rendered unusable.

To avoid the overhead of extra instructions in the binary, in this work we re-purpose RISC-V's compressed opcode space to supply predication hints. As per our new encoding scheme, if the lower two `inst` bits (`inst[1:0]`) are:

- 2'b11, it's a normal instruction without any special meaning (like before),

- 2'b10, it's a normal instruction which is also a merge point, and
- 2'b01, it's a normal instruction which is also an exit point.

And 2'b00 is reserved for future use.

Further, in RISC-V, four opcodes are reserved for custom use: 5'b00010 (custom-0), 5'b01010 (custom-1), 5'b10110 (custom-2), and 5'b11110 (custom-3). We utilize these custom opcodes to annotate aggressor branches (note, an aggressor branch is always a conditional branch). In the proposed scheme, if the opcode bits (inst[6:2]) are:

- 5'b11000, it's a conditional branch without any special meaning (like before),
- 5'b00010 (custom-0), it's a non-loop aggressor branch which has a valid merge point,
- 5'b01010 (custom-1), it's a non-loop aggressor branch which is merge-via-return type,
- 5'b10110 (custom-2), it's a loop aggressor branch whose loop-exit is on the not-taken path, and
- 5'b11110 (custom-3), it's a loop aggressor branch whose loop-exit is on the taken path.

5.5 A revised DMP microarchitecture

Figure 5.6 shows a revised DMP microarchitecture at a high-level. The revised microarchitecture uses six additional structures: *profiling buffer*, *profiling table*, *predicated branch table*, *exit table*, *predicate queue*, and Dynamo (from ACB [Cha+20]).

5.5.1 Online profiling using profiling buffer and profiling table

Merge point, exit point, and aggressor-branch-related hints are decoded in the decode stage, added to the corresponding instructions' ROB entries at the rename stage, and extracted from the ROB for profiling at the retire stage.

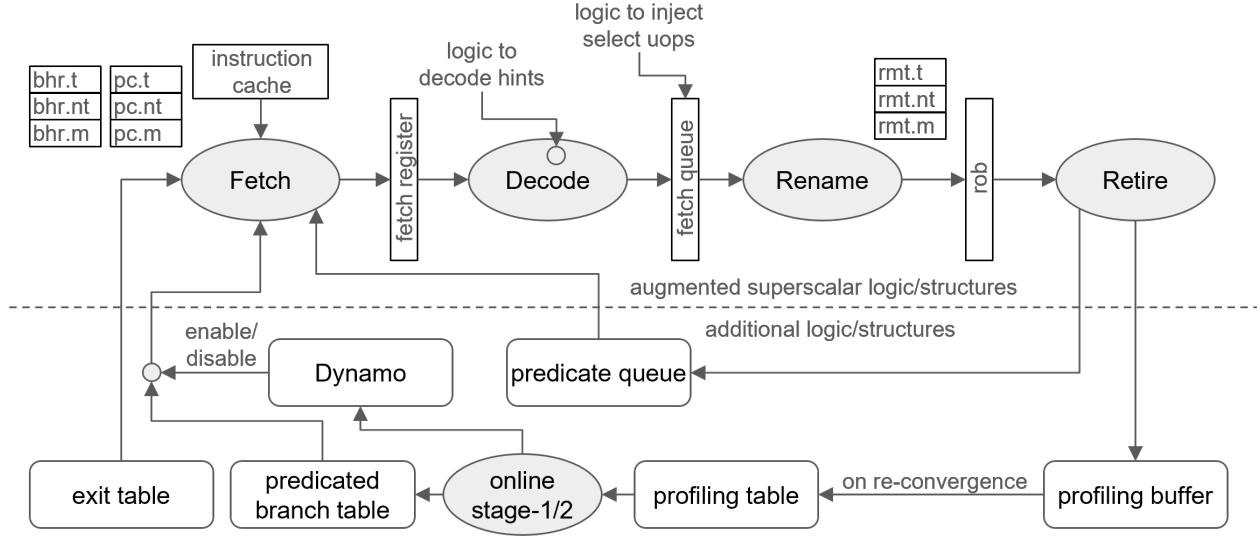


Figure 5.6: A high-level revised DMP microarchitecture.

Profiling buffer

The profiling buffer records key information related to a dynamic visit to an aggressor. It is activated after an aggressor branch is retired. When active, it monitors the retired instruction stream to a) count the number of instructions and branches, and b) log the unique destination logical registers used by the retired instructions. Upon re-convergence, it is deactivated, and the recorded information is forwarded to the profiling table. For a non-loop aggressor, re-convergence is signaled before a merge instruction is retired (for a non-merge-via-return type aggressor) and after a return instruction is retired (for a merge-via-return type aggressor). Whereas, for a loop aggressor, re-convergence is signaled when the corresponding loop is exited.

For the last-retired aggressor, the profiling buffer supplies the following information to the profiling table:

- **is_loop**: whether the aggressor is a loop (1 bit; non-loop/loop)
- **pc**: aggressor PC (62 bits; non-loop/loop),
- **dest_regs**: set of destination logical registers (63 bits; non-loop/loop).

- **n_misps**: whether the aggressor was mispredicted for a non-loop aggressor or number of loop-branch mispredictions for a loop aggressor (6 bits; possible values for non-loop: {0,1}),
- **merge_via_return**: whether it's a merge-via-return type aggressor (1 bit; non-loop only),
- **merge_pc**: merge PC (if applicable; 62 bits; non-loop only),
- **dir**: resolved direction (1 bit; non-loop only),
- **n_insts/n_brs**: number of instructions/branches retired on the resolved path (6+6 bits; non-loop only)
- **loop_exit_dir**: loop-exit direction (1 bit; loop only), and
- **tc**: trip count of the visit (6 bits ⁷; loop only)
- **n_insts_iter/n_brs_iter**: number of instructions/branches retired in the last iteration (6+6 bits; loop only),

The overall cost of the profiling buffer, including a valid bit, is 209 bits (209 bits: non-loop; 152 bits: loop).

If an exit instruction is retired before re-convergence, the profiling buffer is deactivated, and the recorded information is not forwarded to the profiling table.

Profiling table

The profiling table profiles multiple aggressors simultaneously. It accumulates information from multiple past visits to an aggressor into its corresponding profiling table entry. Each aggressor is profiled/trained for a fixed number of dynamic visits determined by a *training*

⁷A trip count higher than 63 is capped at 63.

frequency threshold (default: 1000). When the number of training samples reaches the training frequency threshold, the aggressor is removed from the profiling table, and subjected to online delinquency-based and analytical-model-based checks. If an aggressor passes these checks, it is added to the predicated branch table, and it is not profiled again. On the other hand, a failed aggressor is allowed to re-train – it is re-allocated in the profiling table on the next visit to the aggressor.

Below, we enumerate different components of a profiling table entry (`pte`). We also show how each component is augmented with information from the profiling buffer (`pb`) (on a hit ⁸):

- `n_samples`: number of accumulated visits to the aggressor (10 bits; non-loop/loop)

(8.1) `pte.n_samples++`

- `live_out_regs`: set of live-out registers including false live-outs (63 bits; non-loop/loop)

(8.2) `pte.live_out_regs |= pb.dest_regs`

- `n_misps`: number of mispredictions for the aggressor branch (16 bits; non-loop/loop)

(8.3) `pte.n_misps += pb.n_misps`

- `n_nt`: number of times aggressor branch resolved as not-taken (10 bits; non-loop only)

(8.4) `if (!pb.dir) pte.n_nt++`

- `n_insts_nt/n_brs_nt`: maximum number of instructions/branches retired on the not-taken path (6+6 bits; non-loop only)

(8.5.1) `if (!pb.dir) pte.n_insts_nt = max(pte.n_insts_nt, pb.n_insts)`

(8.5.2) `if (!pb.dir) pte.n_brs_nt = max(pte.n_brs_nt, pb.n_brs)`

⁸On a miss, each component is first reset, and then updated using the same logic.

- `n_insts_t/n_brs_t`: maximum number of instructions/branches retired on the taken path (6+6 bits; non-loop only)

(8.6.1) `if (pb.dir) pte.n_insts_t = max(pte.n_insts_t, pb.n_insts)`

(8.6.2) `if (pb.dir) pte.n_brs_t = max(pte.n_brs_t, pb.n_brs)`

- `sum_tc`: sum of trip counts (16 bits; loop only)

(8.7) `pte.sum_tc += pb.tc`

- `sum_tc_sq`: sum of trip count squares (22 bits; loop only)

(8.7) `pte.sum_tc_sq += (pb.tc * pb.tc)`

- `n_inst/n_br`: maximum number of instructions/branches retired in an iteration (6+6 bits; loop only)

(8.8.1) `pte.n_insts_iter = max(pte.n_insts_iter, pb.n_insts_iter)`

(8.8.2) `pte.n_brs_iter = max(pte.n_brs_iter, pb.n_brs_iter)`

The profiling table is indexed and tagged using aggressor PC. In a 32-entry 4-way set-associative table, each entry requires a 59-bit tag and a 2-bit recency-info. So the cost of each entry, including a valid bit, a 59-bit tag, and 2-bit recency-info, is 201 bits (185 bits: non-loop; 201 bits: loop). And the cost of the profiling table is 6,432 bits (804 B).

Online stage-1 (delinquency-based) and stage-2 (analytical-model-based) checks

A trained aggressor is first checked for delinquency. It passes the delinquency check if its misprediction rate is above the *misprediction rate threshold*. Note, the misprediction rate for a non-loop aggressor is `(pte.n_misps/pte.n_samples)`, but for a loop aggressor it's `(pte.n_misps/pte.sum_tc)`.

For an aggressor which passes the delinquency check, the processor compares its predication overhead against its speculation overhead using the analytical model. The overheads

for a non-loop/loop aggressor are computed just like their offline counterparts (refer to Sections 5.2.2 and 5.3.3 for related discussions on non-loop and loop aggressors, respectively). Note, the standard deviation (`sigma`) for a loop is computed using sum of trip counts (`pte.sum_tc`) and sum of trip count squares (`pte.sum_tc_sq`) [std].

Aggressors which pass the analytical-model check are added to the predicated branch table. Note, for a loop aggressor, the model also provides the number of iterations to predicate for (`pred_iters`).

Re-training to adjust for phase changes

An aggressor which is profitable-to-predicate during a program phase, might not remain so in a subsequent phase. To accommodate for such a phase-based behavior, the profiling table and the predicated branch table are flushed after a fixed number of retired instructions, determined by a *big switch interval* (default: ~10 million instructions).

5.5.2 Core pipeline changes for dynamic predication

Instruction fetch

The predicated branch table is accessed using instruction PCs in parallel with the branch predictor and BTB. On a hit, the processor switches to dynamic predication mode (dpred-mode).

During the switch, `bhr.nt/bhr.t` are first forked from `bhr.m`, and then updated with a not-taken/taken bit, respectively (refer to Section 5.2.3 for more information). `Pc.nt`, `pc.t`, and `pc.m` are initialized using the branch’s not-taken PC, taken PC, and merge PC, respectively. For a predicated loop branch, the merge PC is the same as the not-taken PC if its loop-exit direction is 0, or the same as the taken PC otherwise. For a merge-via-return type branch, the RAS is popped during the switch, and `pc.m` is initialized using the popped return address. Predicated-path return instructions do not pop the RAS: they are predicted using the return address popped by the parent predicated branch.

In dpred-mode, for a non-loop predicated branch, the processor first fetches the not-taken path to completion before fetching the taken path. A predicated path’s fetch is considered complete upon re-convergence. The re-convergence detection logic at fetch is the same as that at retire (discussed in Section 5.5.1). The predicated path’s fetch is also considered complete if the processor fetches the exit instruction⁹. The fetch unit uses the exit table to know which instructions are exit instructions because otherwise such marking is only available at decode. The fetch of an exit instruction is a control-flow divergence which is handled at the retire stage along with other divergences. When the fetch for the taken path is complete, the processor exits dpred-mode.

A loop branch is predicated for a predetermined number of iterations (available as `pred_iters` in the predicated branch table). In dpred-mode, branch prediction for the loop branch is overridden – all fetched loop branches except the last one are predicted to stay within the loop (and they are predicated) whereas the last one is predicted to exit the loop (and it is not predicated). Except for this change, loop predication in this work is the same as DMP’s loop predication scheme (covered in Section 5.3.1).

Predicate registers are managed using a ROB-like queue, called the predicate queue. When a predicated branch is fetched, it is allocated an entry in the predicate queue, at its tail. The predicated branch’s entry number is used as its predicate register tag. The entry at the head of the predicate queue is reclaimed after the oldest predicated branch’s predicated-path instructions, including select micro-ops (if any), are retired. Fetch is stalled if the fetched bundle contains a predicated branch and the predicate queue is full.

Every instruction in the pipeline has a predicate dependency which has two parts: a predicate register identifier and a predicate direction. The predicate dependency is set at fetch, and is used to squash (silence) predicated-path instructions in the pipeline when a predicated branch resolves. Main-path (un-predicated-path) instructions are identified with a special predicate register identifier, and this identifier is never used by a predicated

⁹The exit instruction is included in the predicated path.

branch.

Predicated branch table, exit table, and predicate queue

An entry in the predicated branch table contains the following information:

- **is_loop**: whether it's a loop branch (1 bit; non-loop/loop)
- **live_out_regs**: register live-outs (63 bits; non-loop/loop)
- **merge_via_return**: whether it's a merge-via-return type branch (1 bit; non-loop only)
- **merge_pc**: merge PC (if applicable; 62 bits; non-loop only)
- **loop_exit_dir**: loop-exit direction (1 bit; loop only)
- **pred_iters**: number of predicated loop iterations (4 bits; loop only)

In a 32-entry 4-way set-associative predicated branch table, each entry requires a 59-bit tag and a 2-bit recency-info. So, the cost of each entry, including a valid bit, a 59-bit tag, and a 2-bit recency-info, is 189 bits (189 bits: non-loop; 131 bits: loop). And the cost of the predicated branch table is 6,048 bits (756 B).

The exit table is a BTB-like structure for exit instructions, indexed and tagged using instruction PC, and trained at decode. The pipeline performs a mini-flush if the decoded exit instruction is not cached in the table or if the decoded non-exit instruction is wrongly-cached in the table.

For a 16-entry 4-way set-associative exit table, each entry requires a 60-bit tag and a 2-bit recency-info. So, the cost of each entry including a valid bit, a 60-bit tag, and a 2-bit recency-info is 63 bits. And the total cost of the exit table is 1,008 bits (126 B).

Besides predicate register management, the predicate queue is also used for: a) injecting select micro-ops at decode and detecting a register divergence at retire, and b) pseudo-

resolving loop predicates in case of loop over-iterations. An entry in the predicate queue has the following fields:

- **live_out_regs**: the set of live-out registers initialized using the predicated branch table entry (63 bits; non-loop/loop)
- **nested_loop_predicate**: this field is set for all predicated loop branches in a loop visit except for the first one (1 bit; non-loop/loop)

A 64-entry predicate queue has a cost of 4,096 bits (512 B).

Branch prediction and recovery for un-predicated branches

Conventional superscalars use checkpoints for instant branch misprediction recovery, *i.e.*, recovery from the middle of the ROB. As predicated branches subsume mispredictions, they do not require a checkpoint. But other branches, such as un-predicated branches on the main/un-predicated path and un-predicated branches on a predicated path, still require a checkpoint.

In our implementation, the pipeline allocates checkpoints to un-predicated branches on the main/un-predicated path as usual. Checkpoints are allocated in-order and freed out-of-order as checkpointed branches resolve [Yea96]. All instructions, irrespective of their predication status, inherit a checkpointed-branch mask (indicating all older checkpointed branches that are not yet resolved). The instructions' masks are used to squash them selectively in the back-end when a checkpointed branch resolves as mispredicted. On a misprediction, apart from recovering the conventional structures, a checkpoint also recovers the predicate queue (its checkpointed tail pointer is restored).

On the other hand, the pipeline does not allocate checkpoints to un-predicated branches on a predicated path. If mispredicted, these branches initiate recovery from the head of the ROB (when they retire). We chose not to allocate checkpoints to these branches because

we do not expect them to have a high misprediction rate. Recall, we disallow a hard-to-predict branch to encapsulate other hard-to-predict branches during software profiling (see Sections 5.2.1 and 5.3.3 for related discussions). Moreover, the choice allows us to avoid unnecessary modifications to the existing checkpoint-management hardware.

Injecting select micro-ops into the fetch queue (at decode)

When a predicated branch is decoded, it initializes a *micro-op injection register* using its set of register live-outs (live-outs are available in the branch’s predicate queue entry). The register is used to prepare and inject select micro-op instructions, one for each live-out, after the last predicated-path instruction (and before the first un-predicated-path instruction), into the fetch queue. The same register is also used to inject select micro-ops after every nested predicated loop branch (nesting information is also available in the branch’s predicate queue entry).

Synchronizing memory live-outs in the issue queue

Just like select micro-ops, predicated-path stores are forced to include the parent branch’s predicate as their third source operand. So a predicated-not-taken/predicated-taken path store can issue only after its parent branch predicate has resolved as not-taken/taken ¹⁰.

As memory live-outs are synchronized in the issue queue, the load-store queue doesn’t need any special handling related to memory synchronization.

Over-iteration, exact-iteration, and under-iteration of a predicated loop

If a predicated loop is over-iterated, a predicated loop branch resolves as mispredicted (predicted-dir: loop-continue-dir, resolved-dir: loop-exit-dir). In this case, the pipeline needs to detect and force-resolve predicated loop branches younger than the mispredicted loop branch in the same loop as loop-exit [Kim+06].

¹⁰A predicated-not-taken/predicated-taken path store is squashed if its parent branch resolves as taken/not-taken (just like other predicated-false instructions).

The pipeline obtains the predicates for the loop branches younger than the mispredicted loop branch in the same loop by searching the predicate queue forward, starting from the index next to the mispredicted loop branch and stopping at the index whose `nested_loop_predicate` bit is false.

If a predicated loop is exact-iterated, there is no concern as all related loop branches resolve as correctly predicted.

In the under-iteration case, the last predicated-path loop branch (which is itself not predicated), resolves as mispredicted (predicted-dir: loop-exit-dir, resolved-dir: loop-continue-dir). Since no checkpoint is allocated to this branch, it initiates a squash from the head of the ROB at retirement.

Retirement

With predication, predicated-true instructions are retired as usual, and predicated-false instructions are pseudo-retired, in which case the pipeline recycles their load/store/physical register resources.

The retire stage detects and handles control-flow and register divergences only on a predicated-true path. This is because divergences on the predicated-false path can be safely ignored.

The retire stage detects and handles the following control-flow divergences:

- *An exit instruction was fetched on the predicated-path before re-convergence:* If the processor retires an exit instruction on a predicated path, it initiates a full squash. Fetch resumes from the instruction after the exit instruction.
- *A predicated loop was under-iterated:* As discussed previously, when a predicated loop is under-iterated, the last loop branch resolves as mispredicted. As no checkpoint is allocated to it, it initiates a full squash upon retirement. Fetch resumes from the instruction at the loop-continue target.

- *A predicated-path return instruction was mispredicted:* This divergence may occur for a merge-via-return type predicated branch. As no checkpoint is allocated to it, it initiates a full squash upon retirement. Fetch resumes from the instruction at the resolved target of the return instruction.

The set of live-out registers for an aggressor in the predicated branch table might be incomplete. This situation can occur if some control-dependent paths were missed during its training, and a missed path had at least one unique live-out register. Predicating a branch using an incomplete set of live-out registers can lead to a register divergence.

Whenever a predicated path is retired, the set of live-out registers used at fetch (available as `live_out_regs` in the predicate queue) is compared against the set of destination logical registers used on the retired predicated path (available as `dest_regs` in the profiling buffer). A register divergence is detected if the register set obtained at retire is not a subset of the register set used at fetch.

The processor recovers from a register divergence by initiating a flush. Fetch resumes from the merge point. Additionally, the live-out register set in the predicated branch table is updated: it is OR-ed with the register set obtained at retire.

5.5.3 Controlled predication using Dynamo

Predicated execution can hurt performance due to unwanted front-end or back-end pressure. Until now we focused only on front-end pressure, and minimized it using software and hardware profiling. We now look at sources of back-end pressure, and also at solutions to mitigate unwanted back-end pressure.

In predicated execution, on average, instructions spend more time in the issue queue due to delayed forwarding of live-out values. This pressure coupled with reduced instruction-level parallelism may result in a slowdown when predication is enabled.

Predication also puts more pressure on the ROB (and the physical register file) as a) the resources held by predicated-false instructions are released only when they are due

for retirement (and not when the predicate resolves), and b) every in-flight select micro-op consumes a ROB slot. Like issue-queue pressure, pressure on the ROB can also be deleterious for predicated execution.

A review of Dynamo

To limit negative effects of predication, the authors of ACB proposed Dynamo. Dynamo uses five states for each predicated branch: BAD:0, LIKELY-BAD:1, NEUTRAL:2, LIKELY-GOOD:3, and GOOD:4. Predicated branches start in the NEUTRAL state, and they are gradually trained to one of the two stable states, GOOD or BAD, based on their influence on performance with and without predication. Performance monitoring and training occurs in two alternating phases/epochs: odd-epoch and even-epoch. Each epoch has a fixed number of retired instructions, determined by a *small switch interval* (default: 16,384 instructions).

The following procedure describes the Dynamo’s training algorithm [Cha+20]:

1. For each predicated branch, set its state to NEUTRAL. Assume the procedure begins in odd-epoch.
2. In this epoch (odd-epoch), allow predication only on branches in the GOOD state. During the epoch, measure the number of instructions retired and the number of cycles elapsed (`c_odd`).
3. When the number of instructions retired reaches the *small switch interval*, transition to the next epoch.
4. In this epoch (even-epoch), allow predication on branches which are not in the BAD state. During the epoch, measure the number of instructions retired and the number of cycles elapsed (`c_even`). Additionally, for each untrained branch, *i.e.*, a branch not in the GOOD or the BAD state, measure its retirement count using an *involvement counter*.

5. When the number of instructions retired reaches the *small switch interval*, evaluate how predication of untrained branches impacted performance as follows.

(9.1) `pred_perf_pos = (((c_odd - c_even) / c_odd) > noise_threshold)`

(9.2) `pred_perf_neg = (((c_even - c_odd) / c_odd) > noise_threshold)`

where `pred_perf_pos` tells if the performance improved with predication of untrained branches, `pred_perf_neg` tells if the performance degraded with predication of untrained branches, and `noise_threshold` is the *noise threshold* (default: 0.125).

If `pred_perf_pos` is true, do the following. For each untrained branch, increment its state (move it one step closer to GOOD) if its involvement counter is above the *involvement threshold* (default: 15).

If `pred_perf_neg` is true, do the following. For each untrained branch, decrement its state (move it one step closer to BAD) if its involvement counter is above the *involvement threshold*.

Transition to the next epoch. Go to step:2.

Adding Dynamo to the microarchitecture

In our implementation, we added Dynamo as a separate component (see Figure 5.6) with an unbounded size. Whenever a branch is added to the predicated branch table, it is also added to Dynamo.

At fetch, the predicated branch table and Dynamo are accessed in parallel. On a hit in the predicated branch table, predication is applied to the corresponding branch only if its state is GOOD/not-BAD in the odd-/even-epoch, respectively.

Like the profiling table and the predicated branch table, Dynamo is also reset after a *big switch interval*.

5.5.4 Overall storage cost

Table 5.3 shows the cost accounting of new structures added to the pipeline. We describe Dynamo as a separate component, but it can be integrated into the predicated branch table, in which case each predicated branch table entry needs 7 additional bits (3:state, 4:involvement-counter). The cost accounting of Dynamo assumes this combined organization. It adds 28 B ($32 * 7$ bits) to the storage cost.

Table 5.3: Cost accounting.

Component	Cost
Profiling buffer	26.125 B
Profiling table	804 B
Predicated branch table	756 B
Exit table	126 B
Predicate queue	512 B
Dynamo*	28 B
Total	2.20 KB

5.6 Methodology

We evaluate dynamic predication using an in-house, RISC-V, execution-driven, execute-at-execute superscalar processor simulator. Default parameters of the baseline superscalar processor are shown in Table 5.4.

We compiled the SPEC 2006 and SPEC 2017 integer benchmarks and GAPBS benchmarks [BAP15] using LLVM (repository: [LLVc], branch: release/16.x, commit: 464bda7, optimization flag: -O3). The SPEC 2017 benchmark, exchange_2, is not available as it has Fortran source, which our LLVM RISC-V compiler cannot compile.

For each SPEC benchmark, we use its ref input that has the highest weighted-average MPKI over all its SimPoints. For GAPBS benchmarks, we use three real-world input

Table 5.4: Default parameters of the baseline superscalar processor.

Parameter	Value
fetch-to-execute depth	12 stages
fetch/dispatch/issue/retire widths	8/8/8/8
execution lanes	2 ld/st, 4 simple ALU, 2 complex/fp ALU
ROB/PRF/LQ/SQ/IQ	512/576/256/256/128
branch checkpoints	192
branch predictor	64KB TAGE-SC-L [Sez16]
BTB	8K entries, 4-way
RAS	64 entries
L1 I\$	64KB, 4-way, 64B block
L1 D\$	64KB, 4-way, 64B block, 4-cyc. load-to-use
L2 \$	1MB, 8-way, 64B block, 10 cyc.
L3 \$	8MB, 16-way, 128B block, 30 cyc.
main memory latency	100 cyc.

graphs: Road, Twitter, and Web [BAP15]¹¹. Up to ten 100-million-instruction Sim-Points [She+02] were generated for each benchmark.

Offline profiling was performed using different benchmark inputs (different from those used for evaluation). All train inputs were used for profiling SPEC. Two synthetic graphs (Kronecker and random, both with 2^{19} vertices) and one alternate real-world graph (road-PA [LK14]) were used for profiling GAPBS.

5.7 Results

For four benchmarks, 462.libquantum, 464.h264ref, 483.xalancbmk, and 623.xalancbmk_s, we couldn't obtain any branch after offline stage-1 profiling. We exclude these benchmarks in all our results.

¹¹Sssp benchmark with Web input is excluded in the evaluation as its memory requirement was more than what our machines supported.

5.7.1 Software profiling

Aggressors after stage-1/2/3

Figure 5.7 shows the number of aggressors obtained after offline stage-1, stage-2, and stage-3 processing for each benchmark.

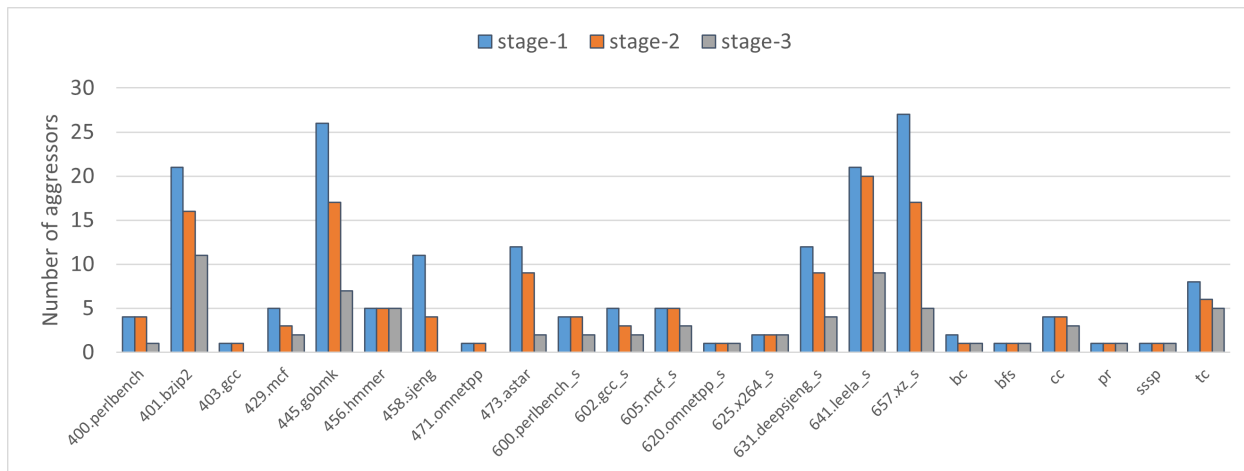


Figure 5.7: Number of aggressors after offline stage-1, stage-2, and stage-3 processing.

In total, there were 180 aggressors after stage-1. These were reduced to 135 after stage-2, and to 68 after stage-3.

Breakdown of aggressor types

Figure 5.8 shows the breakdown of aggressor types after stage-3 processing. Note, there are no merge-via-return type aggressors after stage-3 processing ¹².

5.7.2 Predicating forward-branch aggressors

We now evaluate the performance of our revised DMP-style predication. In this section, predication is a) applied only to forward/non-loop aggressors, and b) not controlled using

¹²In 641.leela_s, there were two merge-via-return type aggressors after stage-1, which were reduced to one after stage-2. The sole aggressor was eliminated after stage-3.

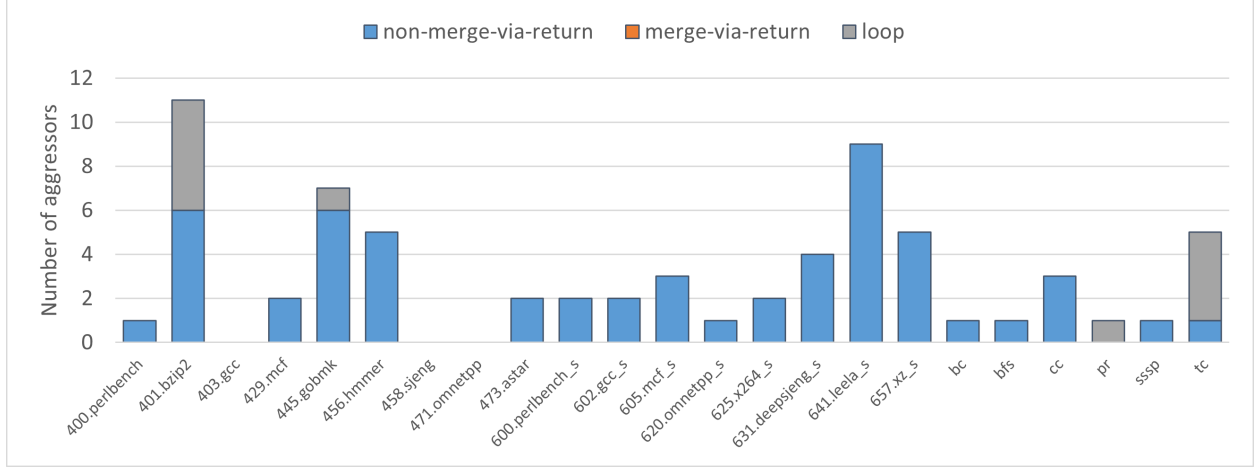


Figure 5.8: Breakdown of aggressor types after stage-3 processing.

Dynamo (Dynamo is always disabled).

On the software side, for each benchmark we consider three different program binaries: a) *OFL1*: binary with aggressors obtained after stage-1, b) *OFL2*: binary with aggressors obtained after stage-1/2, and c) *OFL3*: binary with aggressors obtained after stage-1/2/3.

On the hardware side, we consider two different options: a) *ONL1*: stage-1 check is enabled and stage-2 check is disabled, and b) *ONL2*: stage-1 and stage-2 checks are enabled.

We evaluate results on three different software-hardware configurations, *OFL1+ONL1*, *OFL2+ONL2*, and *OFL3+ONL2*. Figure 5.9 shows speedup of the three configurations over baseline.

The overall speedup with offline/online stage-1 (*OFL1+ONL1*) is -0.87% (a slowdown). The speedup improves to 1.81% after adding an offline/online stage-2 (*OFL2+ONL2*). Finally, after adding an offline stage-3 (*OFL3+ONL2*) the speedup improves to 2.17%.

Figures 5.10 and 5.11 show benchmark-wise speedup for the three configurations for SPEC and GAP, respectively. The benchmarks appear in the order of increasing *OFL3+ONL2* speedup.

In SPEC, adding an offline/online stage-2 was beneficial for 445.gobmk and 641.leela, and slightly detrimental for 473.astar. And adding an offline stage-3 was beneficial for

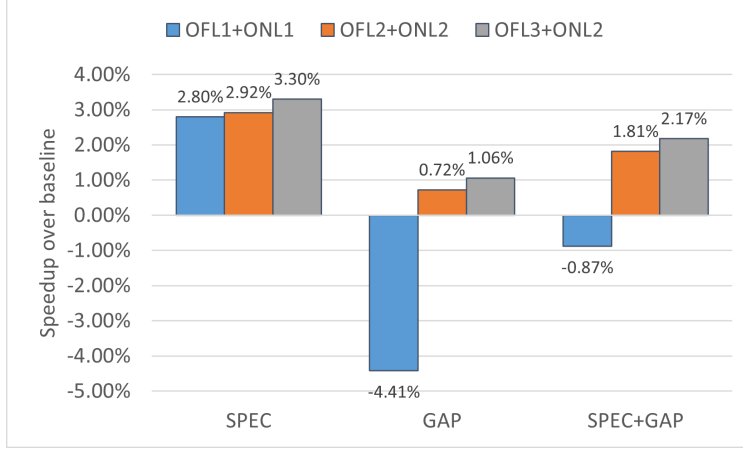


Figure 5.9: Overall speedup when forward branches are predicated.

445.gobmk, 458.sjeng, 471.omnetpp, and 641.leela, and slightly detrimental for 605.mcf.

In GAP, adding an offline/online stage-2 was beneficial for sssp-twitter, bc-twitter, bc-web, bfs-web, and bc-road. And adding an offline stage-3 was beneficial for cc-web.

5.7.3 Enabling Dynamo

We now enable Dynamo and evaluate its performance. Figure 5.12 shows the speedup when Dynamo is enabled ($OFL3 + ONL2 + DYN$). GAP benchmarks see a significant boost with Dynamo, with their speedup going from 1.06% without Dynamo to 3.93% with Dynamo. For all benchmarks, the speedup goes from 2.17% to 3.76%.

Figures 5.13 and 5.14 show the benchmark-wide speedups when Dynamo is enabled for SPEC and GAP benchmarks, respectively. The benchmarks appear in the order of increasing $OFL3 + ONL2 + DYN$ speedup.

Dynamo effectively eliminated slowdowns in 657.xz and 605.mcf, and improved the speedup for 429.mcf. Note, the increase in 429.mcf’s speedup also stems from an eliminated slowdown in one of its SimPoints. Dynamo reduced speedups in 473.astar, 641.leela, and 625.x264. Perhaps the aggressor branches in these benchmarks are profitable-to-predicate, but they take many iterations to cross the performance threshold and reach the GOOD

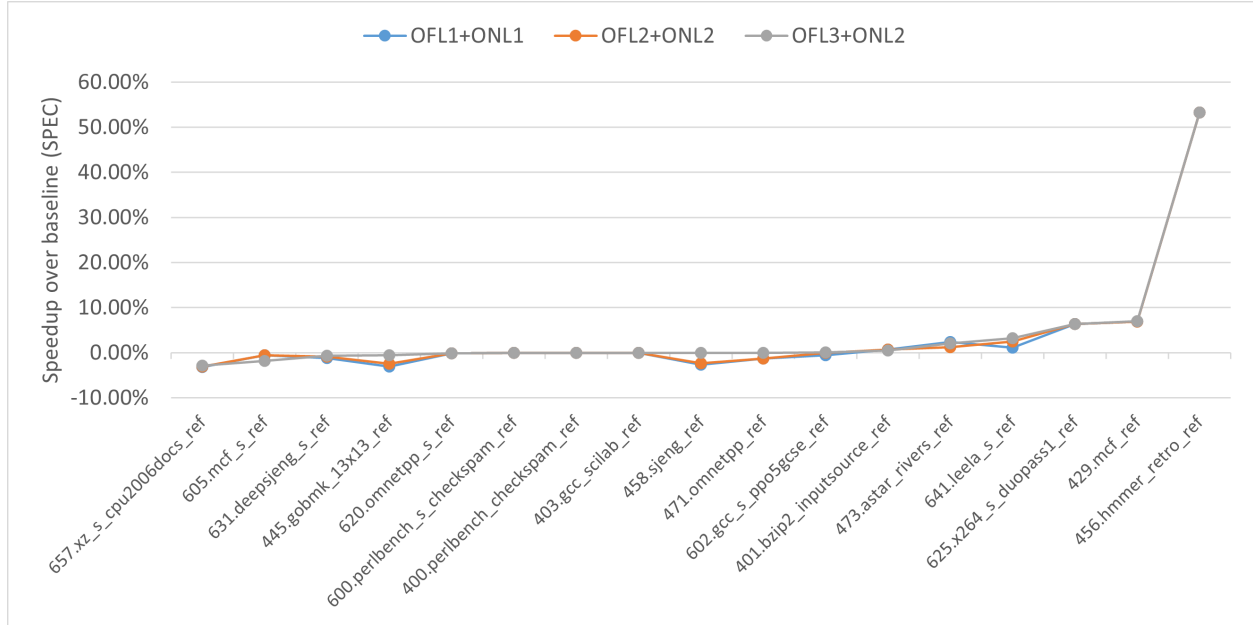


Figure 5.10: Percentage speedup of SPEC benchmarks when forward branches are predicated.

state. And until they reach the GOOD state, they get throttled by Dynamo.

Dynamo shined on GAP benchmarks. It was able to effectively eliminate slowdowns for cc-web, sssp-twitter, bc-web, bc-twitter, bfs-web, bfs-twitter, and cc-road.

5.7.4 Including loop aggressors

Finally, we evaluate performance of loop predication. Loop predication improved the overall performance from 3.76% to 4.34%. Almost all of this increase comes from tc-road. Tc-road shows no speedup without loop predication, and shows a 20.72% speedup with loop predication.

5.8 Future work

- *Poly-path predication*: In many scenarios, a hard-to-predict branch is nested within other hard-to-predict branch(es). This work and all prior works are limited to dual-

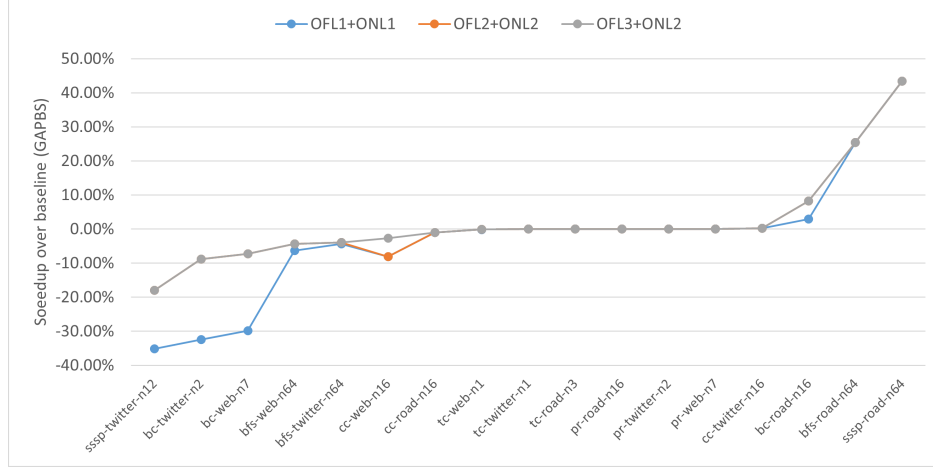


Figure 5.11: Percentage speedup of GAP benchmarks when forward branches are predicated.

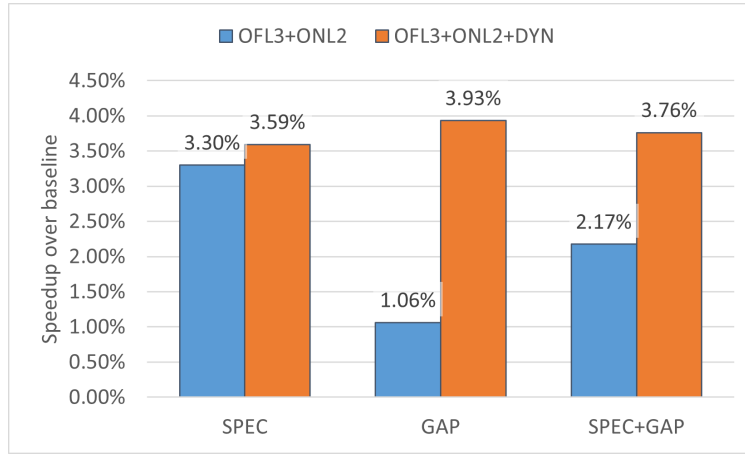


Figure 5.12: Overall speedup when Dynamo is enabled.

path predication, which means only the innermost branch can be predicated effectively in such cases.

The poly-path architecture [KPG98] can be used as a substrate for poly-path predication. Poly-path predication can improve coverage beyond what dual-path predication can achieve. It should also improve performance of loop predication by parallelizing loop branch resolution, which otherwise is serial in dual-path predication.

- *Annotate true live-outs in the binary*: This work and all prior works use a conservative set of live-out registers to synchronize control-independent instructions. The

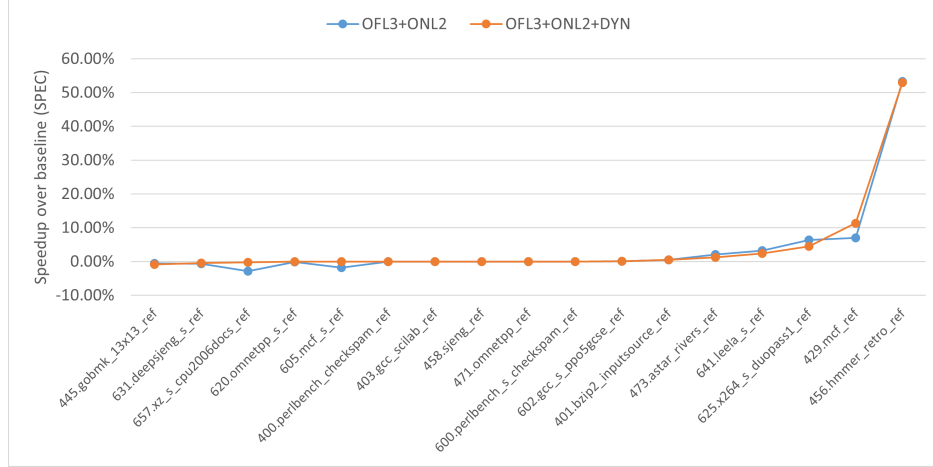


Figure 5.13: Percentage speedup of SPEC benchmarks when Dynamo is enabled.

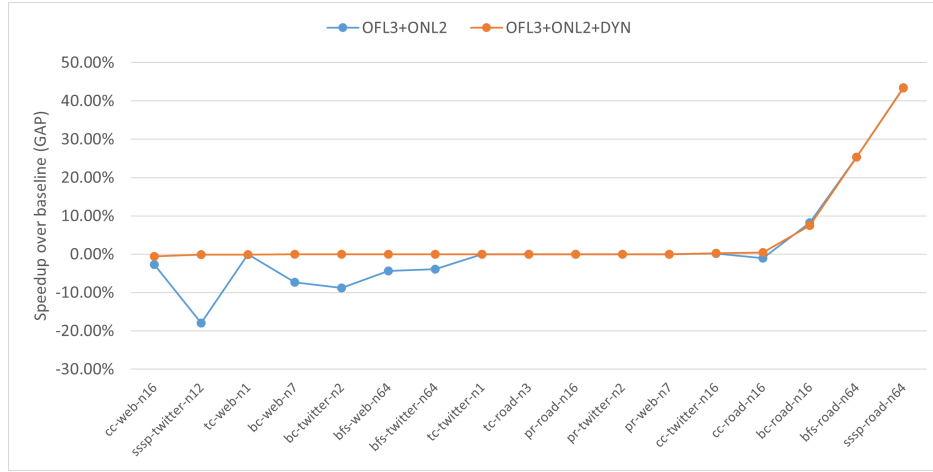


Figure 5.14: Percentage speedup of GAP benchmarks when Dynamo is enabled.

set includes all unique logical registers produced in the control-dependent region of a branch. But for many branches, the real live-out set is much smaller.

In our benchmarks, we found, on average, that the conservative live-out set contains 2.25 registers per aggressor branch ¹³. In contrast, the true live-out set contains only 1.25 registers per aggressor branch. Although extra live-outs do not worsen the critical path directly, they put pressure on the issue queue, ROB, and physical register file, as well as issue slots.

¹³The data is collected using the 68 aggressor branches obtained after offline stage-3 processing.

An instruction which produces a true live-out for an aggressor can be marked in the binary. The reserved opcode space (`inst[1:0] == 2'b00`) can be used for this purpose.

References

- [Sta73] Wayne Stahnke. “Primitive Binary Polynomials.” In: *Mathematics of Computation* 27.124 (1973), pp. 977–980.
- [SBV95] Gurindar S. Sohi, Scott E. Breach, and T. N. Vijaykumar. “Multiscalar processors.” In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture*. ISCA '95. S. Margherita Ligure, Italy: Association for Computing Machinery, 1995, pp. 414–425. ISBN: 0897916980. DOI: 10.1145/223982.224451. URL: <https://doi.org/10.1145/223982.224451>.
- [USH95] A.K. Uht, V. Sindagi, and K. Hall. “Disjoint eager execution: an optimal form of speculative execution.” In: *Proceedings of the 28th Annual International Symposium on Microarchitecture*. 1995, pp. 313–325. DOI: 10.1109/MICRO.1995.476841.
- [BND96] J.O. Bondi, A.K. Nanda, and S. Dutta. “Integrating a misprediction recovery cache (MRC) into a superscalar pipeline.” In: *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO 29*. 1996, pp. 14–23. DOI: 10.1109/MICRO.1996.566446.
- [HS96] Timothy H. Heil and James E. Smith. *Selective Dual Path Execution*. Tech. rep. University of Wisconsin – Madison, 1996. URL: <https://jes.ece.wisc.edu/papers/sdpe.pdf>.
- [JRS96] Erik Jacobsen, Eric Rotenberg, and J. E. Smith. “Assigning confidence to conditional branch predictions.” In: *Proceedings of the 29th Annual ACM/IEEE International Symposium on Microarchitecture. MICRO 29*. Paris, France: IEEE Computer Society, 1996, pp. 142–152. ISBN: 0818676418.
- [LS96] M.H. Lipasti and J.P. Shen. “Exceeding the dataflow limit via value prediction.” In: *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO 29*. 1996, pp. 226–237. DOI: 10.1109/MICRO.1996.566464.
- [Yea96] K.C. Yeager. “The Mips R10000 superscalar microprocessor.” In: *IEEE Micro* 16.2 (1996), pp. 28–41. DOI: 10.1109/40.491460.
- [SS97a] Yiannakis Sazeides and James E. Smith. “The predictability of data values.” In: *Proceedings of the 30th Annual ACM/IEEE International Symposium on Microarchitecture. MICRO 30*. Research Triangle Park, North Carolina, USA: IEEE Computer Society, 1997, pp. 248–258. ISBN: 0818679778.

- [SS97b] A. Sodani and G.S. Sohi. “Dynamic Instruction Reuse.” In: *Conference Proceedings. The 24th Annual International Symposium on Computer Architecture*. 1997, pp. 194–205. DOI: 10.1145/264107.264200.
- [Ahu+98] Pritpal S. Ahuja, Kevin Skadron, Margaret Martonosi, and Douglas W. Clark. “Multipath execution: opportunities and limits.” In: *Proceedings of the 12th International Conference on Supercomputing*. ICS ’98. Melbourne, Australia: Association for Computing Machinery, 1998, pp. 101–108. ISBN: 089791998X. DOI: 10.1145/277830.277854. URL: <https://doi.org/10.1145/277830.277854>.
- [AD98] H. Akkary and M.A. Driscoll. “A dynamic multithreading processor.” In: *Proceedings. 31st Annual ACM/IEEE International Symposium on Microarchitecture*. 1998, pp. 226–236. DOI: 10.1109/MICRO.1998.742784.
- [Kla+98] A. Klauser, T. Austin, D. Grunwald, and B. Calder. “Dynamic hammock predication for non-predicated instruction set architectures.” In: *Proceedings. 1998 International Conference on Parallel Architectures and Compilation Techniques (Cat. No.98EX192)*. 1998, pp. 278–285. DOI: 10.1109/PACT.1998.727261.
- [KPG98] Artur Klauser, Abhijit Paithankar, and Dirk Grunwald. “Selective eager execution on the PolyPath architecture.” In: *Proceedings of the 25th Annual International Symposium on Computer Architecture*. ISCA ’98. Barcelona, Spain: IEEE Computer Society, 1998, pp. 250–259. ISBN: 0818684917. DOI: 10.1145/279358.279393. URL: <https://doi.org/10.1145/279358.279393>.
- [VS98] T.N. Vijaykumar and G.S. Sohi. “Task selection for a multiscalar processor.” In: *Proceedings. 31st Annual ACM/IEEE International Symposium on Microarchitecture*. 1998, pp. 81–92. DOI: 10.1109/MICRO.1998.742771.
- [WCT98] Steven Wallace, Brad Calder, and Dean M. Tullsen. “Threaded multiple path execution.” In: *Proceedings of the 25th Annual International Symposium on Computer Architecture*. ISCA ’98. Barcelona, Spain: IEEE Computer Society, 1998, pp. 238–249. ISBN: 0818684917. DOI: 10.1145/279358.279392. URL: <https://doi.org/10.1145/279358.279392>.
- [Cha+99] R. S. Chappell, J. Stark, S. P. Kim, S. K. Reinhardt, and Y. N. Patt. “Simultaneous subordinate microthreading (SSMT).” In: *Proceedings of the 26th International Symposium on Computer Architecture*. May 1999, pp. 186–195.
- [CFS99] Yuan Chou, Jason Fung, and John Paul Shen. “Reducing branch misprediction penalties via dynamic control independence detection.” In: *Proceedings of the 13th International Conference on Supercomputing*. ICS ’99. Rhodes, Greece: Association for Computing Machinery, 1999, pp. 109–118. ISBN: 158113164X.

DOI: 10.1145/305138.305175. URL: <https://doi.org/10.1145/305138.305175>.

- [RJS99] E. Rotenberg, Q. Jacobson, and J. Smith. “A study of control independence in superscalar processors.” In: *Proceedings Fifth International Symposium on High-Performance Computer Architecture*. 1999, pp. 115–124. DOI: 10.1109/HPCA.1999.744346.
- [RS99] E. Rotenberg and J. Smith. “Control independence in trace processors.” In: *MICRO-32. Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture*. 1999, pp. 4–15. DOI: 10.1109/MICRO.1999.809438.
- [WTC99] S. Wallace, D.M. Tullsen, and B. Calder. “Instruction recycling on a multiple-path processor.” In: *Proceedings Fifth International Symposium on High-Performance Computer Architecture*. 1999, pp. 44–53. DOI: 10.1109/HPCA.1999.744323.
- [PSR00] Zach Purser, Karthik Sundaramoorthy, and Eric Rotenberg. “A study of slipstream processors.” In: *Proceedings of the 33rd Annual ACM/IEEE International Symposium on Microarchitecture*. MICRO 33. Monterey, California, USA: Association for Computing Machinery, 2000, pp. 269–280. ISBN: 1581131968. DOI: 10.1145/360128.360155. URL: <https://doi.org/10.1145/360128.360155>.
- [RS00] A. Roth and G.S. Sohi. “Register integration: a simple and efficient implementation of squash reuse.” In: *Proceedings 33rd Annual IEEE/ACM International Symposium on Microarchitecture. MICRO-33 2000*. 2000, pp. 223–234. DOI: 10.1109/MICRO.2000.898073.
- [SPR00] Karthik Sundaramoorthy, Zach Purser, and Eric Rotenburg. “Slipstream processors: improving both performance and fault tolerance.” In: *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems. ASPLOS IX*. Cambridge, Massachusetts, USA: Association for Computing Machinery, 2000, pp. 257–268. ISBN: 1581133170. DOI: 10.1145/378993.379247. URL: <https://doi.org/10.1145/378993.379247>.
- [CV01] Chen-Yong Cher and T.N. Vijaykumar. “Skipper: a microarchitecture for exploiting control-flow independence.” In: *Proceedings. 34th ACM/IEEE International Symposium on Microarchitecture. MICRO-34*. 2001, pp. 4–15. DOI: 10.1109/MICRO.2001.991101.
- [ZS01] Craig Zilles and Gurindar Sohi. “Execution-based prediction using speculative slices.” In: *Proceedings of the 28th Annual International Symposium on Computer Architecture. ISCA '01*. Göteborg, Sweden: Association for Computing

Machinery, 2001, pp. 2–13. ISBN: 0769511627. DOI: 10.1145/379240.379246. URL: <https://doi.org/10.1145/379240.379246>.

- [Ara+02] Juan L. Aragón, José González, Antonio González, and James E. Smith. “Dual path instruction processing.” In: *Proceedings of the 16th International Conference on Supercomputing*. ICS ’02. New York, New York, USA: Association for Computing Machinery, 2002, pp. 220–229. ISBN: 1581134835. DOI: 10.1145/514191.514223. URL: <https://doi.org/10.1145/514191.514223>.
- [Cha+02] R. S. Chappell, F. Tseng, A. Yoaz, and Y. N. Patt. “Difficult-path branch prediction using subordinate microthreads.” In: *Proceedings of the 29th International Symposium on Computer Architecture*. May 2002, pp. 307–317.
- [She+02] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. “Automatically characterizing large scale program behavior.” In: *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*. San Jose, California, Oct. 2002, pp. 45–57. ISBN: 1581135742. DOI: 10.1145/605397.605403.
- [ASL03] Haitham Akkary, Srikanth T. Srinivasan, and Konrad Lai. “Recycling waste: exploiting wrong-path execution to improve branch prediction.” In: *Proceedings of the 17th Annual International Conference on Supercomputing*. ICS ’03. San Francisco, CA, USA: Association for Computing Machinery, 2003, pp. 12–21. ISBN: 1581137338. DOI: 10.1145/782814.782819. URL: <https://doi.org/10.1145/782814.782819>.
- [PFV03] Il Park, B. Falsafi, and T.N. Vijaykumar. “Implicitly-multithreaded processors.” In: *30th Annual International Symposium on Computer Architecture, 2003. Proceedings*. 2003, pp. 39–50. DOI: 10.1109/ISCA.2003.1206987.
- [CTW04] J.D. Collins, D.M. Tullsen, and Hong Wang. “Control Flow Optimization Via Dynamic Reconvergence Prediction.” In: *37th International Symposium on Microarchitecture (MICRO-37’04)*. 2004, pp. 129–140. DOI: 10.1109/MICRO.2004.13.
- [GAS04] Amit Gandhi, H. Akkary, and S.T. Srinivasan. “Reducing branch misprediction penalty via selective branch recovery.” In: *10th International Symposium on High Performance Computer Architecture (HPCA’04)*. 2004, pp. 254–264. DOI: 10.1109/HPCA.2004.10004.
- [Kim+06] Hyesoon Kim, Jose A. Joao, Onur Mutlu, and Yale N. Patt. “Diverge-Merge Processor (DMP): Dynamic Predicated Execution of Complex Control-Flow Graphs Based on Frequently Executed Paths.” In: *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’06)*. 2006, pp. 53–64. DOI: 10.1109/MICRO.2006.20.

- [Aga+07] Mayank Agarwal, Kshitiz Malik, Kevin M. Woley, Sam S. Stone, and Matthew I. Frank. “Exploiting Postdominance for Speculative Parallelization.” In: *2007 IEEE 13th International Symposium on High Performance Computer Architecture*. 2007, pp. 295–305. DOI: 10.1109/HPCA.2007.346207.
- [HR07] Andrew D. Hilton and Amir Roth. “Ginger: control independence using tag rewriting.” In: *Proceedings of the 34th Annual International Symposium on Computer Architecture*. ISCA ’07. San Diego, California, USA: Association for Computing Machinery, 2007, pp. 436–447. ISBN: 9781595937063. DOI: 10.1145/1250662.1250716. URL: <https://doi.org/10.1145/1250662.1250716>.
- [Kim+07] Hyesoon Kim, Jose A. Joao, Onur Mutlu, and Yale N. Patt. “Profile-assisted Compiler Support for Dynamic Predication in Diverge-Merge Processors.” In: *International Symposium on Code Generation and Optimization (CGO’07)*. 2007, pp. 367–378. DOI: 10.1109/CGO.2007.31.
- [Al+07] Ahmed S. Al-Zawawi, Vimal K. Reddy, Eric Rotenberg, and Haitham H. Akkary. “Transparent control independence (TCI).” In: *Proceedings of the 34th Annual International Symposium on Computer Architecture*. ISCA ’07. San Diego, California, USA: Association for Computing Machinery, 2007, pp. 448–459. ISBN: 9781595937063. DOI: 10.1145/1250662.1250717. URL: <https://doi.org/10.1145/1250662.1250717>.
- [GH08] Alok Garg and Michael C. Huang. “A performance-correctness explicitly-decoupled architecture.” In: *2008 41st IEEE/ACM International Symposium on Microarchitecture*. 2008, pp. 306–317. DOI: 10.1109/MICRO.2008.4771800.
- [Mal+08] Kshitiz Malik, Mayank Agarwal, Sam S. Stone, Kevin M. Woley, and Matthew I. Frank. “Branch-mispredict level parallelism (BLP) for control independence.” In: *2008 IEEE 14th International Symposium on High Performance Computer Architecture*. 2008, pp. 62–73. DOI: 10.1109/HPCA.2008.4658628.
- [PS12] Nathanael Premillieu and Andre Sez nec. “SYRANT: SYmmetric resource allocation on not-taken and taken paths.” In: *ACM Trans. Archit. Code Optim.* 8.4 (Jan. 2012). ISSN: 1544-3566. DOI: 10.1145/2086696.2086722. URL: <https://doi.org/10.1145/2086696.2086722>.
- [LK14] Jure Leskovec and Andrej Krevl. *SNAP Datasets: Stanford Large Network Dataset Collection*. June 2014. URL: <http://snap.stanford.edu/data>.
- [BAP15] Scott Beamer, Krste Asanović, and David Patterson. *The GAP Benchmark Suite*. 2015. arXiv: 1508.03619.

- [Gal15] William P. Galliher. “Squashed Branch Reuse.” Available at <http://www.lib.ncsu.edu/resolver/1840.16/11102>. Master’s thesis. North Carolina State University, Mar. 2015.
- [Sez16] André Seznec. “TAGE-SC-L Branch Predictors Again.” In: *5th JILP Workshop on Computer Architecture Competitions (JWAC-5): Championship Branch Prediction (CBP-5)*. June 2016.
- [Kot+18] Vignyan Reddy Kothinti Naresh, Rami Sheikh, Arthur Perais, and Harold W. Cain. “SPF: Selective Pipeline Flush.” In: *2018 IEEE 36th International Conference on Computer Design (ICCD)*. 2018, pp. 152–155. DOI: 10.1109/ICCD.2018.00032.
- [KH19] Sushant Kondguli and Michael Huang. “R3-DLA (Reduce, Reuse, Recycle): A More Efficient Approach to Decoupled Look-Ahead Architectures.” In: *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2019, pp. 533–544. DOI: 10.1109/HPCA.2019.00064.
- [Cha+20] Adarsh Chauhan, Jayesh Gaur, Zeev Sperber, Franck Sala, Lihu Rappoport, Adi Yoaz, and Sreenivas Subramoney. “Auto-Predication of Critical Branches.” In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 2020, pp. 92–104. DOI: 10.1109/ISCA45697.2020.00019.
- [SCR20] Vinesh Srinivasan, Rangeen Basu Roy Chowdhury, and Eric Rotenberg. “Slipstream Processors Revisited: Exploiting Branch Sets.” In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 2020, pp. 105–117. DOI: 10.1109/ISCA45697.2020.00020.
- [Eye+21] Stijn Eyerman, Wim Heirman, Sam Van Den Steen, and Ibrahim Hur. “Enabling Branch-Mispredict Level Parallelism by Selectively Flushing Instructions.” In: *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO ’21. Virtual Event, Greece: Association for Computing Machinery, 2021, pp. 767–778. ISBN: 9781450385572. DOI: 10.1145/3466752.3480045. URL: <https://doi.org/10.1145/3466752.3480045>.
- [PP21] Stephen Pruett and Yale Patt. “Branch Runahead: An Alternative to Branch Prediction for Impossible to Predict Branches.” In: *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO ’21. Virtual Event, Greece: Association for Computing Machinery, 2021, pp. 804–815. ISBN: 9781450385572. DOI: 10.1145/3466752.3480053. URL: <https://doi.org/10.1145/3466752.3480053>.
- [RMH21] Julian Rosemann, Simon Moll, and Sebastian Hack. “An abstract interpretation for SPMD divergence on reducible control flow graphs.” In: *Proc. ACM Program. Lang.* 5.POPL (Jan. 2021). DOI: 10.1145/3434312. URL: <https://doi.org/10.1145/3434312>.

- [DCP24a] Aniket Deshmukh, Lingzhe Chester Cai, and Yale N. Patt. “Alternate Path Fetch.” In: *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024, pp. 1217–1229. DOI: 10.1109/ISCA59077.2024.00091.
- [DCP24b] Aniket Deshmukh, Lingzhe Chester Cai, and Yale N. Patt. “Timely, Efficient, and Accurate Branch Precomputation.” In: *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 2024, pp. 480–492. DOI: 10.1109/MICRO61859.2024.00043.
- [Sin+24] Sawan Singh, Arthur Perais, Alexandra Jimborean, and Alberto Ros. “Alternate Path μ -op Cache Prefetching.” In: *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024, pp. 1230–1245. DOI: 10.1109/ISCA59077.2024.00092.
- [KC25] Qingxuan Kang and Trevor E. Carlson. “Multi-Stream Squash Reuse for Control-Independent Processors.” In: *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. MICRO ’25. Association for Computing Machinery, 2025, pp. 504–518. ISBN: 9798400715730. DOI: 10.1145/3725843.3762248. URL: <https://doi.org/10.1145/3725843.3762248>.
- [SR25] Anirudh Seshadri and Eric Rotenberg. “Delinquent Loop Pre-execution Using Predicated Helper Threads.” In: *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2025, pp. 44–58. DOI: 10.1109/HPCA61900.2025.00015.
- [LFS] *Linear-feedback shift register*. https://en.wikipedia.org/wiki/Linear-feedback_shift_register.
- [LLVa] *LLVM Compiler Infrastructure Reference*. <https://llvm.org/docs/ConvergenceAndUniformity.html>.
- [LLVb] *LLVM Compiler Infrastructure User Guides*. <https://llvm.org/docs/LoopTerminology.html>.
- [LLVc] *LLVM This is the LLVM organization on GitHub for the LLVM Project: a collection of modular and reusable compiler and toolchain technologies*. <https://github.com/llvm/llvm-project.git>.
- [spi] *Spike RISC-V ISA Simulator*. <https://github.com/riscv-software-src/riscv-isa-sim>.
- [std] *Standard deviation*. https://en.wikipedia.org/wiki/Standard_deviation.